

**PONTIFICIA UNIVERSIDAD
CATÓLICA DEL PERÚ**

FACULTAD DE CIENCIAS SOCIALES



Impacto de la disponibilidad de ChatGPT en el desarrollo de software orientado
a Ciencia de Datos entre 2020 y 2023

Tesis para obtener el título profesional de Licenciado en Economía presentado
por:

Nicho Rosado, Jesus Alberto

Tesis para obtener el título profesional de Licenciado en Economía presentado
por:

Janampa Aparicio, Karl Willem

Asesor:
Quispe Rojas, Alexander Wilder

Lima, 2026

Agradecimientos

La culminación de este trabajo representa para mí un profundo sentimiento de satisfacción y orgullo. Este logro marca un antes y un después en mi trayectoria académica, constituyéndose en un hito que atesoraré siempre. Deseo expresar mi más sincero agradecimiento a mi madre Magali, mi padre Jesús, mi hermano Mathías y mi abuela Luz, cuyo apoyo incondicional, confianza y constante motivación fueron fundamentales para perseverar frente a las adversidades. Asimismo, guardo un especial recuerdo de todas las personas que formaron parte de este camino en la PUCP (profesores, compañeros y personal administrativo), quienes contribuyeron a que esta etapa estuviera llena de aprendizajes, desafíos y valiosas oportunidades. Finalmente, agradezco de manera especial a mi gran amigo Karl y a nuestro asesor, el profesor Alexander Quispe, con quienes conformé un excelente equipo de trabajo para la elaboración de este producto final.

Jesus Nicho

Agradezco a todas las personas que hicieron posible la culminación de esta tesis, la cual representa un hito en mi crecimiento personal y profesional, al constituir una etapa transformadora que fortaleció mi compromiso, el aprendizaje en el área, la experiencia en investigación, el trabajo colaborativo y mi perspectiva sobre los inicios de la vida laboral y creativa. Valoro el respaldo incondicional de mis padres, Selene y Kleber, cuya confianza, formación en valores y perseverancia han sido fundamentales en mi desarrollo, así como la motivación constante de mis hermanos, quienes confiaron en mí incluso en momentos de duda. Reconozco también la guía de mi asesor, Alexander, por su orientación académica, compromiso y motivación, además del aporte de mi compañero Jesús, quien enriqueció mi conocimiento y amplió mi visión hacia la innovación. Finalmente, destaco el apoyo firme y constante de Camila durante este proceso. Concluir esta etapa cierra un ciclo significativo y refleja una versión más sólida de mí mismo, constituyendo un motivo de orgullo personal.

Karl Janampa

Resumen

En los últimos años, la Inteligencia Artificial ha comenzado a transformar de manera significativa la producción de software y la forma en que los desarrolladores interactúan con herramientas de programación. En particular, la aparición de ChatGPT ha proporcionado a los programadores un nuevo recurso para el análisis, la generación y la depuración de código, así como para la resolución de problemas técnicos y la aceleración de procesos de aprendizaje. Este tipo de herramientas, basadas en modelos de lenguaje de gran escala, permite sugerir fragmentos de código, explicar funciones complejas y asistir en la documentación de proyectos, lo cual puede reducir los tiempos de desarrollo y mejorar la productividad de los equipos de programación. En este contexto, el objetivo del presente estudio es evaluar el impacto de ChatGPT en la producción de software durante el periodo comprendido entre el primer trimestre de 2020 y el cuarto trimestre de 2023. Para ello, se utiliza la base de datos GitHub Innovation Graph, que proporciona información agregada sobre actividad de desarrollo a nivel internacional. La estrategia empírica se basa en la aplicación de los métodos de Diferencias en Diferencias, Control Sintético y Diferencias en Diferencias Sintética con el fin de identificar cambios en la actividad de desarrollo asociados con la disponibilidad del servicio. Los resultados muestran un impacto diferenciado en el número de *unique pushers* por cada 100 mil habitantes en los países con acceso a ChatGPT en comparación con aquellos donde el servicio no está disponible. Según el estimador SDiD, los mayores efectos corresponden a JavaScript (+8,303 *unique pushers* por cada 100 mil habitantes), Python (+4,525) y TypeScript (+3,078), todos significativos al 1 %, mientras que lenguajes como C# no muestran impacto estadísticamente significativo.

Palabras clave: chatgpt, LLM, diferencia en diferencias sintética

Abstract

In recent years, Artificial Intelligence has begun to significantly transform software production and the way developers interact with programming tools. In particular, the emergence of ChatGPT has provided programmers with a new resource for code analysis, generation, and debugging, as well as for solving technical problems and accelerating learning processes. Tools based on large language models can suggest code snippets, explain complex functions, assist in the documentation of projects, and support developers during different stages of the software development cycle. These capabilities may reduce development time, improve productivity, and facilitate knowledge acquisition among programmers with different levels of experience. Within this context, the objective of this study is to evaluate the impact of ChatGPT on software production during the period from the first quarter of 2020 to the fourth quarter of 2023. To accomplish this, the study uses the GitHub Innovation Graph database, which provides aggregated information on software development activity across countries. The empirical strategy relies on the application of Difference-in-Differences, Synthetic Control, and Synthetic Difference-in-Differences methods in order to identify changes in development activity associated with the availability of the service. The results show a differentiated impact on the number of *unique pushers* per 100,000 inhabitants in countries that have access to ChatGPT compared with those where the service is not available. Using the SDiD estimator, the largest effects are observed for JavaScript (+8.303 unique pushers per 100,000 inhabitants), Python (+4.525), and TypeScript (+3.078), all significant at the 1 % level, while languages such as C# show no statistically significant impact.

Keywords: chatgpt, LLM, synthetic difference in differences

Índice de contenido

Introducción	1
1. Revisión de Literatura	4
1.1. ChatGPT: Un Modelo de Lenguaje de Gran escala	4
1.2. Las versiones de ChatGPT: línea de tiempo de mejoras y limitaciones (2020–2023)	4
1.3. Capacidades de ChatGPT	6
1.4. Marco teórico económico	7
1.5. Evidencia empírica	11
2. Datos	15
3. Estadísticos descriptivos	17
4. Metodología	19
4.1. Diferencia en Diferencias (DiD)	19
4.2. Control Sintético (CS)	20
4.3. Diferencia en Diferencias Sintética (SDiD)	21
4.4. Inferencia	22
4.5. Pruebas de robustez: placebos in-space y sensibilidad del pool de donantes	22
5. Resultados	24
Tablas	32
Figuras	35
Conclusiones	28
Referencias bibliográficas	52
Anexos	56
Anexo 1: Disponibilidad global de ChatGPT	56
Anexo 2: Variables de control y justificación de su no inclusión en el modelo base	60
Anexo 3: Análisis de evento SDID: validación de pre-tendencias	65
Anexo 4: Derivación formal del modelo teórico de exposición y absorción	71

Índice de Figuras

Figura 1: Distribución de lenguajes de programación más populares en países con ChatGPT disponible (2020-2023)	35
Figura 2: Distribución de lenguajes de programación más populares en países sin ChatGPT disponible (2020-2023)	36
Figura 3: Distribución de <i>unique pushers</i> por cada 100 mil habitantes según lenguaje en países con ChatGPT (2020–2023)	37
Figura 4: Distribución de <i>unique pushers</i> por cada 100 mil habitantes según lenguaje en países sin ChatGPT (2020–2023)	38
Figura 5: Tendencias y ponderaciones estimadas de TypeScript <i>unique pushers</i> por cada 100 mil personas	39
Figura 6: Tendencias y ponderaciones estimadas de JavaScript <i>unique pushers</i> por cada 100 mil personas	40
Figura 7: Tendencias y ponderaciones estimadas de Python <i>unique pushers</i> por cada 100 mil personas	41
Figura 8: Tendencias y ponderaciones estimadas de Java <i>unique pushers</i> por cada 100 mil personas	42
Figura 9: Tendencias y ponderaciones estimadas de Go <i>unique pushers</i> por cada 100 mil personas	43
Figura 10: Tendencias y ponderaciones estimadas de Ruby <i>unique pushers</i> por cada 100 mil personas	44
Figura 11: Tendencias y ponderaciones estimadas de PHP <i>unique pushers</i> por cada 100 mil personas	45
Figura 12: Tendencias y ponderaciones estimadas de C# <i>unique pushers</i> por cada 100 mil personas	46
Figura 13: Tendencias y ponderaciones estimadas de C++ <i>unique pushers</i> por cada 100 mil personas	47
Figura 14: Tendencias y ponderaciones estimadas de C <i>unique pushers</i> por cada 100 mil personas	48
Figura 15: Robustez SDID: ATT observado, rango <i>leave-one-out</i> y banda placebo 95 %	49
Figura 16: Distribución de ATT placebo por lenguaje (prueba no paramétrica de Abadie)	50
Figura 17: Línea de tiempo de ChatGPT: mejoras y limitaciones (2020–2023)	51
Figura 18: Mapa de disponibilidad global de ChatGPT	56
Figura 19: Análisis de evento SDID — C	65
Figura 20: Análisis de evento SDID — C#	66
Figura 21: Análisis de evento SDID — C++	66
Figura 22: Análisis de evento SDID — Go	67
Figura 23: Análisis de evento SDID — Java	67
Figura 24: Análisis de evento SDID — JavaScript	68
Figura 25: Análisis de evento SDID — PHP	68
Figura 26: Análisis de evento SDID — Python	69
Figura 27: Análisis de evento SDID — Ruby	69
Figura 28: Análisis de evento SDID — TypeScript	70

Índice de Tablas

Tabla 1: Estadísticos descriptivos del número de <i>unique pushers</i> por cada 100 mil habitantes según lenguaje de programación en países del grupo de control (periodos pre y postratamiento)	32
Tabla 2: Estadísticos descriptivos del número de <i>unique pushers</i> por cada 100 mil habitantes según lenguaje de programación en países del grupo de tratamiento (periodos pre y postratamiento)	32
Tabla 3: Impacto de ChatGPT en el número de programadores	33
Tabla 6: Robustez SDID: placebos in-space y sensibilidad del pool de donantes (LOO)	34
Tabla 7: Mapeo de la cantidad de muestras	61
Tabla 8: Comparación por grupo de muestras	61
Tabla 9: Países eliminados	61
Tabla 10: Missing values o datos vacíos por país	62
Tabla 4: Impacto de ChatGPT en el número de programadores (con controles)	63
Tabla 5: Robustez: SDID con grupo de control restringido (excluye censores severos)	64

Introducción

Planteamiento del problema

Antes de la introducción de ChatGPT, la industria del software ya estaba comprometida con la búsqueda de herramientas y metodologías más eficientes y automatizadas, puesto que pretendía reducir la carga cognitiva de los desarrolladores y acelerar los ciclos de desarrollo. Estos esfuerzos se han reflejado en el incremento de ocho veces más de las patentes de Inteligencia Artificial comparadas con las demás (World Intellectual Property Organization, 2022). Respecto a esta cifra, resaltan los avances en el subcampo de Procesamiento y Generación de Lenguaje, el cual se enfoca en desarrollar modelos de aprendizaje automático entrenados con lenguaje humano para analizar, predecir y generar texto.

La arquitectura *transformer* se ha vuelto fundamental en dicho subcampo, debido a que optimiza tanto el entrenamiento como la implementación de modelos centrados en la interacción entre palabras en el lenguaje natural. OpenAI ha liderado la construcción de esta arquitectura con el desarrollo de la serie de modelos GPT (Generative Pretrained Transformer), ejemplos extraordinarios de Grandes Modelos de Lingüísticos (LLM). Dichos modelos, diseñados para procesar y generar texto coherente y relevante, han sido implementados en ChatGPT, un chatbot que aprovecha su habilidad para generar respuestas basadas en una amplia base de datos de temas variados, incluyendo consultas de programación (OpenAI, 2018). Lo cual ha impulsado su uso en la industria del software al cumplir el rol de orientador y asistente sobre diferentes lenguajes de programación.

Desafortunadamente, se ha identificado a un grupo que no posee la disponibilidad del servicio del chatbot. La razón principal de ello se debe al establecimiento de restricciones de acceso a ChatGPT en países como China, Irán, Siria, Rusia, Corea del Norte, Cuba e Italia, los cuales han sido promovidos por preocupaciones de seguridad nacional, privacidad o cumplimiento de regulaciones locales (Martínez, 2023). Al respecto, algunos autores han señalado que esta limitación ha inhibido la innovación y provocado que estas regiones se mantengan atrasadas en la economía digital global, dado que los desarrolladores de software no han podido aprovechar los beneficios del chatbot (Del Río-Chanona et al., 2023).

Ante ello, recientes estudios han manifestado los desafíos de ChatGPT. Gallea (2023) observa un rendimiento deficiente del chatbot en responder a consultas complejas de código en los lenguajes de programación Python y R. Brynjolfsson et al. (2023) mencionan que las alucinaciones son responsables de que herramientas populares basadas en modelos de lenguaje, como ChatGPT, generen información falsa o engañosa de forma impredecible. Demirci et al. (2023) ponen de relieve el problema de la reducción de oferta de trabajos ligados a la codificación para trabajadores independientes y

enfatan la necesidad de encontrar soluciones. Por su parte, otros especialistas manifiestan su preocupación por la escasez en la producción de datos públicos en internet, los cuales son necesarios para entrenar a modelos de lenguaje como ChatGPT. De esta manera, resaltan la menor interacción de usuarios (a través de consultas y respuestas) de plataformas abiertas populares como Stack Overflow (Del Río-Chanona et al., 2023).

Pese a la creciente fama y la veloz integración de ChatGPT en diversos campos, se ha identificado la ausencia de investigaciones empíricas centradas en evaluar el impacto con evidencia empírica robusta. Por lo que este estudio, se orienta a llenar ese vacío en la literatura y se concentra en determinar si ChatGPT ha contribuido a incrementar el desarrollo de software en los últimos años. Asimismo, se expondrán los beneficios y los desafíos relacionados al uso de la generación automatizada de código. Con este enfoque, se pretender brindar un análisis integral a manera de balance entre los beneficios y riesgos potenciales con tal de proponer soluciones mediante mejores prácticas o futuras políticas en las corporaciones a propósito de la adopción de la IA en la industria.

En el presente estudio, se considera al número de *unique pushers*¹ por cada 100 mil habitantes como la variable de interés del estudio. Dicha variable será usada como *proxy* de la actividad de desarrollo de software y, con la cual, se realizará una evaluación del impacto heterogéneo de la disponibilidad de ChatGPT en los países con acceso al chatbot comparado con aquellos que sin su disponibilidad. De esta forma, este análisis será capaz de inferir cómo el uso de la IA influye en la incorporación de nuevos desarrolladores de software y, de forma indirecta, al incremento de la actividad productiva en entornos digitales, en particular, en los diez lenguajes de programación más empleados en ciencia de datos.

Motivación/Justificación

La integración de la inteligencia artificial en el desarrollo de software, particularmente a través de herramientas avanzadas como ChatGPT, está redefiniendo las metodologías tradicionales de este campo. Con un mercado global de software que se proyecta superar los 600 mil millones de dólares para 2025, la eficiencia y la innovación en el desarrollo de software no solo son deseables, sino imperativas para mantener la competitividad global (Statista, 2023). La capacidad de ChatGPT para generar código y asistir en la solución de problemas de programación de manera eficiente presenta una oportunidad crucial para mejorar estos procesos. Sin embargo, la adopción y las implicaciones de esta tecnología no se han evaluado completamente, lo que destaca

¹La métrica *unique pushers* denota la frecuencia de desarrolladores únicos que, al menos, han subido un código en GitHub en determinado trimestre.

la necesidad crítica de una investigación exhaustiva.

Este estudio abordará la comprensión de cómo las herramientas basadas en modelos de lenguaje, específicamente, ChatGPT pueden afectar la productividad y la calidad en el desarrollo de software. Pese al uso progresivo de inteligencia artificial en muchos ámbitos, son aún muy pocos los estudios que han analizado el impacto de estas tecnologías sobre el desarrollo de software. Esta brecha resulta particular incluyendo a las limitaciones por la eficacia y precisión del código generado en determinados lenguajes de programación. Bajo esta premisa, se busca llenar ese vacío brindando un análisis empírico detallado de las utilidades y desafíos que representa la aplicación de ChatGPT en los flujos de trabajo de desarrollo.

Adicionalmente, se examinan los efectos adversos derivados de la adopción de estas tecnologías. Tal como refiere Chui et al. (2023), el aumento del 14 % asociado al uso de la IA en las compañías a nivel mundial desde 2021 sugiere una creciente dependencia en diferentes sectores. Así, la literatura existente indica que esta nueva herramienta de IA puede afectar la dinámica laboral, la seguridad de los datos y la equidad en el acceso tecnológico. Por otra parte, los sesgos presentes en los modelos de lenguaje y las limitaciones en la calidad de código generado constituyen desafíos importantes que deben ser solucionados en el mediano plazo por dicha dependencia.

La prohibición de ChatGPT en varios países se ha debido primordialmente sobre preocupaciones de seguridad nacional y privacidad (ONU, 2023). Sin embargo, se vuelve importante esclarecer si estas medidas benefician o limitan a la industria de software a nivel mundial. En ese sentido, resulta de vital importancia que el presente estudio mantenga un enfoque integral que abarque tanto las ventajas como las limitaciones de estas tecnologías. Asimismo, en caso de que estas políticas de prohibición posean un impacto negativo, no basta con nombrar su derogatoria sino también de reforzar el impacto positivo de la accesibilidad al servicio brindando una serie de recomendaciones de políticas estratégicas en el sector.

Hipótesis

La accesibilidad a ChatGPT en un país se asociaría positivamente con el desarrollo de software en los diez lenguajes de programación más populares de ciencia de datos. En particular, habría un incremento del número de *unique pushers* por cada 100 mil habitantes en países que cuentan con ChatGPT frente a aquellos sin acceso al servicio. Además, tal efecto positivo se acentuaría en lenguajes como Python y JavaScript dada la vasta documentación técnica, ejercicios y consultas en foros especializados. Tales recursos servirían como datos de entrenamiento valiosos para que el modelo de lenguaje resuelva de manera eficiente en múltiples contextos de programación.

1. Revisión de Literatura

Los Modelos de Lenguaje Grande (LLM o Large Language Models por sus siglas en inglés) son sistemas de inteligencia artificial con habilidades de alta precisión para la generación y comprensión de textos en lenguaje humano. Tales sistemas han sido entrenados con extensos volúmenes de datos textuales con el objetivo de aprender y predecir información con una gran lucidez. Dichos LLM se enmarca en el campo del Machine Learning, pues utilizan sus algoritmos más avanzados: las redes neuronales. La dinámica de estos algoritmos se centra en aprender y adaptarse logrando mejores resultados de rendimiento a medida que mejore la data de entrenamiento y el tiempo de ejecución.

1.1. ChatGPT: Un Modelo de Lenguaje de Gran escala

ChatGPT funciona como un chatbot, con el cual un usuario simula conversar con alguna persona con amplios conocimientos, quien en realidad es la propia inteligencia artificial. Dicho modelo fue creado por la compañía norteamericana OpenAI tomando como base la arquitectura GPT (Generativa Pre-trained Transformer) (Tlili et al., 2023). Además, Kalla et al. (2023) y Abu et al. (2023) identifican a ChatGPT como una LLM por su propia estructura de redes neuronales profundas multicapa de transformadores, los cuales están diseñados para procesar y generar respuestas con coherencia en un lenguaje humano. Por su parte, Shen et al. (2024) complementa que los algoritmos del chatbot han sido entrenado con extensas bases de datos de texto extraídas de internet, evidenciando una amplitud de su capacidad para brindar respuestas y realizar tareas que imitan al cerebro humano.

1.2. Las versiones de ChatGPT: línea de tiempo de mejoras y limitaciones (2020–2023)

ChatGPT no es un único modelo, sino una sucesión de versiones que evolucionaron rápidamente a lo largo de la ventana de análisis. Cada versión incorporó capacidades nuevas y enfrentó limitaciones técnicas distintas, factores que en conjunto condicionan tanto la magnitud como la trayectoria del efecto que se busca identificar. La Figura 17 sintetiza visualmente los hitos relevantes del período; los párrafos siguientes los desarrollan en orden cronológico, prestando especial atención a las restricciones que pudieron moderar el efecto en el corto plazo.

- **Antecedente: GitHub Copilot (junio 2021 – junio 2022).** Antes del lanzamiento público de ChatGPT, el principal exponente de la asistencia con IA para programadores fue GitHub Copilot, desarrollado conjuntamente por GitHub y OpenAI

sobre el modelo Codex. Copilot se lanzó en *technical preview* en junio de 2021 y se comercializó en su versión de pago en junio de 2022, con un costo de 10 USD mensuales. Aunque integraba la sugerencia automática de código en editores como Visual Studio Code, su adopción se concentró en desarrolladores profesionales y suscriptores individuales, lo que limitó su penetración en términos cuantitativos. Esta escala lo diferencia radicalmente del *shock* que se analiza en este estudio —un servicio gratuito y masivo— y permite interpretar a Copilot como una primera ola de asistencia con IA cuya magnitud fue muy inferior, en órdenes de difusión, al lanzamiento de noviembre de 2022.

- **GPT-3.5: la ola masiva (noviembre 2022 – febrero 2023).** La versión GPT-3.5 se liberó al público general como producto gratuito el 30 de noviembre de 2022 y produjo una adopción global prácticamente inmediata; constituye el inicio del período de tratamiento del presente estudio. Con capacidades avanzadas de comprensión y generación de lenguaje natural y de código, optimizadas mediante las APIs Chat Completions, esta versión es la que concentra el análisis empírico predominante en la literatura, incluido el de esta tesis. Sin embargo, presentaba tres limitaciones técnicas relevantes que probablemente moderaron el efecto en los primeros meses tras el lanzamiento: su corte de entrenamiento, fijado en septiembre de 2021, le impedía conocer cambios posteriores en lenguajes y APIs; sus alucinaciones eran frecuentes en problemas de complejidad media-alta, incluyendo la generación de citas a librerías inexistentes; y su ventana de contexto, restringida a 4,096 *tokens*, dificultaba el análisis de archivos extensos. Gallea (2023) documenta, en línea con lo anterior, un desempeño deficiente del modelo en consultas complejas de Python y R durante este período.
- **GPT-4 y la expansión de capacidades (marzo – diciembre 2023).** En marzo de 2023, OpenAI liberó GPT-4 a través de la suscripción ChatGPT Plus, un modelo multimodal con un costo de 20 USD mensuales que redujo significativamente las alucinaciones en la generación de código, mejoró el razonamiento sobre estructuras complejas y amplió la ventana de contexto a entre 8,192 y 32,768 *tokens*. La integración progresiva de Code Interpreter y de *plugins* a partir de julio de 2023 amplió aún más el alcance funcional. No obstante, el acceso pagado y los límites de mensajes por hora restringieron el uso intensivo de GPT-4 a un subconjunto de la base de usuarios, lo que implica que, durante los trimestres centrales de 2023, los usuarios gratuitos del grupo tratado siguieron interactuando primordialmente con GPT-3.5. En paralelo, se consolidaron ecosistemas competidores —GitHub Copilot Chat, Google Bard y Anthropic Claude, entre otros— que pudieron contribuir a una expansión generalizada de la productividad asistida por IA durante el cierre de la ventana de análisis.

Implicaciones para la interpretación del efecto estimado. La heterogeneidad de capacidades y modelos de acceso dentro del período postratamiento (Q4-2022 a Q4-2023)² implica que el efecto identificado debe interpretarse como un promedio sobre subperíodos cualitativamente distintos. El primero, dominado por GPT-3.5 y sus limitaciones iniciales, es plausiblemente el de menor productividad marginal por usuario; los subperíodos posteriores combinaron la disponibilidad de GPT-4 para usuarios premium con un mejor aprovechamiento del modelo gratuito por parte del público general. Estos elementos hacen plausible un efecto no monotónico en el corto plazo y constituyen una hipótesis pertinente para futuras investigaciones que dispongan de ventanas postratamiento más amplias y desagregadas por subperíodo.

1.3. Capacidades de ChatGPT

Diversos autores han propuesto métricas y benchmarks con el objetivo de estimar la precisión y calidad del código generado por ChatGPT. En esta línea, Chai et al. (2024) presentan MCEVAL, un benchmark multitarea y multilingüe para la evaluación de generación de código, que abarca 40 lenguajes de programación y más de 16.000 ejemplos diseñados manualmente. A diferencia de benchmarks anteriores como HumanEval o MultiPL-E, los cuales se centran principalmente en Python o dependen de traducciones automáticas a otros lenguajes, MCEVAL fue construido desde cero mediante anotaciones humanas. Dicho benchmark incluye tareas específicas de generación, explicación y verificación de código, lo que garantiza una evaluación más representativa de las capacidades multilingües de los modelos de lenguaje de gran escala (LLM). En cuanto al desempeño de ChatGPT-3.5 Turbo, los resultados evidencian una alta variabilidad según el lenguaje de programación: se observan tasas de éxito relativamente elevadas en lenguajes populares como Python (60 %), Java (71.7 %), PHP (68 %) y Ruby (76 %), mientras que se identifican limitaciones significativas en lenguajes menos difundidos como Lisp (26 %), Scheme (46 %), Markdown (24 %) y VimL (24 %). Estos hallazgos sugieren que, si bien el modelo alcanza un desempeño sólido en lenguajes de propósito general ampliamente utilizados, aún enfrenta desafíos importantes en lenguajes con menor presencia en los datos de entrenamiento.

De manera consistente, Li y Krishnamachari (2024) muestran que el rendimiento de ChatGPT-3.5 Turbo depende fuertemente del lenguaje de programación empleado. Para ello, utilizan un benchmark compuesto por 1.475 problemas extraídos de la plataforma LeetCode, ampliamente utilizado para la preparación de entrevistas técnicas, y evalúan la capacidad del modelo para generar soluciones correctas en seis lenguajes de programación: Python (lenguaje base), Java, C++, Elixir, Erlang y Racket. En un

²Versiones lanzadas con posterioridad a la ventana de análisis, como GPT-4o (mayo 2024), no son consideradas en la estimación.

subconjunto de problemas previamente resueltos en Python, el modelo logró resolver el 70 % de los casos en Java y el 50 % en C++, e incluso presentó soluciones exitosas en estos lenguajes tras haber fallado en Python, lo que sugiere una sensibilidad del modelo a las características específicas de cada lenguaje. En contraste, no resolvió ningún problema en lenguajes funcionales menos comunes como Elixir, Erlang y Racket, resultado que los autores atribuyen a su baja representación en los datos de entrenamiento y las diferencias en sus paradigmas y estructuras sintácticas. En conjunto, estos resultados confirman que el desempeño de ChatGPT-3.5 Turbo es altamente dependiente del lenguaje de programación considerado.

Por su parte, Buscemi (2023) propone un benchmark *ad hoc* para evaluar la capacidad de generación de código de ChatGPT-3.5 Turbo en 10 lenguajes de programación, a partir de 40 tareas organizadas en cuatro categorías: algoritmos simples, juegos, seguridad informática y ciencia de datos. El benchmark se construyó mediante consultas diseñadas por el propio autor y ejecutadas a través de la API de OpenAI, solicitando explícitamente la implementación del código sin el uso de librerías externas y la inclusión de una única prueba por tarea. A diferencia de benchmarks existentes como HumanEval , MBPP o CodeContests , que suelen limitarse a pocos lenguajes o presentan una cobertura de pruebas reducida, este enfoque permite una comparación multilingaje más exhaustiva, con 10 ejecuciones por tarea y lenguaje, lo que totaliza 4,000 evaluaciones individuales. En la categoría de ciencia de datos, Julia se posicionó como el lenguaje con mayor tasa de éxito en la generación de código ejecutable, alcanzando un 81.5 %, seguido por R y Python. Estos resultados sugieren que ChatGPT-3.5 Turbo presenta un mejor desempeño funcional en lenguajes de alto nivel diseñados específicamente para análisis de datos y modelado estadístico, lo que refleja una relación entre la familiaridad del modelo con dichos lenguajes, posiblemente asociada a su popularidad en los datos de entrenamiento, y su capacidad para resolver tareas complejas de ciencia de datos de manera.

1.4. Marco teórico económico

La hipótesis del estudio se sustenta en tres vertientes complementarias de la teoría económica que permiten explicar por qué la disponibilidad de ChatGPT debería incrementar la producción de software, medida a través de la entrada de nuevos desarrolladores activos. La primera proviene de la teoría del cambio tecnológico y la reasignación de tareas; la segunda, de la economía de la información y los costos de búsqueda; la tercera formaliza ambos mecanismos en un modelo de tareas con decisión ocupacional de entrada, del cual se deriva la especificación empírica del estudio. Los tres marcos, que se desarrollan a continuación, enmarcan la evidencia empírica revisada en la subsección siguiente.

La primera vertiente se apoya en el marco de reasignación de tareas propuesto por Acemoglu y Restrepo (2018, 2019), que concibe el cambio tecnológico no como un *shock* aumenta-factor uniforme, sino como un proceso que redistribuye las tareas constitutivas de la producción entre el capital (algoritmos) y el trabajo (programadores). Desde esta perspectiva, una tecnología como ChatGPT opera a través de dos canales simultáneos. Por un lado, ejerce un *displacement effect*, al automatizar tareas antes realizadas por programadores —la búsqueda de sintaxis, la redacción de *boilerplate* o la depuración rutinaria—; por otro, ejerce un *reinstatement effect*, al reducir el costo marginal de producir código y volver rentables proyectos y tareas que previamente no lo eran. Cuando la oferta de programadores experimentados es relativamente inelástica y la productividad marginal del trabajo no automatizado se incrementa tras la introducción del modelo, el efecto neto esperado es un aumento de la actividad de desarrollo y, en particular, una mayor incorporación de desarrolladores marginales que antes enfrentaban barreras de capital humano demasiado elevadas. Esta predicción es coherente con el argumento de Acemoglu y Restrepo (2018), según el cual las nuevas tecnologías reasignan tareas hacia los segmentos del mercado laboral con menor productividad pre-tecnológica, lo que en el presente contexto se traduce en la entrada de desarrolladores junior o autodidactas al ecosistema productivo.

La segunda vertiente proviene de la economía de la información. Desde Stigler (1961), la búsqueda se concibe como una actividad costosa que el agente emprende para reducir su incertidumbre sobre las alternativas relevantes; en el ámbito del desarrollo de software, ese costo se materializa en el tiempo que el programador dedica a la documentación, a foros como Stack Overflow, a tutoriales o a la experimentación. ChatGPT puede interpretarse como una tecnología que reduce el costo marginal de búsqueda y aprendizaje a niveles cercanos a cero, al ofrecer respuestas contextualizadas y casi instantáneas a consultas que antes exigían minutos u horas de exploración. Bajo esta lente, la disponibilidad del modelo equivale a un *shock* negativo de costos sobre la función de producción de código del desarrollador: aun manteniendo constante la oferta de programadores experimentados, cabría esperar un incremento de la actividad, especialmente entre quienes enfrentaban costos de búsqueda proporcionalmente más altos respecto de su productividad esperada, esto es, los desarrolladores nuevos o autodidactas.

La tercera vertiente integra los dos mecanismos anteriores en un modelo formal de tareas con decisión ocupacional de entrada, en la tradición del marco de tareas de Acemoglu y Restrepo (2018) y de su actualización para la IA generativa desarrollada por Acemoglu (2024), cuyo propósito es trasladar la intuición tecnológica —ChatGPT no mejora todas las tareas por igual, sino principalmente aquellas en las que su productividad supera a la humana— hacia una ecuación estimable. En el modelo, la producción de un desarrollador potencial j en el lenguaje c se obtiene agregando un continuo

de tareas elementales $z \in [0, 1]$, en cada una de las cuales el individuo escoge el método más productivo entre su productividad humana $h_c(z) > 0$ y la productividad de ChatGPT $a_c(z) \geq 0$, condicional a su acceso a la herramienta $\mu_j \in \{0, 1\}$:

$$y_{jc}(\mu_j) = \int_0^1 \max\{h_c(z), \mu_j a_c(z)\} dz.$$

El conjunto de tareas automatizables del lenguaje, $\mathcal{Z}_c^{\text{auto}} = \{z \in [0, 1] : a_c(z) > h_c(z)\}$, tiene medida τ_c —la fracción de tareas en las que ChatGPT supera al humano— y, dentro de él, el ahorro relativo por tarea $s_c(z) = [a_c(z) - h_c(z)]/h_c(z)$ tiene promedio condicional ρ_c . A partir de estas definiciones, la ganancia individual de acceder a ChatGPT admite una expresión exacta, libre de supuestos adicionales:

$$\Delta y_{jc} = y_{jc}(1) - y_{jc}(0) = \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) s_c(z) dz = \tau_c [\bar{h}_c \rho_c + \text{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c)],$$

donde $\bar{h}_c$ denota la productividad humana promedio dentro de las tareas automatizables. Bajo el supuesto de covarianza nula entre la productividad humana base y el ahorro relativo, y normalizando la escala de producción de modo que $\bar{h}_c = 1$, la ganancia individual se reduce a

$$\Delta y_{jc} = \tau_c \rho_c \equiv \theta_c,$$

donde θ_c se interpreta como la exposición del lenguaje c a ChatGPT: combina cuántas tareas del lenguaje son automatizables (τ_c) y cuán grande es la mejora promedio en esas tareas (ρ_c). Esta formulación adapta el modelo de automatización parcial de Acemoglu (2024), en el cual la fracción de tareas automatizables y el ahorro de costos promedio por tarea (*cost saving share*) determinan, a través de su producto, la exposición efectiva de un sector a la IA generativa. A su vez, θ_c constituye el análogo lingüístico de los índices de exposición ocupacional de Eloundou et al. (2023), quienes, sobre la clasificación de tareas de O*NET, miden la fracción de tareas de cada ocupación en las que un modelo de lenguaje reduce sustantivamente el tiempo de trabajo, y documentan que la exposición es sistemáticamente mayor donde la tecnología dispone de mayor representación en su corpus de entrenamiento. En esa misma línea, el parámetro recoge de forma compacta la heterogeneidad documentada en los *benchmarks* por lenguaje (Buscemi, 2023; Li y Krishnamachari, 2024), asociada a la representación de cada lenguaje en los datos de entrenamiento del modelo.

El paso de la productividad a la entrada de nuevos desarrolladores se modela como una decisión ocupacional con umbral. Un individuo con habilidad η entra a programar si la utilidad de hacerlo, $\eta + y_{jc}(\mu_j)$, supera la utilidad de reserva r_i de su país, lo que define el umbral mínimo de habilidad $\eta_c^*(\mu) = r_i - y_{jc}(\mu)$. Dado que el acceso a

ChatGPT incrementa la producción, el umbral se reduce exactamente en la magnitud de la ganancia individual:

$$\eta_c^*(1) = \eta_c^*(0) - \theta_c.$$

Este desplazamiento del umbral formaliza el mecanismo de la segunda vertiente: la reducción de los costos de búsqueda y aprendizaje opera con mayor intensidad sobre los individuos marginales, cuya habilidad se sitúa entre el umbral antiguo y el nuevo. Agregando sobre la distribución de habilidad F_i del país i , con densidad f_i , y aplicando una aproximación de Taylor de primer orden, el cambio en el número de desarrolladores activos por cada 100,000 habitantes se factoriza en un componente de país y un componente de lenguaje:

$$\Delta Y_{ic} \approx \phi_i \theta_c, \quad \phi_i = 10^5 f_i(\eta_c^*(0)),$$

donde ϕ_i mide la capacidad de absorción tecnológica del país, es decir, la masa de personas situadas cerca del umbral de entrada a la programación. Finalmente, incorporando la disponibilidad de ChatGPT mediante el indicador $A_{it} \in \{0, 1\}$ y descomponiendo la línea base en efectos fijos, el modelo entrega directamente la especificación de panel del estudio:

$$Y_{ict} = \alpha_i + \beta_t + \delta_c + \theta_c \phi_i A_{it} + \eta_{ict},$$

cuya interpretación causal requiere que, condicional a los efectos fijos de país (α_i), tiempo (β_t) y lenguaje (δ_c), la disponibilidad de ChatGPT no esté correlacionada con los *shocks* no observados η_{ict} . Esta ecuación fundamenta la estrategia empírica del estudio y predice que el efecto del tratamiento es heterogéneo: creciente en la exposición del lenguaje (θ_c) y en la capacidad de absorción del país (ϕ_i). La derivación completa del modelo, paso a paso, junto con la discusión formal de sus supuestos y un ejemplo numérico ilustrativo, se presenta en el Anexo 4.

En conjunto, las tres vertientes convergen en una misma predicción: la disponibilidad de ChatGPT debería incrementar el número de *unique pushers* en GitHub en los países con acceso al servicio, con efectos heterogéneos por lenguaje de programación que reflejan tanto la representación de cada lenguaje en los datos de entrenamiento del modelo (Chai et al., 2024; Buscemi, 2023) como la elasticidad de la demanda de tareas automatizables en cada ecosistema lingüístico. La evidencia empírica revisada a continuación resulta consistente con ambos mecanismos: Brynjolfsson et al. (2023) documentan el patrón de reasignación de tareas en agentes de servicio al cliente — la asistencia con IA eleva la productividad sobre todo entre los trabajadores menos experimentados—, mientras que Del Rio-Chanona et al. (2023) aportan evidencia del *shock* sobre los costos de búsqueda al registrar una caída del 16 % en la actividad de

Stack Overflow tras el lanzamiento de ChatGPT.

1.5. Evidencia empírica

La veloz integración de la Inteligencia Artificial (IA) en diversos campos ha provocado un considerable interés académico, vinculado en evaluar el impacto en la economía global, la transformación de las dinámicas laborales y el impulso de la innovación tecnológica. En esta subsección, se revisan estudios empíricos que analizan los efectos negativos por la aplicación de las tecnologías disruptivas con énfasis en los modelos de lenguaje grande como lo es ChatGPT. Dichos estudios emplean metodologías de evaluación de impacto para estimar sus efectos en variables como la productividad, calidad del trabajo y la dinámica del empleo. Asimismo, se brinda un reporte de las principales pruebas de rendimiento realizadas a la calidad de código de ChatGPT.

Brynjolfsson et al. (2023) examinan la adopción de la IA generativa como chatbot en servicios de atención al cliente y, precisamente, su impacto en la productividad. El estudio usa un modelo de diferencia en diferencias (DiD) con un total de 5,179 agentes de soporte al cliente de la empresa Fortune 500. La variable dependiente es la productividad de los agentes, medida por el número de problemas resueltos por hora, y las variables independientes incluyen el acceso a la herramienta de IA y características del agente como experiencia y habilidad. Los hallazgos revelan un incremento del 14 % en la productividad, especialmente notable en trabajadores menos experimentados y de menor habilidad, mejoras en el sentimiento del cliente y una reducción en las intervenciones gerenciales. Los resultados sugieren que la IA puede facilitar la diseminación de conocimiento tácito y acelerar el aprendizaje en nuevos empleados, aunque no se exploran los efectos globales sobre el empleo o los salarios.

Gallea (2023) investiga el impacto de la IA generativa en la dinámica laboral dentro de las comunidades de codificación en línea, utilizando como caso de estudio a ChatGPT y su efecto en Stack Overflow. El estudio emplea un modelo de diferencias en diferencias (DiD) para comparar las preguntas relacionadas con los lenguajes de programación Python y R antes y después del lanzamiento de ChatGPT, utilizando datos de Stack Overflow desde octubre de 2022 hasta marzo de 2023. La unidad de análisis se define por el lenguaje de programación y la semana, evidenciando una disminución significativa en el número semanal de preguntas publicadas, particularmente en el caso de Python, que actúa como grupo tratado debido al mayor desempeño de ChatGPT en este lenguaje, mientras que R funciona como grupo de control. Esta reducción sugiere que la disponibilidad de la IA estaría resolviendo de forma eficiente consultas rutinarias fuera de la plataforma. Además, se observa un aumento en la complejidad y la calidad de las preguntas restantes, lo que indica un desplazamiento hacia tareas que requieren mayor especialización y esfuerzo cognitivo. El estudio concluye que, si

bien ChatGPT potencia la productividad al encargarse de tareas repetitivas, también plantea interrogantes sobre los efectos a largo plazo de una mayor dependencia de la IA en las habilidades de resolución de problemas de los programadores.

Liu et al. (2023) analizan el impacto del lanzamiento de ChatGPT como un choque exógeno en los mercados laborales online. Utilizan un modelo de DiD para analizar el volumen de transacciones de freelancers y trabajos en una plataforma de labor online. Las variables independientes incluyen la exposición a ChatGPT, el volumen de transacciones anterior y la calidad del servicio basada en calificaciones previas. La unidad de análisis comprende trabajos y freelancers clasificados por su nivel de exposición a ChatGPT. La base de datos proviene de una plataforma prominente de freelancers que facilita una variedad de servicios digitales y físicos. Las conclusiones señalan que, aunque el lanzamiento de ChatGPT resultó en una disminución significativa en el volumen de transacciones para trabajos expuestos, los freelancers que adaptaron sus servicios para complementar la tecnología IA experimentaron beneficios notables, resaltando la dualidad del impacto de la IA en el trabajo y la necesidad de más investigaciones para comprender plenamente sus efectos.

Wu et al. (2024) investigan el impacto del lanzamiento de ChatGPT en el retorno anormal acumulado (CAR) de las empresas cotizadas en China. Utilizando un modelo de DiD combinado con un estudio de eventos, la investigación evalúa cómo los lanzamientos de GPT-3.5 y GPT-4 influyeron positivamente en las firmas enfocadas en ChatGPT. La unidad de análisis identifica a las compañías del índice A-share del mercado chino, con la variable dependiente siendo el CAR y las independientes incluyendo un indicador de tratamiento basado en la mención de ChatGPT en informes anuales y variables de tiempo post-lanzamiento. Los datos provienen de la base CSMAR y de índices de atención en Baidu, WeChat y Douyin. Las conclusiones indican que los eventos de lanzamiento de ChatGPT aumentaron significativamente los CAR de las empresas enfocadas en esta tecnología, especialmente bajo alta atención social, sugiriendo que la integración de IA generativa podría ser beneficiosa para las empresas y la formulación de políticas.

Del Rio-Chanona et al. (2023) analizan el impacto de los modelos de lenguaje como ChatGPT en la generación de datos públicos. Utilizando un modelo de DiD, los investigadores examinaron las publicaciones en Stack Overflow en comparación con foros similares menos susceptibles de ser sustituidos por ChatGPT. La unidad de análisis es a nivel de plataforma y por semana, considerando las variaciones semanales en la actividad de publicación. Las variables independientes incluyen el tiempo transcurrido desde el lanzamiento de ChatGPT y los lenguajes de programación más populares. La base de datos comprende millones de publicaciones desde la creación de Stack Overflow hasta junio de 2023. Los hallazgos muestran una reducción del 16 % en la actividad de publicación tras la introducción de ChatGPT, sugiriendo una significativa

sustitución por parte de los usuarios que ahora prefieren utilizar modelos de lenguaje para resolver consultas. Esto plantea preocupaciones sobre la disminución de datos públicos disponibles para entrenar futuros modelos, destacando la necesidad de políticas que gestionen el impacto de los LLM en los bienes públicos digitales.

Demirci et al. (2023) examinan el impacto de la IA generativa (GenAI) en las plataformas de freelancing online, centrándose en cómo tecnologías como ChatGPT y otras herramientas de generación de imágenes han cambiado la demanda de trabajos susceptibles a la automatización. Utilizan un modelo de análisis de DiD, incluyendo versiones sintéticas y robustas doblemente, para evaluar estos efectos comparando los períodos antes y después de la introducción de GenAI. La unidad de análisis comprende las ofertas individuales de trabajo en la plataforma, clasificadas semanalmente por país y tipo de trabajo. La variable dependiente es el logaritmo del número de ofertas de trabajo, mientras que las variables independientes incluyen indicadores del período post-introducción de GenAI y un índice de volumen de búsqueda de Google (SVI) como proxy del interés público en estas tecnologías. La base de datos contiene más de 1.38 millones de ofertas de trabajo desde julio de 2021 hasta julio de 2023. Las conclusiones destacan una reducción significativa en la demanda de trabajos propensos a la automatización y generación de imágenes, sugiriendo que la GenAI está reemplazando funciones laborales específicas y subrayando la necesidad de estrategias para integrar la IA en los entornos laborales para mitigar posibles impactos negativos en el mercado laboral.

Hui et al. (2023) examinan el impacto de la IA generativa, como ChatGPT, sobre el empleo y los ingresos de los freelancers en la plataforma en línea Upwork. Utilizando un diseño de DiD, este análisis empírico se centra en las ocupaciones más susceptibles a ser afectadas por la IA, como la redacción y edición. La unidad de análisis son los freelancers individuales, cuyos resultados se miden en términos del número de trabajos e ingresos mensuales. Las variables independientes incluyen un indicador post-introducción de ChatGPT, ocupaciones potencialmente afectadas y características de los freelancers como puntuaciones de *feedback* e ingresos mensuales pasados. La base de datos proviene de la API de Upwork, que ofrece acceso a perfiles completos de empleo y atributos relevantes. Las conclusiones refieren que la introducción de la IA tiene efectos negativos significativos en los freelancers en ocupaciones afectadas, disminuyendo tanto la cantidad de trabajos como los ingresos. Esto sugiere que la IA generativa puede actuar como un sustituto en el corto plazo, reduciendo la demanda general por trabajadores del conocimiento y posiblemente igualando las brechas de productividad entre trabajadores de diferentes calidades.

La literatura revisada indica que ChatGPT y otros modelos de lenguaje grandes han influido significativamente en diversas áreas, desde la generación de datos públicos hasta la dinámica laboral y la productividad. Los estudios destacan tanto beneficios

como desafíos, subrayando la importancia de un análisis equilibrado del impacto neto. Sin embargo, la mayoría de estos estudios no se han centrado específicamente en el impacto de ChatGPT en la productividad del desarrollo de software en diferentes lenguajes de programación. Este estudio se propone llenar ese vacío, proporcionando un análisis empírico confiable que considera los beneficios y desafíos de integrar ChatGPT en los flujos de trabajo de desarrollo de software.

2. Datos

ChatGPT fue lanzado en noviembre de 2022 por el *startup* estadounidense OpenAI, cuya sede se encuentra en San Francisco. Desde su lanzamiento, la herramienta ha alcanzado una adopción masiva, siendo utilizada por millones de personas a nivel global (Kreitmeir & Raschky, 2023). Sin embargo, su disponibilidad no ha sido homogénea entre países, ya que algunos gobiernos han decidido restringir o bloquear su acceso debido a preocupaciones vinculadas a la protección de datos personales, la potencial difusión de desinformación, la ausencia de mecanismos efectivos de regulación de contenidos en línea y conflictos geopolíticos con Estados Unidos (BBC News Mundo, 2023; Ijaz, 2023). En el Anexo 1, se presenta un análisis detallado del estado de accesibilidad de ChatGPT por país.

GitHub es la plataforma de almacenamiento de código más grande en el mundo, utilizada para almacenar y trabajar conjuntamente en proyectos de codificación a través de repositorios. Para este trabajo, la totalidad de los datos mostrados provienen del base de datos Innovation Graph de GitHub, específicamente de la tabla *languages*, la cual contiene a los *unique pushers* a nivel de lenguaje de programación y por trimestre. Esta métrica, denominada *unique pushers*, representa a la cantidad total de desarrolladores únicos que, al menos, realizaron un *gitpush* (subida de código) a un repositorio a GitHub con un lenguaje de programación determinado. Los datos abarcan desde el primer trimestre de 2020 hasta el cuarto trimestre de 2024, pero se tomará solo hasta el año 2023, y están desglosados por lenguaje de programación, incluyendo un total de 50 lenguajes. De estos, se han mantenido los 10 más populares en el campo de la ciencia de datos.

En vista de la gran cantidad de lenguajes de programación recurridos en la ciencia de datos, la selección de los 10 más utilizados se ha basado en diversos rankings y en la cantidad de observaciones disponibles. Así, diferentes fuentes en este campo coinciden en que los lenguajes más importantes son JavaScript, Java, Python, TypeScript, C, C++, C#, PHP, Ruby y Go (Coursera Staff, 2024; Data Camp, 2023; Rice CS, 2023). Aunque otros lenguajes como SQL, R, Julia y Matlab también son reconocidos, fueron excluidos de esta selección debido a su baja cantidad de observaciones, lo que limita su representatividad y análisis comparativo.

Además, el filtrado realizado al conjunto de datos inició con la exclusión de las observaciones relacionadas con la Unión Europea, debido a que no es un país sino un conjunto, lo que ocasiona problema de doble conteo. Asimismo, Kosovo se eliminó por la falta de información disponible en la librería *pycountry*. También se suprimió a Hong Kong del análisis, ya que mostraba un nivel atípicamente alto de *unique pushers* en comparación con otros países. El período de tratamiento comenzó en el cuarto trimestre de 2022, alineándose con el lanzamiento de ChatGPT, ocurrido el 30 de noviembre

de ese mismo año.

Las unidades tratadas, en el presente trabajo, corresponden a los países que originalmente comunicaron que tendrían acceso a la aplicación ChatGTP. Por tanto, durante el período de tiempo indicado, no se añadieron nuevos países al grupo de tratamiento. Esto constituye un diseño de bloques, como se describe en Arkhangelsky et al. (2021), lo cual facilita el cálculo de los estimadores. El diseño del trabajo incluye 130 unidades de tratamiento y 29 unidades de control; 12 períodos de pretratamiento, 4 períodos de tratamiento y un total de 25.440 observaciones.

La especificación principal del estudio no incorpora variables de control adicionales en las estimaciones DiD, SC ni SDiD. Esta decisión, ampliada en el Anexo 2, responde a tres consideraciones: (i) las únicas variables candidatas con cobertura razonable entre los 159 países del panel —porcentaje de uso de computadora y porcentaje de uso de internet del Banco Mundial— presentan información faltante en 19 países (2.640 observaciones, equivalentes al 10,4 % del panel) que debieron imputarse con cero, lo que introduce error de medida correlacionado con el estatus de tratamiento; (ii) las variables disponibles son de frecuencia anual e interpoladas a trimestres, lo que las vuelve insensibles a la dinámica que se busca capturar entre Q4-2022 y Q4-2023; y (iii) los estimadores SC y SDiD ya absorben la heterogeneidad estructural pretratamiento mediante la ponderación de unidades y períodos, lo que reduce el valor marginal de incluir controles observables. La estimación con controles se reporta en la Tabla 4 únicamente como ejercicio de robustez.

3. Estadísticos descriptivos

En esta sección se presentan los estadísticos descriptivos fundamentales que caracterizan los datos utilizados en el estudio. Dichos datos, obtenidos del Innovation Graph de GitHub, incluyen información detallada sobre los *unique pushers* a nivel de lenguaje de programación y trimestre. El análisis de las principales tendencias y patrones observados en los datos proporciona un contexto claro y comprensivo para la interpretación de los resultados subsecuentes.

En la Figura 1, se muestra la distribución porcentual de los lenguajes de programación más usados en ciencia de datos en países con disponibilidad de ChatGPT durante el periodo 2020–2023. En este grupo, JavaScript incrementa su participación de 30.9 % en 2020 a 33.1 % en 2023, consolidando su liderazgo. Python pasa de 18.4 % a 19.4 % en el mismo periodo, evidenciando un crecimiento moderado, mientras que TypeScript registra la expansión relativa más notable, al aumentar de 6.1 % a 9.7 %. En contraste, Java disminuye de 11.5 % a 9.7 %, y Ruby presenta una caída más pronunciada, de 7.1 % a 3.7 %. En contraste, en la Figura 2, correspondiente a países sin disponibilidad de ChatGPT, si bien la estructura general es similar, se observan algunas diferencias cuantitativas relevantes. JavaScript se mantiene prácticamente estable, pasando de 30.5 % en 2020 a 30.0 % en 2023. Python experimenta un crecimiento más intenso que en el grupo tratado, aumentando de 15.7 % a 18.4 %. TypeScript también crece de forma marcada, de 4.5 % a 9.6 %, replicando una tendencia similar a la observada en la Figura 1. Por su parte, Java presenta una reducción más acentuada, de 16.3 % a 10.9 %, y Ruby desciende de 6.0 % a 2.9 %.

La comparación entre la Tabla 1 y la Tabla 2 evidencia diferencias relevantes entre el grupo de control y el grupo de tratamiento en términos de actividad de desarrollo medida por el número de *unique pushers*. En el periodo pretratamiento, el grupo tratado presenta medias superiores en la mayoría de los lenguajes, por ejemplo, JavaScript (51.00 frente a 18.84) y Python (22.71 frente a 13.30), diferencia que se mantiene en el periodo postratamiento (58.98 vs. 34.29 y 33.46 vs. 22.21, respectivamente). En ambos grupos los valores mínimos permanecen en 0.00, lo que evidencia heterogeneidad en la participación entre países, mientras que los valores máximos y las desviaciones estándar aumentan tras el periodo de análisis, especialmente en lenguajes con mayor dinamismo, indicando una expansión general pero desigual de la actividad en GitHub. Estos patrones sugieren la presencia de tendencias temporales comunes junto con diferencias estructurales entre grupos, lo que respalda la aplicación de una metodología de diferencias en diferencias para estimar el efecto del acceso a ChatGPT sobre la actividad de desarrollo en GitHub.

En la Figura 3, correspondiente a países con disponibilidad de ChatGPT, se observa un crecimiento sostenido en el número de *unique pushers* por cada 100 mil

habitantes entre 2020 y 2023, con una aceleración más evidente a partir de 2022. JavaScript concentra los niveles más altos de actividad durante todo el periodo, incrementándose desde aproximadamente 32 unique pushers por cada 100 mil habitantes en 2020-Q1 hasta cerca de 61 en 2023-Q4. Por su parte, Python también muestra un crecimiento considerable, pasando de alrededor de 18 a aproximadamente 36 en el mismo intervalo. Asimismo, TypeScript presenta una expansión notable, aumentando desde aproximadamente 6 al inicio del periodo hasta cerca de 21 en 2023-Q4, lo que evidencia una rápida adopción en los últimos años. En contraste, lenguajes como Java muestran incrementos más moderados, creciendo aproximadamente de 12 a cerca de 17.

En la Figura 4 correspondiente a países sin disponibilidad de ChatGPT, también se identifica una tendencia creciente, aunque con valores menores y una evolución más gradual. En este caso, JavaScript aumenta desde aproximadamente 14 unique pushers por cada 100 mil habitantes en 2020-Q1 hasta cerca de 35 en 2023-Q4. Python, por su parte, pasa de alrededor de 8 a aproximadamente 24 durante el mismo periodo. De manera similar, TypeScript crece progresivamente desde cerca de 3 hasta aproximadamente 13 hacia el final del periodo analizado. En el caso de Java, el incremento resulta más moderado, evolucionando desde aproximadamente 6 hasta cerca de 9–10.

En conjunto, las cuatro figuras evidencian patrones similares en la evolución del ecosistema de desarrollo entre países con y sin disponibilidad de ChatGPT, aunque con diferencias en la magnitud del crecimiento observado. La Figura 1 y Figura 2 muestran que la popularidad de los principales lenguajes utilizados en ciencia de datos se mantiene dominada por JavaScript y Python en ambos grupos, mientras que TypeScript presenta el mayor incremento relativo durante el periodo analizado. Sin embargo, al considerar los niveles de unique pushers por cada 100 mil habitantes presentados en las Figura 3 y Figura 4, se observa que el aumento es más pronunciado en los países con disponibilidad de ChatGPT, especialmente a partir del cuarto trimestre de 2022, coincidiendo con el lanzamiento público de la herramienta. En contraste, lenguajes más tradicionales como Java y Ruby muestran una pérdida relativa de participación o un crecimiento más moderado en ambos contextos.

4. Metodología

En todos los métodos se asume el modelo de resultados potenciales. Para la unidad i en el período t , el resultado observado se descompone como:

$$Y_{it} = Y_{it}^{(0)} + \tau_{it} W_{it}$$

donde $Y_{it}^{(0)}$ es el resultado potencial bajo no-tratamiento y $W_{it} \in \{0, 1\}$ es el indicador de tratamiento, igual a 1 si el país i tiene disponibilidad de ChatGPT en el período $t \geq T_0 + 1$:

$$W_{it} = \mathbf{1}[i \in \mathcal{T}, t \geq T_0 + 1]$$

El parámetro de interés es el efecto promedio del tratamiento sobre los tratados (ATT), definido como:

$$\tau = \frac{1}{N_{tr} T_{post}} \sum_{i \in \mathcal{T}} \sum_{t > T_0} \tau_{it}$$

4.1. Diferencia en Diferencias (DiD)

El modelo DiD se aplica en casos que exista un número considerable de unidades expuestas al programa y se centra en estimar el efecto de la exposición al tratamiento resolviendo la regresión de efectos fijos sin ponderaciones temporales ni unitarias. Dicho método se emplea para estimar el efecto del lanzamiento de ChatGPT en la actividad de desarrollo de software orientado en la ciencia de datos, comparando el comportamiento de esta actividad antes y después del lanzamiento de chatbot con un grupo de control que no fue afectado por esta intervención. Arkhangelsky (2021) menciona que los supuestos que debe cumplir el modelo DiD incluyen las tendencias paralelas; es decir, las trayectorias del grupo tratado y del grupo de control deben ser idénticas a lo largo del tiempo, en ausencia de tratamiento. Además, se asume que el programa no produce efectos antes de su implementación; en otras palabras, los efectos del tratamiento son homogéneos dentro del grupo de tratamiento y que no hay otros eventos indistintamente a los grupos durante el mismo período. Asimismo, existe un supuesto menos estricto en la asignación aleatoria de las unidades tratadas. Esto facilita su aplicación, la validación de otros supuestos y mejorando así la robustez de la estimación del modelo.

La especificación econométrica del modelo de efectos fijos de dos vías es:

$$Y_{it} = \mu + \alpha_i + \beta_t + \tau^{DiD} W_{it} + \varepsilon_{it}$$

donde α_i son efectos fijos de unidad, β_t son efectos fijos de tiempo y ε_{it} es el

término de error. El estimador de diferencias en diferencias puede expresarse explícitamente como:

$$\hat{\tau}^{DiD} = (\bar{Y}_{post, \mathcal{T}} - \bar{Y}_{pre, \mathcal{T}}) - (\bar{Y}_{post, \mathcal{C}} - \bar{Y}_{pre, \mathcal{C}})$$

4.2. Control Sintético (CS)

El método de CS se aplica en entornos con una sola o pocas unidades expuestas al tratamiento y su objetivo es compensar la falta de tendencias paralelas a través de la ponderación de las unidades para que coincidan con sus tendencias previas a la exposición. Dicho modelo se utiliza para crear un contrafactual creíble de la unidad tratada mediante la ponderación de unidades de control. Arkhangelsky (2021) expone los supuestos que deben existir en el método CS, que incluyen la capacidad de ponderar unidades de control o no tratadas para alinear sus tendencias pretratamiento con las de las unidades tratadas. Estas ponderaciones deben permanecer constantes en el tiempo, no debe haber efectos de tratamientos no observados que tengan un impacto significativo y una asignación de pesos adecuada debe garantizar un ajuste adecuado en el período pretratamiento. Se supone que las ponderaciones temporales equilibran los tiempos previos y posteriores al tratamiento.

El estimador de CS se obtiene comparando el resultado observado del grupo tratado con el contrafactual sintético construido a partir del grupo de control ponderado:

$$\hat{\tau}^{SC} = \frac{1}{N_{tr} T_{post}} \sum_{i \in \mathcal{T}} \sum_{t > T_0} \left(Y_{it} - \sum_{j \in \mathcal{C}} \hat{\omega}_j^{sc} Y_{jt} \right)$$

Los pesos $\hat{\omega}^{sc}$ se obtienen minimizando la distancia entre las trayectorias pretratamiento del grupo tratado y del contrafactual sintético:

$$\hat{\omega}^{sc} = \arg \min_{\omega} \sum_{t=1}^{T_0} \left(\sum_{j \in \mathcal{C}} \omega_j Y_{jt} - \bar{Y}_t^{\mathcal{T}} \right)^2$$

Sujeto a:

$$\omega_j \geq 0 \quad \forall j \in \mathcal{C}$$

$$\sum_{j \in \mathcal{C}} \omega_j = 1$$

4.3. Diferencia en Diferencias Sintética (SDiD)

El modelo de SDiD combina las técnicas de DiD y CS para mejorar la precisión de la estimación del efecto del tratamiento, creando y asegurando el supuesto de tendencias paralelas en caso no se cumplan. Según Arkhangelsky et al. (2021), el método asigna ponderaciones tanto a las unidades de control como a los períodos de tiempo, lo que permite alinear las tendencias pretratamiento del grupo tratado con las del contrafactual y mejorar el ajuste en el período postratamiento.

El estimador SDiD se obtiene resolviendo el siguiente problema de optimización de mínimos cuadrados ponderados:

$$(\hat{\tau}^{sdid}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \arg \min_{\tau, \mu, \alpha, \beta} \sum_{i=1}^N \sum_{t=1}^T \hat{\omega}_i^{sdid} \hat{\lambda}_t^{sdid} (Y_{it} - \mu - \alpha_i - \beta_t - W_{it} \tau)^2$$

Los pesos de unidades $\hat{\omega}^{sdid}$ se obtienen minimizando la distancia entre las trayectorias pretratamiento del grupo tratado y el contrafactual ponderado, incluyendo un término de regularización:

$$(\hat{\omega}_0, \hat{\omega}^{sdid}) = \arg \min_{\omega_0, \omega} \sum_{t=1}^{T_0} \left(\omega_0 + \sum_{j=1}^{N_c} \omega_j Y_{jt} - \bar{Y}_t^T \right)^2 + \zeta^2 T_0 \|\omega\|_2^2$$

Sujeto a:

$$\omega_j \geq 0 \quad \forall j \in \{1, \dots, N_c\}, \quad \sum_{j=1}^{N_c} \omega_j = 1$$

El parámetro de regularización ζ y la varianza $\hat{\sigma}^2$ se definen como:

$$\zeta = \hat{\sigma} (N_{tr} T_{post})^{1/4}$$

$$\hat{\sigma}^2 = \frac{1}{N_c (T_0 - 1)} \sum_{j=1}^{N_c} \sum_{t=1}^{T_0-1} (\Delta Y_{jt} - \bar{\Delta}_j)^2, \quad \Delta Y_{jt} = Y_{j,t+1} - Y_{j,t}$$

Para las unidades tratadas se asigna peso uniforme: $\hat{\omega}_i^{sdid} = 1/N_{tr}$ para todo $i \in \mathcal{T}$.

Los pesos temporales $\hat{\lambda}^{sdid}$ se obtienen haciendo que los períodos postratamiento sean comparables con los pretratamiento en el grupo de control:

$$(\hat{\lambda}_0, \hat{\lambda}^{sdid}) = \arg \min_{\lambda_0, \lambda} \sum_{i=1}^{N_c} \left(\lambda_0 + \sum_{t=T_0+1}^T \lambda_t Y_{it} - \bar{Y}_i^{pre} \right)^2$$

Sujeto a:

$$\lambda_t \geq 0 \quad \forall t \in \{T_0 + 1, \dots, T\}, \quad \sum_{t=T_0+1}^T \lambda_t = 1$$

4.4. Inferencia

Los datos empleados en este estudio son exclusivos, puesto que permiten incluir un gran número de unidades tratadas, todas expuestas de manera simultánea. Esta característica brinda la oportunidad de aplicar el estimador *bootstrap*, que Arkhangelsky et al. (2021) demostraron ser asintóticamente normal cuando se cuenta con un gran número de unidades tratadas. Para el diseño del algoritmo *bootstrap* destinado a estimar el error estándar de los estimadores del ATT, se remite a Clarke et al. (2023).

En la práctica, el *bootstrap* se implementa remuestreando con reemplazo las unidades tratadas y, en el caso del SC y el SDiD, también las unidades de control. Para los estimadores DiD y SDiD se emplean 100 replicaciones, número suficiente dado el gran tamaño de muestra ($N_{tr} = 130$). Para el estimador CS, en cambio, se utilizan 2 000 replicaciones: el problema de optimización cuadrática que determina los pesos unitarios introduce mayor variabilidad muestral por iteración, por lo que un número mayor de replicaciones es necesario para obtener un error estándar estable. Los errores estándar *bootstrap* son los reportados entre paréntesis en las Tablas 3, 4 y 5.

4.5. Pruebas de robustez: placebos in-space y sensibilidad del pool de donantes

Más allá de la inferencia *bootstrap*, se reportan dos ejercicios de robustez recomendados por la literatura de Control Sintético y SDiD (Abadie, 2010, 2021; Arkhangelsky et al., 2021) para evaluar la solidez del efecto estimado frente a la elección del grupo de control y frente a la hipótesis nula de no efecto.

El primer ejercicio consiste en una prueba de placebos *in-space*, siguiendo a Abadie (2010, 2021). Para cada uno de los 29 países del grupo de control se asigna, en una corrida separada, un tratamiento placebo y se reestima el SDiD empleando los 28 países de control restantes como pool de donantes, con la muestra restringida al grupo de control original —esto es, omitiendo a los 130 países efectivamente tratados—. Este procedimiento genera una distribución empírica de 29 ATT placebo por lenguaje, bajo la hipótesis nula de que el tratamiento no produce efecto alguno. A partir de dicha distribución, el p-valor placebo no paramétrico se define como la fracción de placebos cuyo ATT, en valor absoluto, iguala o supera al del ATT observado en la especificación principal:

$$p^{\text{placebo}} = \frac{1}{N_c} \sum_{j \in \mathcal{C}} \mathbf{1} \left[|\hat{\tau}_j^{\text{placebo}}| \geq |\hat{\tau}^{\text{sdid}}| \right]$$

Un p-valor bajo indica que el efecto estimado en la especificación principal es atípico respecto a lo que se obtendría asignando el tratamiento aleatoriamente a unidades no expuestas, lo cual robustece la interpretación causal.

El segundo ejercicio evalúa la sensibilidad del resultado al pool de donantes mediante un análisis *leave-one-out*. El objetivo es determinar cuán dependiente es el ATT principal de la presencia de cada país en dicho pool: para cada uno de los 29 países de control se elimina al país j y se reestima el SDID con los 28 países de control restantes más los 130 países tratados originales. Se reportan el mínimo, la mediana y el máximo de las 29 estimaciones *leave-one-out* por lenguaje. Un rango LOO estrecho en torno al ATT principal indica que ningún país individual conduce el resultado, mientras que un rango muy amplio o que cambia de signo revelaría una dependencia excesiva de donantes específicos. La Tabla 6 reúne ambos diagnósticos por lenguaje y la Figura 15 los resume gráficamente.

5. Resultados

En esta sección se presentan los resultados del análisis estadístico sobre el impacto de la disponibilidad de ChatGPT en la actividad de desarrollo de software, enfocándonos en los diez lenguajes de programación más utilizados. Para este análisis, hemos empleado tres enfoques metodológicos robustos: Diferencia en Diferencias, Control Sintético y Diferencia en Diferencias Sintéticas. El objetivo es medir el número de *unique pushers* por cada 100 mil habitantes y evaluar las diferencias entre los países con acceso a ChatGPT y aquellos sin acceso. Los resultados, desglosados por lenguaje de programación y país, proporcionan una perspectiva detallada de cómo ChatGPT ha influido en la productividad de los desarrolladores a lo largo del tiempo. A continuación, se presentan las conclusiones clave, ilustradas mediante figuras comparativas y tablas de estadísticas descriptivas.

La Tabla 3 presenta el impacto de la disponibilidad de ChatGPT en el desarrollo de software por lenguaje de programación. Los resultados se organizan a continuación en tres grupos según la consistencia de la evidencia entre los métodos DiD, CS y SDiD.

En primer lugar, un conjunto de lenguajes muestra efectos positivos y estadísticamente significativos tanto en DiD como en SDiD. Para el lenguaje C, el DiD estima un incremento de 2,689 *unique pushers* por cada 100 mil habitantes ($SE = 0,365$; $p < 0,01$) y el SDiD confirma el efecto con 1,276 ($SE = 0,451$; $p < 0,01$); el CS arroja 1,634 ($SE = 1,137$), sin alcanzar significancia estadística. Para C++, el DiD reporta 2,210 ($SE = 0,417$; $p < 0,01$) y el SDiD, 1,131 ($SE = 0,435$; $p < 0,01$). En Go, el DiD estima 1,411 ($SE = 0,177$; $p < 0,01$) y el SDiD, 0,720 ($SE = 0,269$; $p < 0,01$). Para TypeScript, el DiD es de 7,181 ($SE = 0,832$; $p < 0,01$) y el SDiD, 3,078 ($SE = 0,873$; $p < 0,01$). En Java, el DiD asciende a 2,810 ($SE = 0,827$; $p < 0,01$) y el SDiD, a 1,725 ($SE = 0,872$; $p < 0,05$). En todos estos lenguajes el estimador CS presenta coeficientes positivos que no alcanzan significancia estadística, resultado esperable dado que el método CS es más exigente en términos de ajuste del contrafactual cuando la unidad tratada es un agregado heterogéneo de países.

En segundo lugar, los lenguajes con mayor impacto absoluto son JavaScript, Python y PHP. JavaScript muestra el efecto más elevado: DiD = 13,635 ($SE = 2,719$; $p < 0,01$), SC = 9,451 ($SE = 5,307$; $p < 0,10$) y SDiD = 8,303 ($SE = 3,180$; $p < 0,01$). No obstante, el análisis de evento SDiD para JavaScript (Figura 24) revela una tendencia creciente en los períodos anteriores a Q4-2022, lo que sugiere cautela en la interpretación causal de este resultado. Python registra el segundo mayor efecto: DiD = 7,688 ($SE = 1,193$; $p < 0,01$) y SDiD = 4,525 ($SE = 1,321$; $p < 0,01$). El CS para Python arroja 1,604 ($SE = 3,552$), sin significancia estadística (estadístico $t \approx 0,45$); este resultado refleja la elevada varianza del contrafactual puro cuando la unidad tratada agregada es heterogénea, y no debe interpretarse como ausencia de efecto. En

PHP, el DiD estima 0,904 (SE = 0,507; $p < 0,10$) y el SDiD, 0,916 (SE = 0,465; $p < 0,05$).

Finalmente, tres lenguajes presentan patrones mixtos entre métodos. En C#, el DiD (1,499; SE = 0,381; $p < 0,01$) es significativo, pero el SDiD (0,443; SE = 0,379) no alcanza significancia estadística, posiblemente debido a la mayor varianza introducida por la ponderación temporal del contrafactual sintético. En Ruby, el patrón es inverso: el DiD no es significativo (0,262; SE = 0,266), mientras que el SDiD sí lo es (0,808; SE = 0,274; $p < 0,01$), lo que sugiere que el control sintético construye un mejor contrafactual para este lenguaje. El CS de Ruby (1,521; SE = 0,946) tampoco alcanza significancia estadística.

Por otra parte, de la Figura 5 – Figura 14, se muestran las tendencias y ponderaciones estimadas de *unique pushers* por cada 100 mil personas para diferentes lenguajes de programación, utilizando los métodos Diferencia en Diferencias (DiD), Control Sintético (CS) y Diferencia en Diferencias Sintética (SDiD). Cada figura se divide en seis paneles: los paneles (a) y (b) presentan los resultados del método DiD, los paneles (c) y (d) muestran los resultados del método CS, y los paneles (e) y (f) ilustran los resultados del método SDiD.

En cuanto al ajuste y validación del contrafactual, el método DiD, como se observa en los paneles (a), muestra divergencia en las tendencias para el grupo tratado y el grupo de control antes del tratamiento, indicando que la suposición de tendencias paralelas no se cumple, llevando a estimaciones sesgadas del impacto del tratamiento. Por otro lado, el método CS, ilustrado en los paneles (c), muestra tendencias más similares antes del tratamiento, sugiriendo un mejor ajuste del contrafactual. Sin embargo, la falta de ponderación temporal puede afectar la precisión post-tratamiento. El método SDiD, representado en los paneles (e), muestra tendencias muy similares antes del tratamiento, validando así la construcción del contrafactual. El área sombreada en estos paneles indica la ponderación temporal, destacando la importancia relativa de cada periodo pre-tratamiento y mejorando la precisión del ajuste. La divergencia clara entre las líneas después del tratamiento sugiere un impacto significativo del tratamiento, indicando que SDiD ha proporcionado un ajuste preciso y confiable. Como validación adicional, el Anexo 3 presenta los gráficos de análisis de evento SDiD para los diez lenguajes, donde el gap trimestral entre el grupo tratado y su control sintético se normaliza para que el periodo pre-tratamiento se concentre en torno a cero; la estabilidad de estos gaps pre-tratamiento confirma la adecuación del contrafactual construido.

En el método DiD, representado en los paneles (b), todos los estados de control reciben el mismo peso, lo cual es indicado por la línea roja horizontal constante. Este método no considera la heterogeneidad entre las unidades de control, lo que puede ser una limitación significativa si las tendencias previas al tratamiento no son paralelas. Por otro lado, el método CS, ilustrado en los paneles (d), asigna pesos específicos a las unidades de control, permitiendo una mayor flexibilidad y una mejor adaptación a

las características específicas de las unidades de control. La línea roja horizontal en estos paneles refleja la media ponderada de los pesos asignados, que puede cambiar dependiendo de la distribución de los pesos. En CS, los puntos rojos representan pesos negligibles —pesos inferiores al 30 % del peso uniforme $1/N_c$, esto es, donantes que contribuyen marginalmente al contrafactual sin necesariamente ser exactamente cero— y los puntos azules representan pesos significativos, con el tamaño de los puntos indicando la magnitud del peso. El método SDiD, mostrado en los paneles (f), asigna pesos variables tanto a las unidades de control como a los periodos de tiempo, proporcionando una asignación de pesos más robusta y flexible. La línea roja horizontal en estos paneles también refleja la media ponderada de los pesos, cambiando significativamente debido a la ponderación adicional por el tiempo. En SDiD, al igual que en CS, los puntos rojos representan pesos negligibles y los puntos azules pesos significativos, con el tamaño de los puntos indicando la magnitud del peso.

Al comparar la Figura 14 (C) y la Figura 10 (Ruby), se observa que Ruby muestra una mayor dispersión y variabilidad en la asignación de pesos en comparación con C, y las tendencias pre-tratamiento no cumplen con la suposición de paralelismo tan bien como en C. En Ruby, los puntos rojos que indican pesos negligibles son más frecuentes, sugiriendo que varias unidades de control contribuyen solo marginalmente al contrafactual. Una mayor variabilidad de pesos implica que algunas unidades de control tienen una influencia mucho mayor en la construcción del contrafactual, lo cual puede reflejar una mayor heterogeneidad en las características del grupo de control. Al comparar la Figura 8 (Java) y la Figura 7 (Python), se nota que Python presenta una mayor variabilidad en la asignación de pesos y una divergencia post-tratamiento más marcada. Los puntos en Python indican una mayor heterogeneidad en las unidades de control en comparación con Java. Una menor variabilidad de pesos, como se observa en Java, sugiere que las unidades de control son más homogéneas y que cada una contribuye de manera más equitativa a la construcción del contrafactual. Asimismo, la Figura 12 (C#), Figura 13 (C++), Figura 9 (Go), Figura 6 (JavaScript), Figura 11 (PHP) y Figura 5 (TypeScript) muestran tendencias pre y post-tratamiento relativamente paralelas y homogéneas. Las asignaciones de pesos en CS y SDiD son bastante similares, con poca variabilidad y pocos puntos rojos representando pesos negligibles. Esto sugiere que, para estos lenguajes, las unidades de control son bastante homogéneas y la ponderación temporal no muestra grandes variaciones.

Las pruebas de robustez de los placebos in-space y la sensibilidad del pool de donantes se reúnen en la Tabla 6 y se visualizan en la Figura 15. La distribución de los 29 ATT placebo por lenguaje, obtenida al asignar el tratamiento a un país de control a la vez dentro del subpanel de control, sirve como prueba no paramétrica de la hipótesis nula de no efecto. La Figura 16 presenta los histogramas de placebo por lenguaje, con el ATT observado superpuesto en rojo, lo que permite visualizar gráficamente la

posición del estimador principal frente a la distribución nula.

La lectura conjunta del p-valor placebo y del rango *leave-one-out* permite clasificar la evidencia en tres niveles según su solidez. **Evidencia fuerte:** en Go y Ruby el p-valor placebo es 0,034, lo que ubica al ATT observado fuera del 95 % de la distribución placebo, y el rango LOO se mantiene estrecho y siempre por encima de cero (Go: [0,587; 0,839]; Ruby: [0,667; 0,853]). **Evidencia moderada:** en JavaScript, Python y TypeScript el p-valor placebo es 0,069, marginalmente significativo al 10 %; sus rangos LOO son amplios pero permanecen por encima de cero (JavaScript: [4,642; 9,322]; Python: [3,553; 5,111]; TypeScript: [1,904; 3,598]), lo que sustenta el efecto principal pese a la magnitud de las cifras. **Evidencia débil:** en C, C++, Java, PHP y C# los p-valores placebo se sitúan entre 0,103 y 0,207, es decir, no permiten rechazar la hipótesis nula al 10 % bajo la distribución placebo; sus rangos LOO también son los más estrechos en magnitud relativa al ATT observado pero los placebos sugieren que el efecto detectado podría no ser distinguible de la variabilidad observada al asignar el tratamiento aleatoriamente dentro del grupo de control.

Esta clasificación matiza la lectura de la Tabla 3: la significancia obtenida por *bootstrap* para C, C++ y Java (todos al 1 %) coexiste con una significancia placebo más débil, lo cual sugiere que el ATT estimado para estos lenguajes podría reflejar tanto el tratamiento como heterogeneidad estructural del grupo de control no completamente absorbida por el contrafactual SDID. En contraste, los efectos en JavaScript, Python y TypeScript —los lenguajes priorizados por la hipótesis del estudio dada su mayor representación en datos de entrenamiento del modelo— son los que mejor sobreviven simultáneamente a ambas pruebas, lo que respalda la interpretación causal en estos casos. Importantes para política son por tanto los efectos en Go, Ruby, Python, JavaScript y TypeScript; los demás merecen interpretarse con cautela y como insumo para futuras investigaciones con ventanas postratamiento más amplias.

Conclusiones

En conclusión, los resultados obtenidos mediante el modelo SDID indican que la disponibilidad de ChatGPT ha tenido un impacto positivo y diferenciado en el desarrollo de software en varios lenguajes de programación. Específicamente, se observó un aumento notable en la cantidad de desarrolladores que utilizan lenguajes como JavaScript, Python y TypeScript, hallazgo que la hipótesis del estudio (Sección 3.3) había anticipado explícitamente: estos son justamente los lenguajes con mayor representación de documentación, tutoriales y consultas en plataformas abiertas, lo cual constituye la base de entrenamiento sobre la que ChatGPT despliega mejor su capacidad de asistencia (Buscemi, 2023; Chai et al., 2024). La confirmación empírica de esta predicción teórica robustece la interpretación del mecanismo subyacente: la disponibilidad del modelo reduce los costos de búsqueda y aprendizaje (Stigler, 1961) y permite la entrada al desarrollo de programadores que antes enfrentaban barreras de capital humano demasiado altas (Acemoglu y Restrepo, 2018), precisamente en aquellos lenguajes donde la productividad del modelo es mayor.

Las pruebas de robustez incorporadas estratifican la fortaleza de la evidencia en tres niveles: fuerte para Go y Ruby (p -valor placebo $\leq 0,05$), moderada para JavaScript, Python y TypeScript ($p \leq 0,10$) y débil para C, C++, Java, PHP y C# ($p > 0,10$). En conjunto, los lenguajes para los cuales la evidencia es simultáneamente significativa por *bootstrap*, robusta a la elección del pool de donantes y atípica respecto a la distribución placebo son los que la teoría priorizaba ex ante y, por tanto, los que deben fundamentar las recomendaciones de política. Lenguajes como C# no mostraron un impacto distinguible de la variabilidad placebo, lo que sugiere que factores específicos del lenguaje o del contexto de uso —su predominancia en ecosistemas corporativos cerrados (.NET, Windows) con menor interacción con foros abiertos— pueden moderar la efectividad de ChatGPT. El impacto, además, muestra variaciones entre contextos nacionales, lo que resalta que factores como infraestructura tecnológica, adopción y políticas de apoyo desempeñan un papel relevante en su magnitud.

Por tanto, aunque ChatGPT no constituye un asistente de codificación perfecto y presenta desafíos en cuanto a precisión y calidad del código generado, la evidencia muestra que su disponibilidad ha contribuido de manera significativa al crecimiento del número de desarrolladores activos en el campo de la ciencia de datos. Esta herramienta ha facilitado la entrada de nuevos programadores, ha acelerado la resolución de problemas comunes y ha promovido una mayor participación en plataformas colaborativas como GitHub. Así, su impacto positivo en el ecosistema de desarrollo justifica una discusión profunda sobre su integración responsable en los flujos de trabajo tecnológicos y su papel en la configuración futura del trabajo en la economía digital.

En línea con los resultados obtenidos, y reconociendo que la ventana postra-

tamiento disponible es de solo cuatro trimestres (Q4-2022 a Q4-2023) lo que limita la solidez de inferencias de largo plazo, se proponen las siguientes recomendaciones de política como punto de partida sujeto a validación con datos futuros. Una primera recomendación consiste en dotar de acceso gratuito a versiones avanzadas de ChatGPT para desarrolladores junior orientados a ciencia de datos dentro de empresas vinculadas al sector tecnológico. Esta medida podría implementarse mediante incentivos para que las empresas inviertan en la adquisición de suscripciones a la versión premium de la herramienta por periodos iniciales de seis meses renovables. La evidencia sugiere que los desarrolladores juniors utilizan ChatGPT principalmente para resolver consultas básicas y superar barreras iniciales en el proceso de programación, mientras que, a medida que adquieren mayor experiencia, continúan utilizándolo para acelerar su productividad. Asimismo, las versiones avanzadas del modelo presentan mayores capacidades de asistencia en programación, lo que puede facilitar el desarrollo de tareas técnicas y mejorar la integración de nuevos programadores en entornos laborales reales. El periodo de seis meses permitiría evaluar razonablemente la evolución del desempeño del trabajador, por lo que se recomienda establecer criterios de elegibilidad verificables, como la exigencia de cuentas de GitHub validadas y con actividad reciente, permitiendo observar la generación de código a lo largo del tiempo y asegurar que el beneficio se dirija efectivamente a perfiles junior.

De manera complementaria, se recomienda implementar incentivos a la productividad asociados al uso de herramientas de inteligencia artificial dentro de las empresas del sector software, mediante la asignación de bonos económicos a trabajadores que evidencien mejoras en su rendimiento atribuibles al uso de estas tecnologías. El monto del incentivo podría ser determinado por cada empresa según sus objetivos y capacidades internas. Para garantizar una evaluación objetiva, el acceso a este beneficio debería sustentarse en mecanismos de seguimiento bimensuales que consideren indicadores como la cantidad de horas de uso de herramientas de inteligencia artificial, el volumen de tareas completadas con su asistencia y el cumplimiento de estándares mínimos de calidad establecidos por los equipos de aseguramiento de calidad (Quality Assurance, QA). De esta forma, ambas medidas contribuirían a fomentar una adopción productiva y responsable de la inteligencia artificial, fortaleciendo la participación de desarrolladores en el ecosistema tecnológico y promoviendo mejoras medibles en el desempeño laboral.

El presente estudio tiene varias limitaciones que deben tenerse en cuenta al interpretar los resultados. En primer lugar, la composición del grupo de control constituye la principal amenaza a la validez interna del diseño. Los países que bloquearon el acceso a ChatGPT no lo hicieron de forma aleatoria: en su mayoría presentan mayor control estatal sobre internet, menores niveles de libertad digital y ecosistemas tecnológicos estructuralmente distintos a los del grupo tratado (ej. China, Rusia, Irán). Esta

no-aleatoriedad puede generar un sesgo de selección en la estimación del ATT que el SDiD mitiga, pero no elimina por completo. Como verificación de robustez, la Tabla 5 reporta las estimaciones SDiD excluyendo del grupo de control los seis censores severos de internet —China, Cuba, Irán, Bielorrusia, Rusia y Siria, clasificados como “No Libres” por Freedom House (2022)— y los resultados son cualitativamente similares a los de la especificación principal, lo que sugiere que la presencia de estos países no distorsiona las estimaciones de forma significativa. Futuros trabajos deberían explorar estrategias de identificación alternativas que permitan controlar más directamente por estas diferencias estructurales.

En segundo lugar, existe un problema de interferencia en el tratamiento derivado del uso de VPN. Una proporción no despreciable de desarrolladores en países clasificados como “sin acceso” puede haber utilizado redes privadas virtuales para acceder a ChatGPT durante el período analizado. Si este fenómeno es generalizado, especialmente en países tecnológicamente activos como China o Rusia, el efecto estimado estaría subestimado (atenuación hacia cero), ya que la distinción entre grupo tratado y de control sería menos nítida de lo que el diseño asume.

En tercer lugar, la ventana postratamiento es corta: solo cuatro períodos (Q4-2022 a Q4-2023). Este horizonte dificulta separar el efecto causal de ChatGPT de una tendencia general de expansión del ecosistema de software que ya venía observándose, y que se vio reforzada en 2023 por la proliferación simultánea de otras herramientas de IA (GitHub Copilot, Bard, entre otras). Un seguimiento más prolongado permitiría identificar con mayor precisión efectos de largo plazo.

Vinculado a lo anterior, una preocupación específica es la presencia de GitHub Copilot durante el período pretratamiento. Copilot fue lanzado en *technical preview* en junio de 2021 y como producto comercial en junio de 2022, es decir, antes de Q4-2022. Si su disponibilidad estuvo también restringida en los países del grupo de control —algo plausible al compartir parcialmente el mapa geopolítico de OpenAI— entonces la falta de tendencias paralelas observada en el período pretratamiento podría agravarse, pues estaría contaminada por una primera ola de asistencia con IA. No obstante, las diferencias de escala entre ambas herramientas actúan a favor de la identificación: Copilot fue desde el inicio un producto pagado (10 USD mensuales para individuos, 19 USD para empresas) integrado en editores de código y orientado a desarrolladores profesionales, mientras que ChatGPT fue lanzado como servicio gratuito con interfaz conversacional masiva, lo que produjo una adopción cualitativamente distinta y mucho más rápida en magnitud (millones de usuarios en las primeras semanas). El *shock* relevante para este estudio es, por tanto, la disponibilidad de la herramienta a escala masiva en Q4-2022, no la existencia de asistencia con IA *per se*.

En cuarto lugar, la divergencia entre métodos observada para algunos lenguajes y, sobre todo, la heterogeneidad en la fortaleza de la evidencia revelada por las pruebas

de robustez de la Tabla 6 imponen una interpretación cautelosa. En Ruby, el DiD no resulta significativo (0,262) mientras que el CS y el SDiD sí lo son (1,522 y 0,808, respectivamente), pero la prueba de placebo confirma el efecto ($p = 0,034$), lo que sugiere que el control sintético construye un mejor contrafactual para ese lenguaje. En C#, el DiD y el CS son significativos (1,499 y 1,001, respectivamente) pero el SDiD (0,443) no alcanza significancia estadística y el p-valor placebo (0,207) refuerza esa lectura, sugiriendo ausencia de efecto detectable. Para Python el CS (1,604) no resulta significativo aunque DiD (7,688) y SDiD (4,525) sí lo son; el p-valor placebo (0,069) y el rango LOO estrecho confirman que la elevada varianza del CS responde a la heterogeneidad de la unidad tratada agregada y no a un efecto espurio.

Más generalmente, la prueba de placebo *in-space* estratifica la evidencia en tres categorías —fuerte (Go, Ruby), moderada (JavaScript, Python, TypeScript) y débil (C, C++, Java, PHP, C#)— y sugiere que la significancia obtenida por *bootstrap* para los lenguajes de la categoría débil podría reflejar heterogeneidad estructural del grupo de control no absorbida completamente por el contrafactual SDiD. Esto no invalida las estimaciones de los lenguajes débiles, pero sí implica que solo los efectos sobre lenguajes priorizados por la teoría del estudio (JavaScript, Python, TypeScript, Go y Ruby) deberían considerarse como evidencia robusta para la formulación de política. Los demás permanecen como hallazgos exploratorios que ameritan replicación con ventanas postratamiento más amplias.

Finalmente, la variable dependiente (*unique pushers* en GitHub) mide cantidad de desarrolladores activos, pero no captura la intensidad del trabajo, la calidad del código ni el tipo de contribuciones realizadas. Además, los países del grupo de control con plataformas de control de versiones alternativas (como Gitee en China) podrían mostrar menor actividad en GitHub independientemente de ChatGPT, lo que introduciría un sesgo adicional en la comparación.

Tablas

Tabla 1

Estadísticos descriptivos del número de *unique pushers* por cada 100 mil habitantes según lenguaje de programación en países del grupo de control (periodos pre y postratamiento)

Lenguaje	Pretratamiento				Postratamiento			
	Media	Mínimo	Máximo	Desv. Est.	Media	Mínimo	Máximo	Desv. Est.
C	4,88	0,00	210,10	22,41	8,73	0,00	215,00	36,00
C#	3,01	0,00	61,62	8,33	4,41	0,00	60,82	11,48
C++	5,28	0,00	211,28	22,82	8,84	0,00	216,88	36,25
Go	1,74	0,00	149,67	12,29	4,85	0,00	142,99	24,10
Java	6,59	0,00	271,23	25,82	10,81	0,00	233,03	39,55
JavaScript	18,84	0,00	640,90	63,46	34,29	0,00	605,37	102,79
PHP	2,27	0,00	63,75	7,88	3,46	0,00	48,18	8,36
Python	13,30	0,00	454,19	49,96	22,21	0,00	482,93	79,05
Ruby	3,13	0,00	108,09	12,00	4,00	0,00	102,32	17,21
TypeScript	3,46	0,00	224,19	17,55	11,79	0,00	275,74	45,14

Nota. Estadísticos descriptivos calculados a partir de la base de datos GitHub Innovation Graph (tabla *languages*), que registra el número de *unique pushers* por cada 100 000 habitantes, desagregado por país, lenguaje de programación y trimestre. El periodo de pretratamiento comprende Q1-2020 a Q3-2022 (12 trimestres) y el de postratamiento Q4-2022 a Q4-2023 (4 trimestres). El grupo de control incluye los 29 países sin acceso a ChatGPT al momento de su lanzamiento en noviembre de 2022. Fuente: GitHub Innovation Graph (<https://github.com/github/innovationgraph>). Elaboración propia.

Tabla 2

Estadísticos descriptivos del número de *unique pushers* por cada 100 mil habitantes según lenguaje de programación en países del grupo de tratamiento (periodos pre y postratamiento)

Lenguaje	Pretratamiento				Postratamiento			
	Media	Mínimo	Máximo	Desv. Est.	Media	Mínimo	Máximo	Desv. Est.
C	7,16	0,00	82,05	10,91	10,91	0,00	117,10	14,88
C#	7,39	0,00	62,97	10,45	9,64	0,00	65,53	12,53
C++	7,77	0,00	85,60	11,64	10,82	0,00	115,69	15,10
Go	3,63	0,00	47,68	6,83	4,05	0,00	76,18	7,91
Java	13,58	0,00	118,42	16,28	16,14	0,00	156,79	20,07
JavaScript	51,00	0,00	348,52	56,30	58,98	0,00	455,33	62,87
PHP	7,54	0,00	39,27	8,29	8,24	0,00	39,27	8,42
Python	22,71	0,00	270,36	33,47	33,46	0,00	365,32	46,23
Ruby	6,07	0,00	84,47	10,00	6,28	0,00	84,47	10,01
TypeScript	17,32	0,00	181,23	21,61	18,83	0,00	181,23	23,56

Nota. Estadísticos descriptivos calculados a partir de la base de datos GitHub Innovation Graph (tabla *languages*), que registra el número de *unique pushers* por cada 100 000 habitantes, desagregado por país, lenguaje de programación y trimestre. El periodo de pretratamiento comprende Q1-2020 a Q3-2022 (12 trimestres) y el de postratamiento Q4-2022 a Q4-2023 (4 trimestres). El grupo de tratamiento incluye los 130 países con acceso a ChatGPT al momento de su lanzamiento en noviembre de 2022. Fuente: GitHub Innovation Graph (<https://github.com/github/innovationgraph>). Elaboración propia.

Tabla 3
Impacto de ChatGPT en el número de programadores

Lenguaje	DID	SC	SDID	Obs.	Baseline Mean
C	2,689*** (0,365)	1,634 (1,137)	1,276*** (0,451)	2544	6,232
C#	1,499*** (0,381)	1,001 (1,263)	0,443 (0,379)	2544	6,557
C++	2,210*** (0,417)	1,212 (0,959)	1,131*** (0,435)	2544	6,807
Go	1,411*** (0,177)	0,686 (0,606)	0,720*** (0,269)	2544	2,230
Java	2,810*** (0,827)	2,206 (2,089)	1,725** (0,872)	2544	11,467
JavaScript	13,635*** (2,719)	9,451* (5,307)	8,303*** (3,180)	2544	36,571
PHP	0,904* (0,507)	1,078 (1,013)	0,916** (0,465)	2544	6,381
Python	7,688*** (1,193)	1,604 (3,552)	4,525*** (1,321)	2544	20,003
Ruby	0,262 (0,266)	1,521 (0,946)	0,808*** (0,274)	2544	5,592
TypeScript	7,181*** (0,832)	3,263 (2,290)	3,078*** (0,873)	2544	8,571

Nota. Estimaciones del efecto promedio del tratamiento (ATT) del acceso a ChatGPT sobre el número de *unique pushers* por cada 100 000 habitantes. Las columnas DID, SC y SDID corresponden a Diferencias en Diferencias, Control Sintético (Abadie et al., 2010) y Diferencias en Diferencias Sintética (Arkhangelsky et al., 2021), respectivamente, todos estimados mediante la librería *synthdid* de Python (Clarke et al., 2023). Los errores estándar se obtienen por *bootstrap* (100 replicaciones). El periodo de tratamiento inicia en Q4-2022, coincidiendo con el lanzamiento de ChatGPT. Fuente: GitHub Innovation Graph, tabla *languages* (<https://github.com/github/innovationgraph>). Elaboración propia. Errores estándar entre paréntesis. * p<0.10, ** p<0.05, *** p<0.01

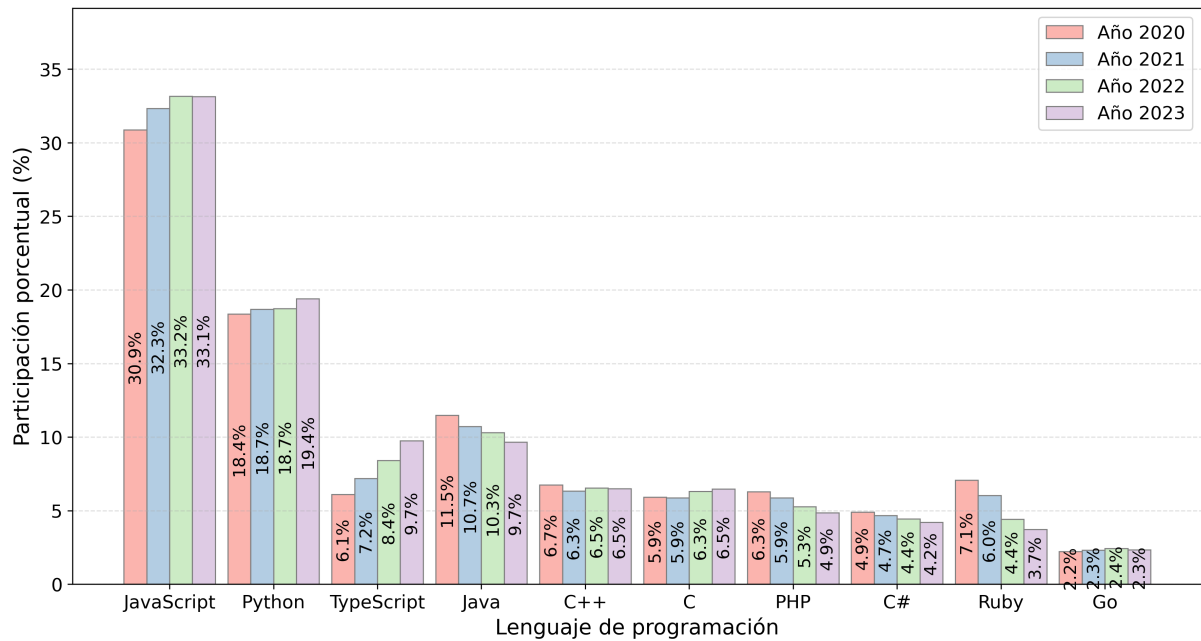
Tabla 6
Robustez SDID: placebos in-space y sensibilidad del pool de donantes (LOO)

Lenguaje	ATT observado	Media placebos	p-valor placebo	LOO mín.	LOO mediana	LOO máx.
C	1,276	0,039	0,103	1,095	1,278	1,436
C#	0,443	-0,010	0,207	0,411	0,435	0,824
C++	1,131	0,071	0,138	1,016	1,122	1,278
Go	0,720	-0,007	0,034	0,587	0,721	0,839
Java	1,725	-0,065	0,103	1,072	1,726	2,646
JavaScript	8,303	-0,161	0,069	4,642	8,300	9,322
PHP	0,916	-0,243	0,172	0,415	0,909	1,124
Python	4,525	0,355	0,069	3,553	4,520	5,111
Ruby	0,808	-0,009	0,034	0,667	0,807	0,853
TypeScript	3,078	0,022	0,069	1,904	3,081	3,598

Nota. Pruebas de robustez del estimador SDID. *Placebos in-space*: para cada uno de los 29 países del grupo de control se le asigna un tratamiento placebo y se reestima el SDID restringiendo la muestra al grupo de control; el p-valor placebo es la fracción de placebos cuyo $|ATT|$ es mayor o igual al $|ATT|$ observado, una prueba no paramétrica a la Abadie (2010, 2021). *Donor pool LOO*: se elimina un país de control a la vez del pool de donantes y se reestima el SDID con los 28 países restantes más los 130 países tratados; las columnas reportan el mínimo, mediana y máximo de las 29 estimaciones *leave-one-out* (Arkhangelsky et al., 2021). Fuente: GitHub Innovation Graph (<https://github.com/github/innovationgraph>). Elaboración propia.

Figuras

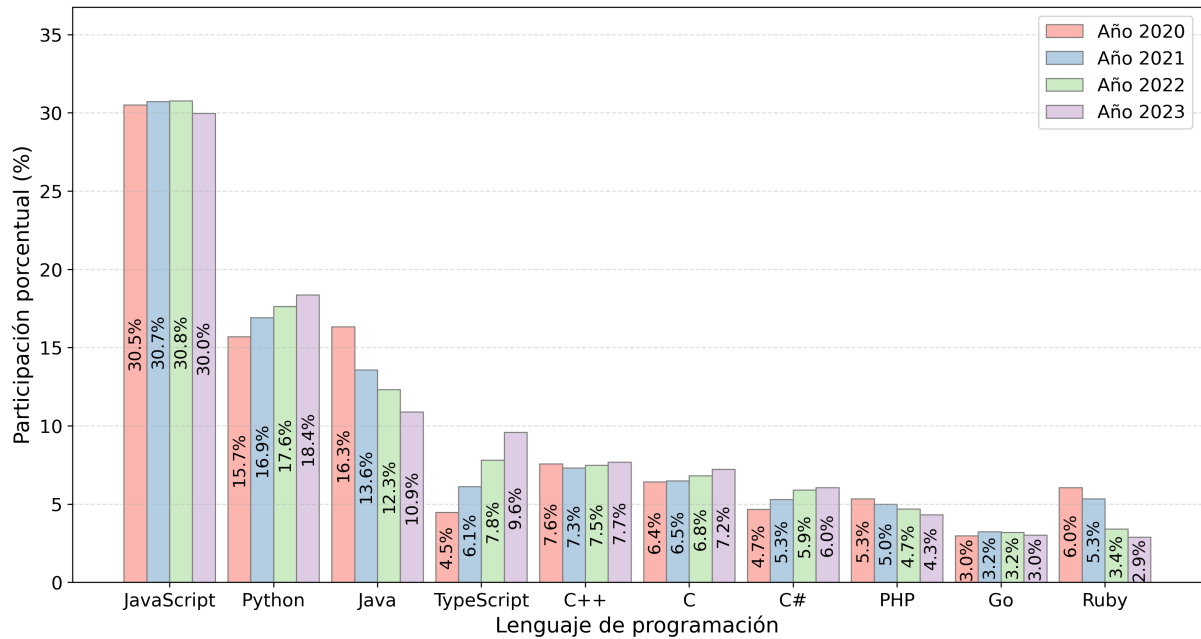
Figura 1
Distribución de lenguajes de programación más populares en países con ChatGPT disponible (2020-2023)



Nota. La figura presenta una distribución porcentual de la participación de los lenguajes más usados en ciencia de datos en los países cuyo acceso a ChatGPT está vigente. La base de datos proviene de GitHub Innovation Graph y, para este caso, se han usado un total de 21280 observaciones. Por otra parte, la cantidad de lenguajes de programación filtrados y usados son 10 y el número de años incluidos es 4. Respecto a la figura, se puede identificar un aumento constante de los lenguajes JavaScript, Python y TypeScript entre 2020 y 2023, siendo los más usados por los desarrolladores en esos períodos de tiempo, mientras que C#, Ruby y Go resaltan por ser los menos utilizados en el mismo segmento temporal. Elaboración propia con datos de GitHub (2020–2023).

Figura 2

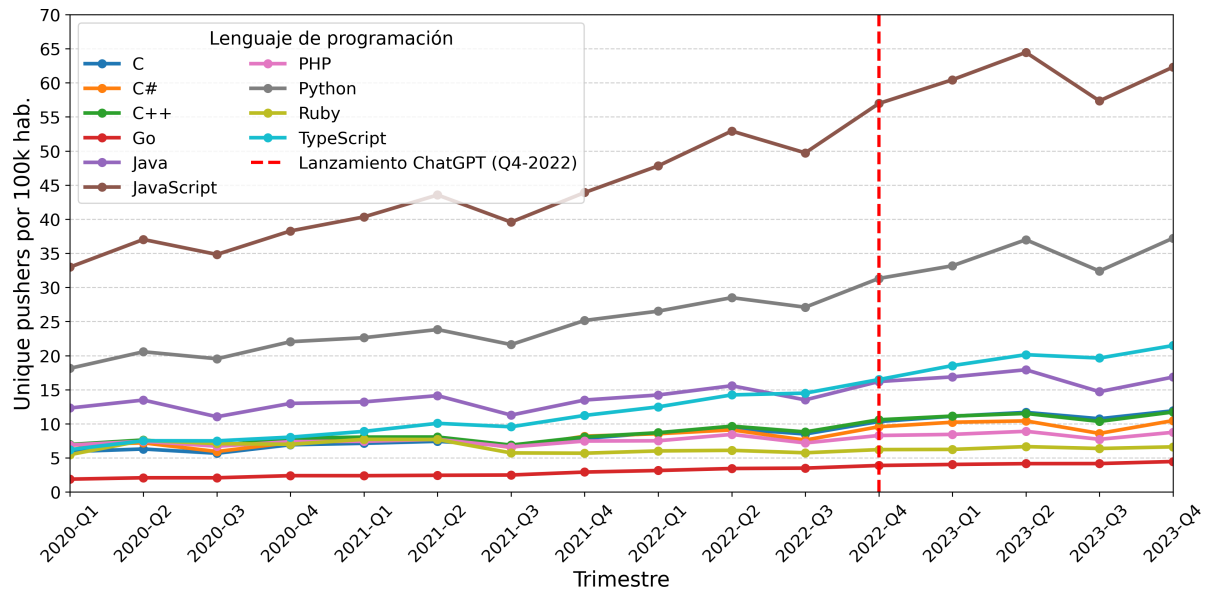
Distribución de lenguajes de programación más populares en países sin ChatGPT disponible (2020-2023)



Nota. La figura presenta una distribución porcentual de la participación de los lenguajes más usados en ciencia de datos en los países cuyo acceso a ChatGPT está restringido. La base de datos proviene de GitHub Innovation Graph y, para este caso, se han usado un total de 5120 observaciones. Asimismo, la cantidad de lenguajes de programación filtrados y usados son 10 y el número de años incluidos es 4. Respecto a la figura, se puede identificar un aumento casi constante de los lenguajes JavaScript y Python entre 2020 y 2023, siendo los más usados por los desarrolladores en esos períodos de tiempo, mientras que PHP, Go y Ruby destacan por ser los menos utilizados en el mismo segmento temporal. Elaboración propia con datos de GitHub (2020–2023).

Figura 3

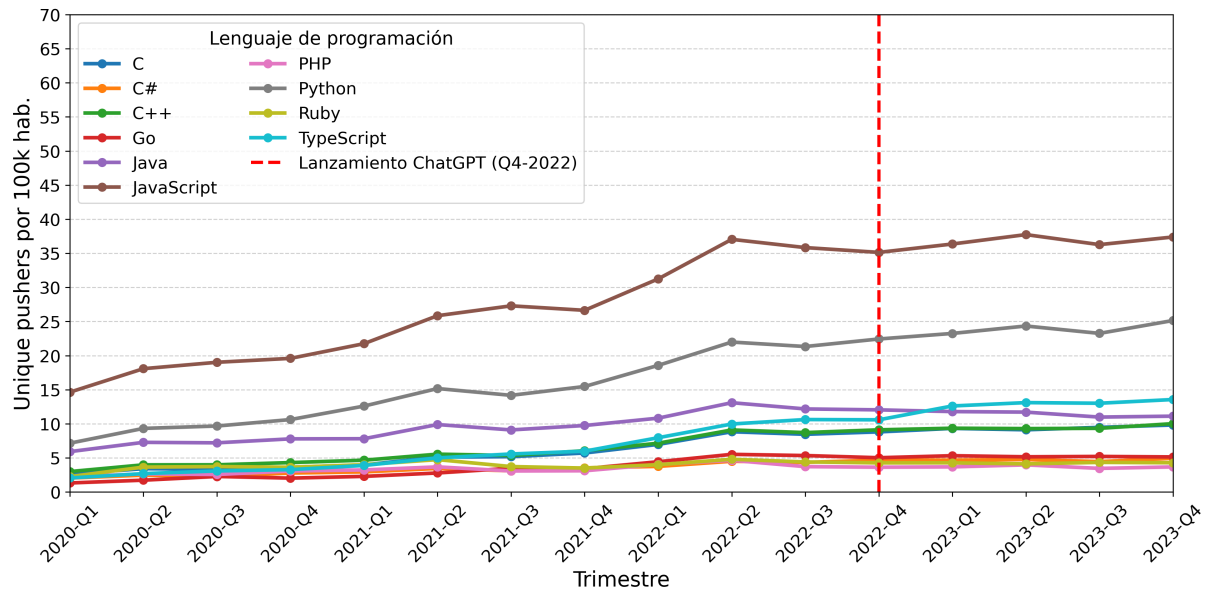
Distribución de *unique pushers* por cada 100 mil habitantes según lenguaje en países con ChatGPT (2020–2023)



Nota. La figura muestra una serie temporal sobre la dinámica de los *unique pushers* de cada lenguaje de programación en países cuyo acceso a ChatGPT está disponible. La base de datos proviene de GitHub Innovation Graph y, para este caso, se han usado un total de 160 observaciones. Además, la cantidad de lenguajes de programación filtrados y usados son 10 y el número de trimestres analizados es 16. Sobre la figura, se puede observar un aumento alto y progresivo del número de desarrolladores que al menos han hecho un push en GitHub por cada 100 mil personas para los lenguajes de JavaScript, Python y Java, siendo los dos primeros con mayor representación. Los lenguajes restantes obtuvieron crecimientos muy ligeros o se mantuvieron constantes a lo largo del período en cuestión. Elaboración propia con datos de GitHub (2020–2023).

Figura 4

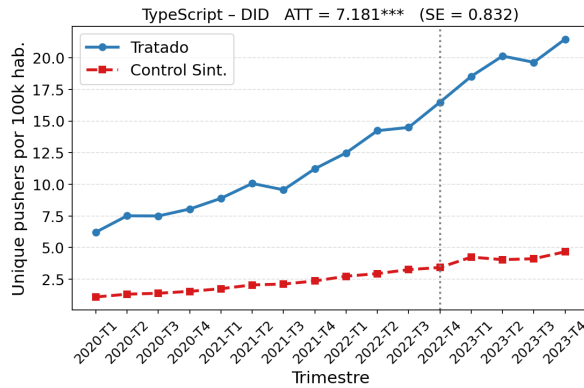
Distribución de *unique pushers* por cada 100 mil habitantes según lenguaje en países sin ChatGPT (2020–2023)



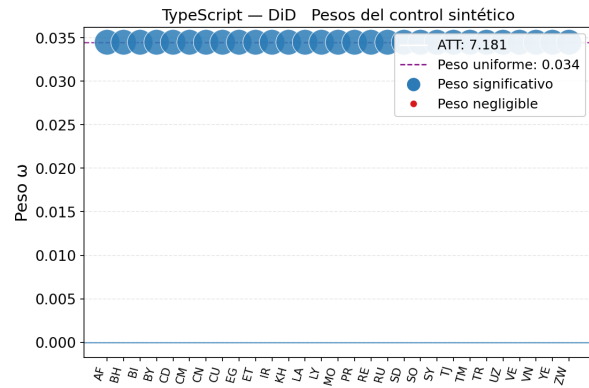
Nota. La figura muestra una serie temporal sobre la dinámica de los *unique pushers* de cada lenguaje de programación en países cuyo acceso a ChatGPT está restringido. La base de datos proviene de GitHub Innovation Graph y, para este caso, se han usado un total de 160 observaciones. Además, la cantidad de lenguajes de programación filtrados y usados son 10 y el número de trimestres analizados es 16. Sobre la figura, se puede observar un aumento mediano y progresivo del número de desarrolladores que al menos han hecho un push en GitHub por cada 100 mil personas para los lenguajes de JavaScript, Python y TypeScript, siendo los dos primeros con mayor representación. Los lenguajes restantes obtuvieron crecimientos muy ligeros, se mantuvieron constantes o cayeron a lo largo del período en cuestión. Elaboración propia con datos de GitHub (2020–2023).

Figura 5

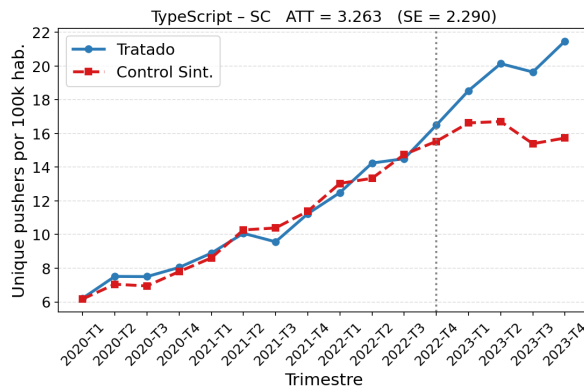
Tendencias y ponderaciones estimadas de TypeScript *unique pushers* por cada 100 mil personas



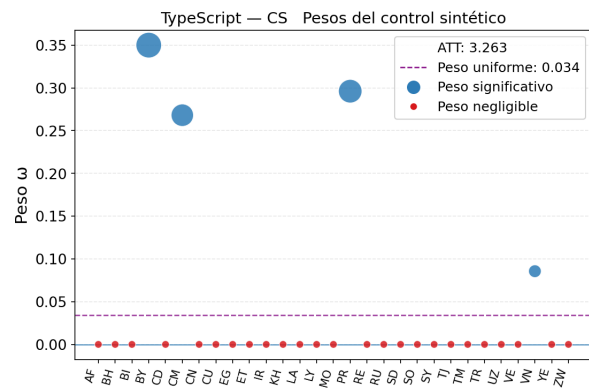
(a) Tendencias estimadas de DiD



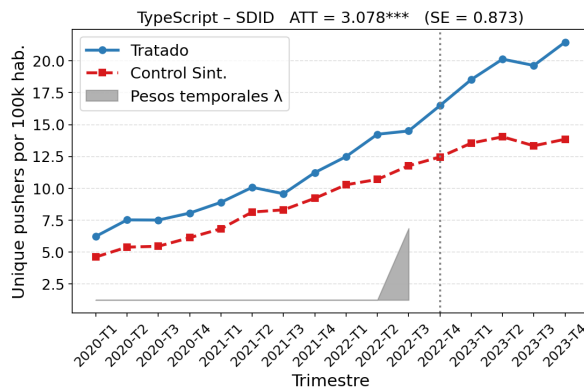
(b) Pesos DiD estimados



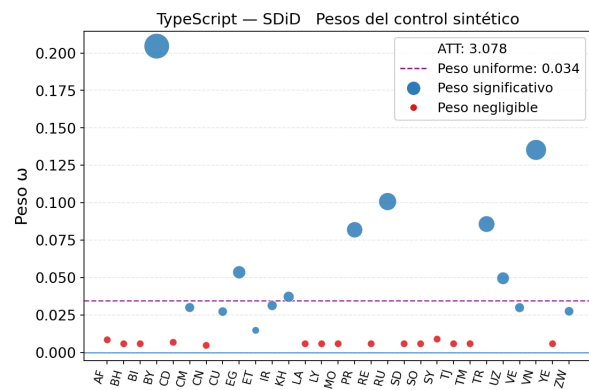
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



(e) Tendencias estimadas de SDiD

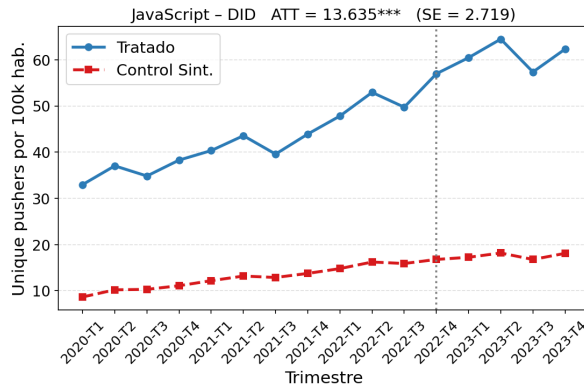


(f) Pesos SDiD estimados

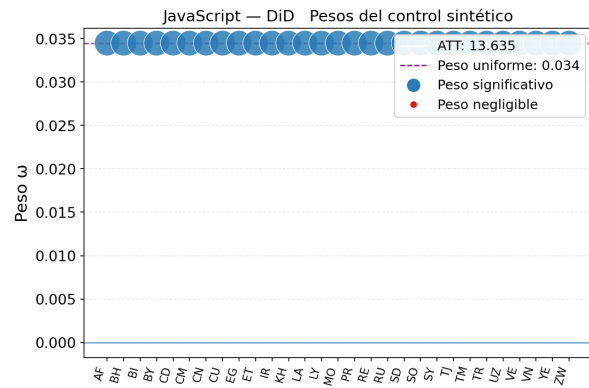
Nota. Elaboración propia.

Figura 6

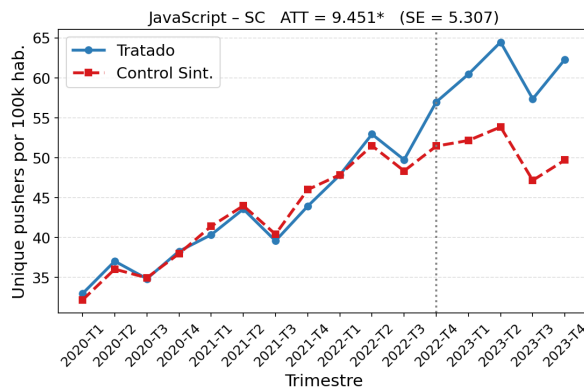
Tendencias y ponderaciones estimadas de JavaScript *unique pushers* por cada 100 mil personas



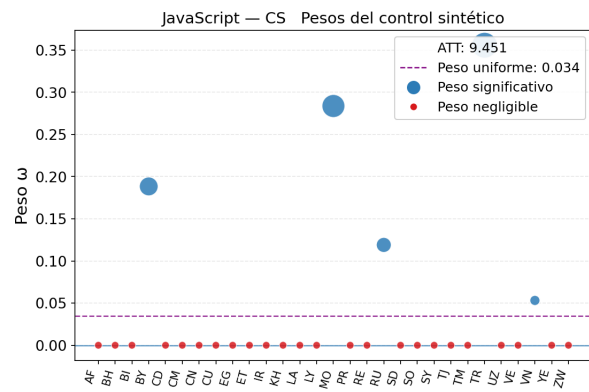
(a) Tendencias estimadas de DiD



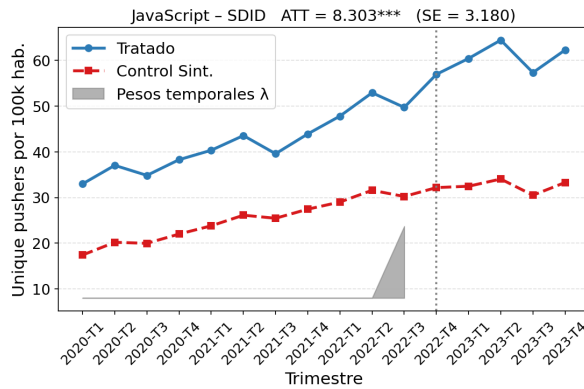
(b) Pesos DiD estimados



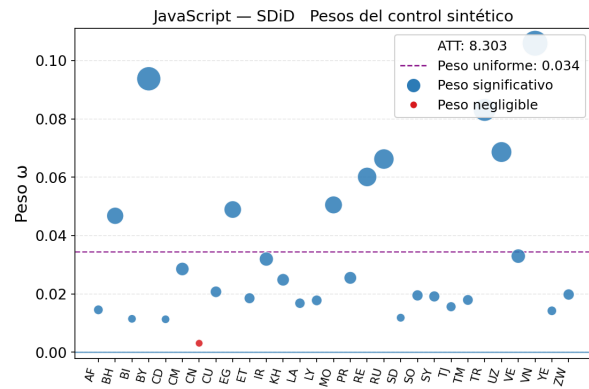
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



(e) Tendencias estimadas de SDiD

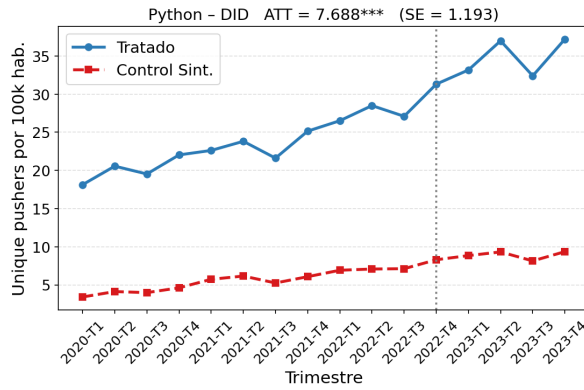


(f) Pesos SDiD estimados

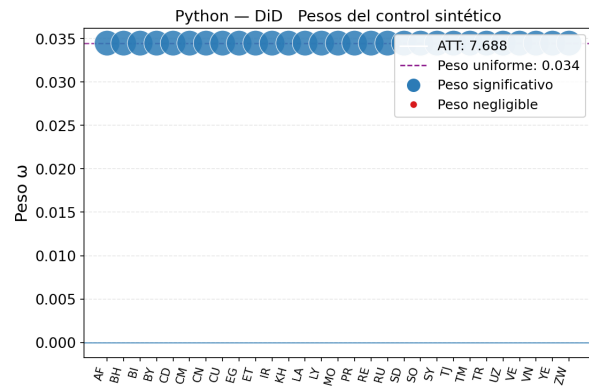
Nota. Elaboración propia.

Figura 7

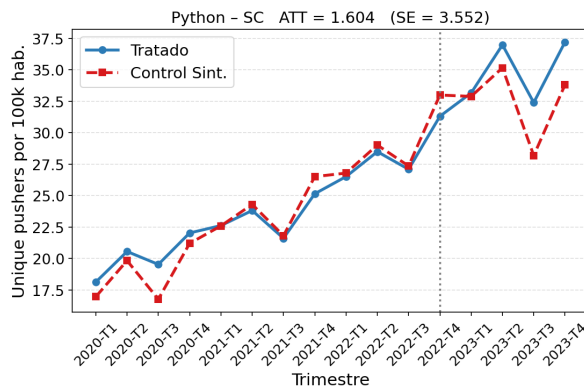
Tendencias y ponderaciones estimadas de Python *unique pushers* por cada 100 mil personas



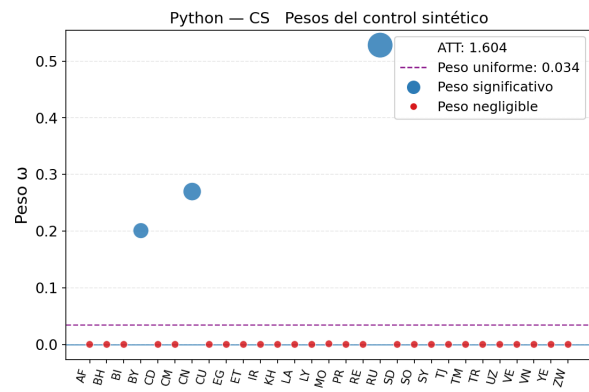
(a) Tendencias estimadas de DiD



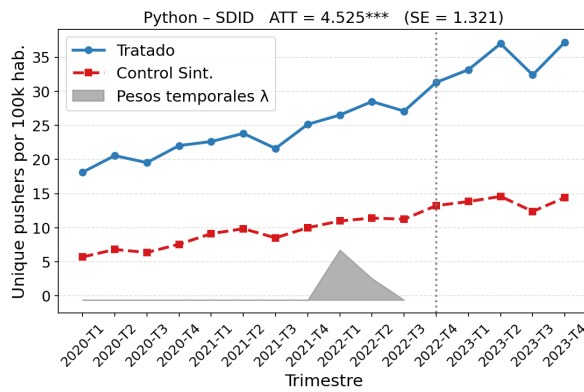
(b) Pesos DiD estimados



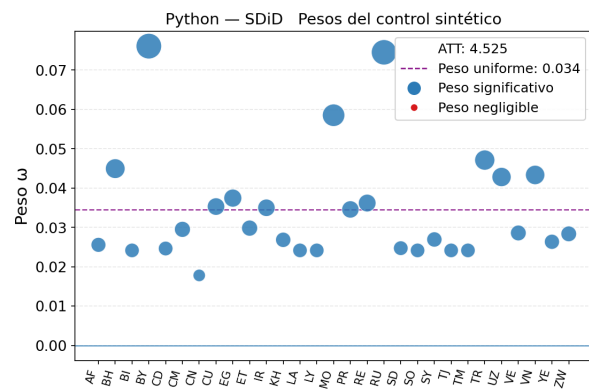
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



(e) Tendencias estimadas de SDiD

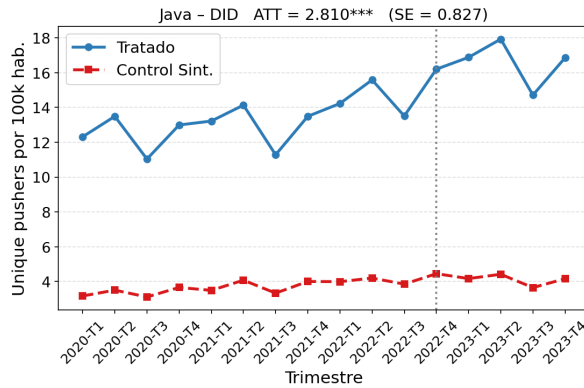


(f) Pesos SDiD estimados

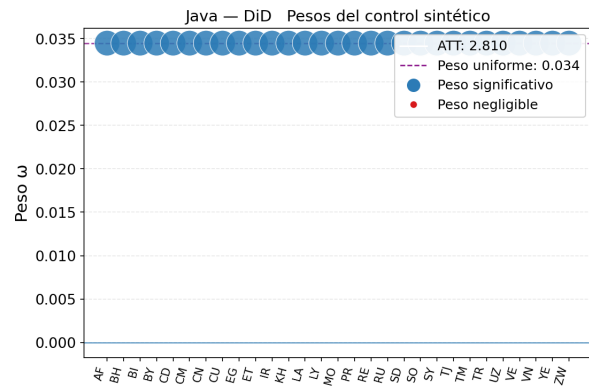
Nota. Elaboración propia.

Figura 8

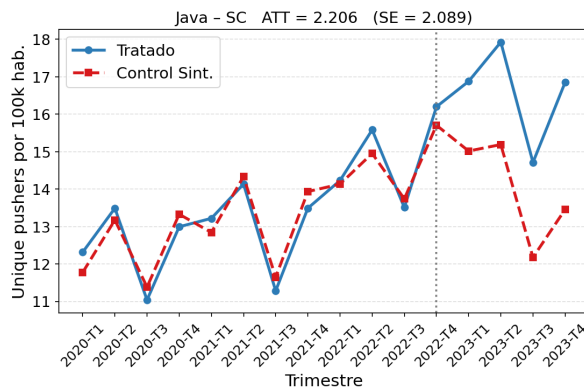
Tendencias y ponderaciones estimadas de Java *unique pushers* por cada 100 mil personas



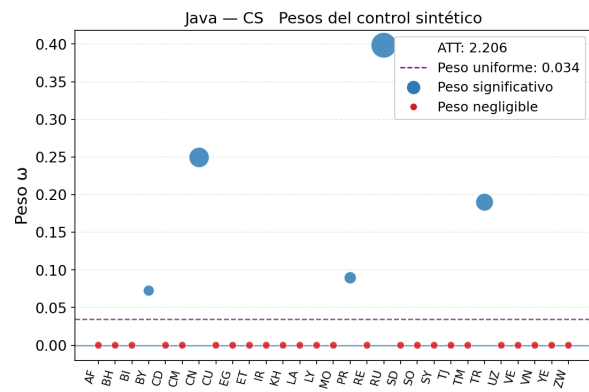
(a) Tendencias estimadas de DiD



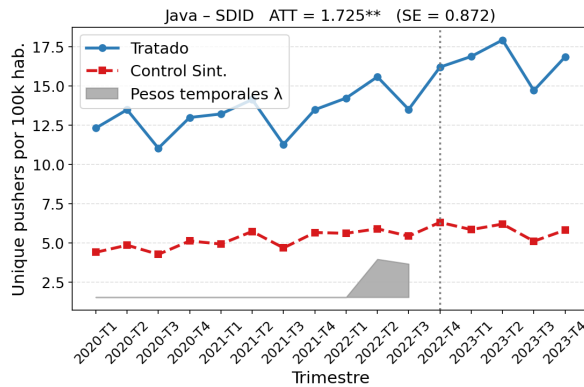
(b) Pesos DiD estimados



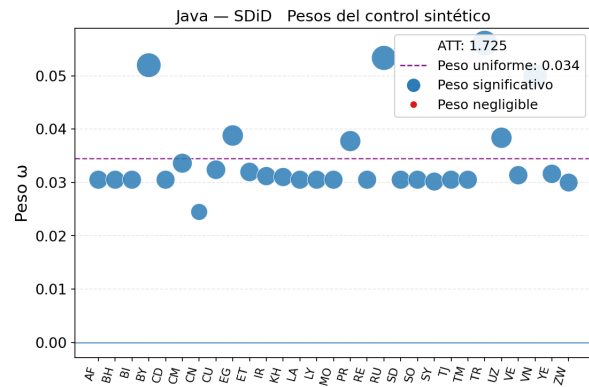
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



(e) Tendencias estimadas de SDiD

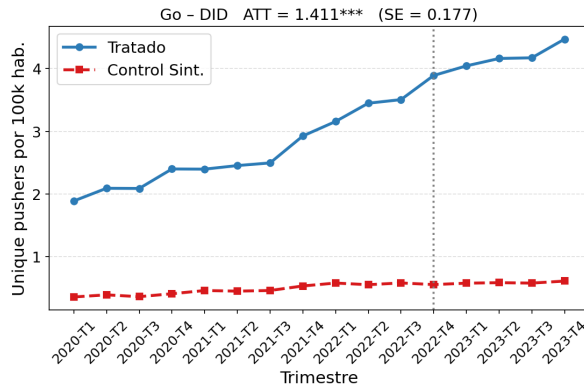


(f) Pesos SDiD estimados

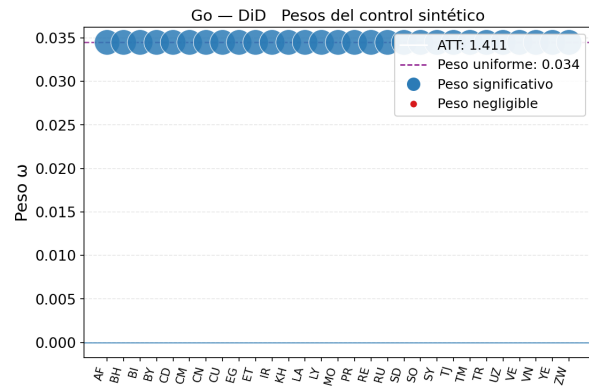
Nota. Elaboración propia.

Figura 9

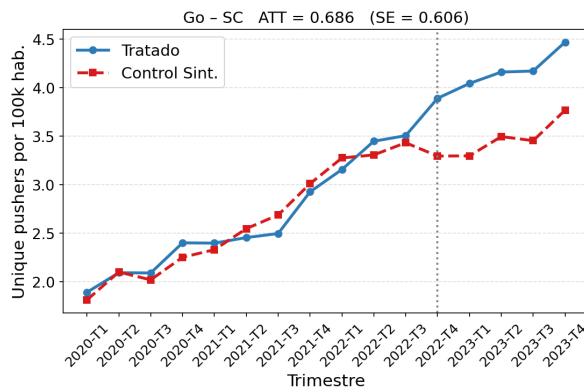
Tendencias y ponderaciones estimadas de *Go unique pushers* por cada 100 mil personas



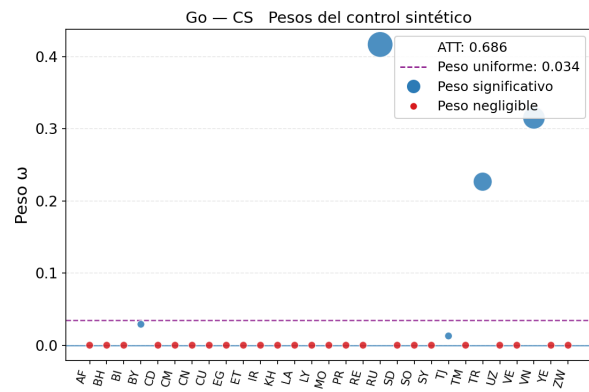
(a) Tendencias estimadas de DiD



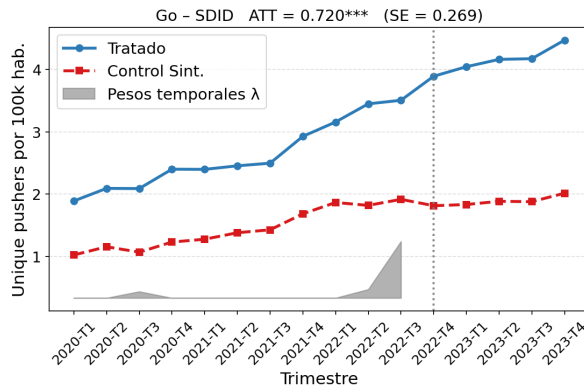
(b) Pesos DiD estimados



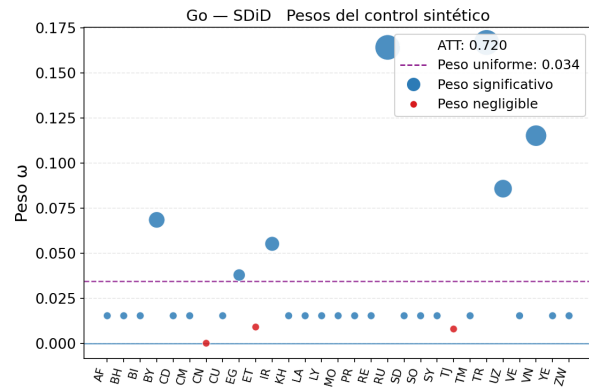
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



(e) Tendencias estimadas de SDiD

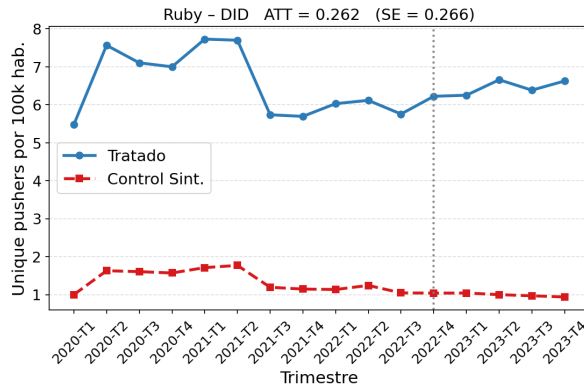


(f) Pesos SDiD estimados

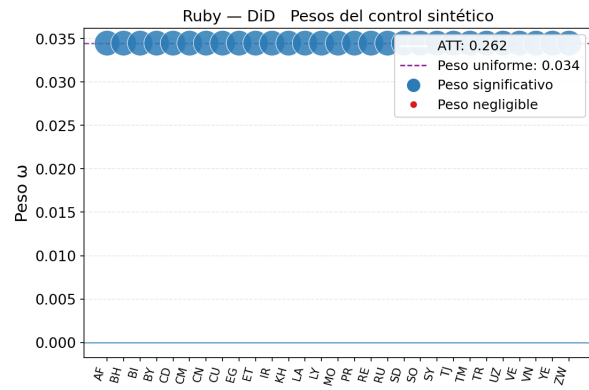
Nota. Elaboración propia.

Figura 10

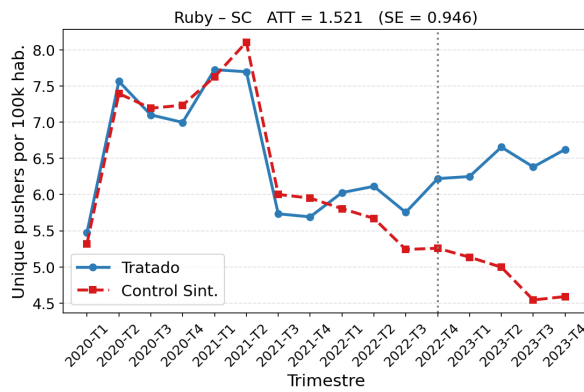
Tendencias y ponderaciones estimadas de Ruby *unique pushers* por cada 100 mil personas



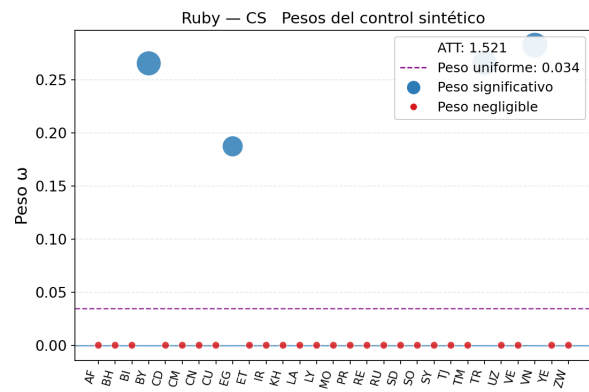
(a) Tendencias estimadas de DiD



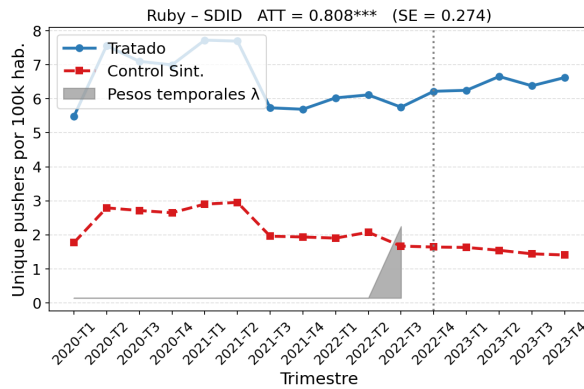
(b) Pesos DiD estimados



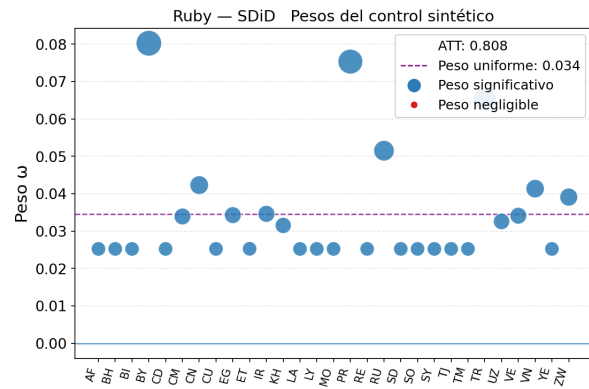
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



(e) Tendencias estimadas de SDiD

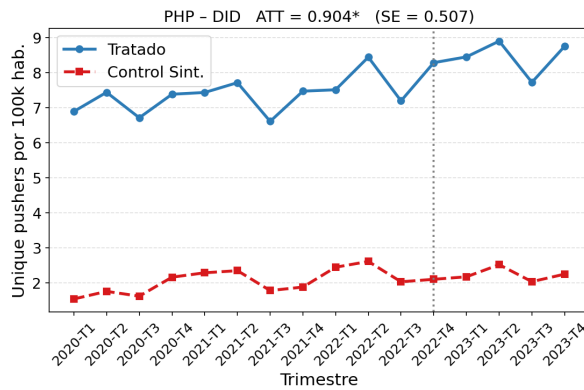


(f) Pesos SDiD estimados

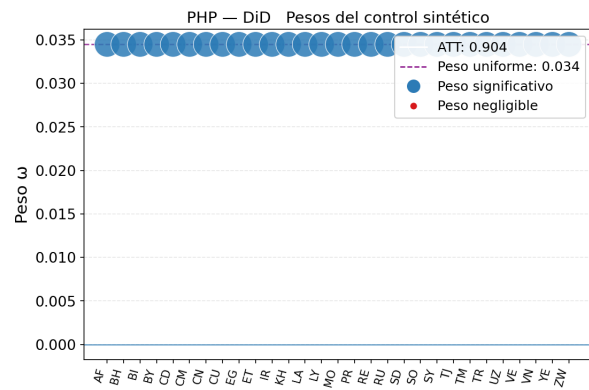
Nota. Elaboración propia.

Figura 11

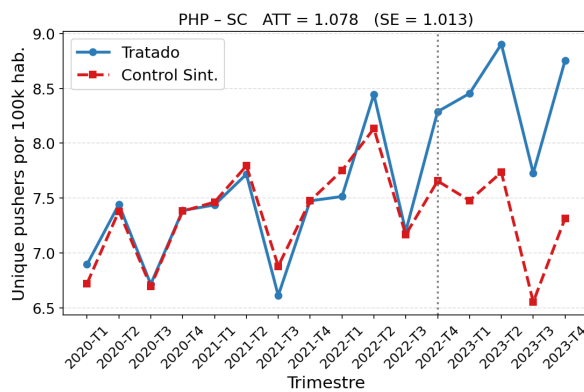
Tendencias y ponderaciones estimadas de PHP *unique pushers* por cada 100 mil personas



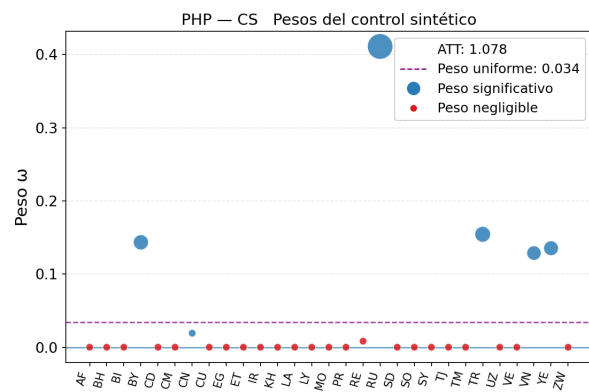
(a) Tendencias estimadas de DiD



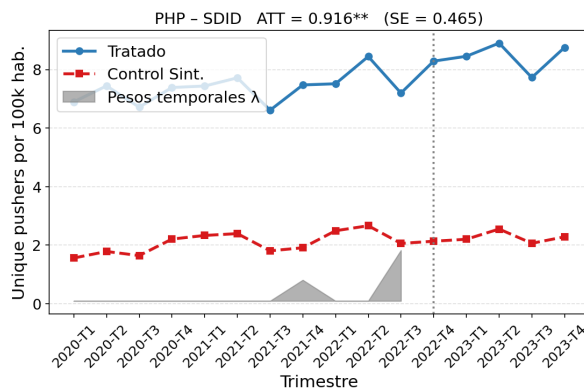
(b) Pesos DiD estimados



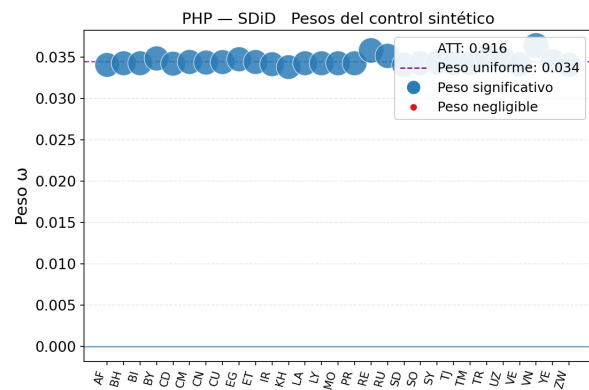
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



(e) Tendencias estimadas de SDiD

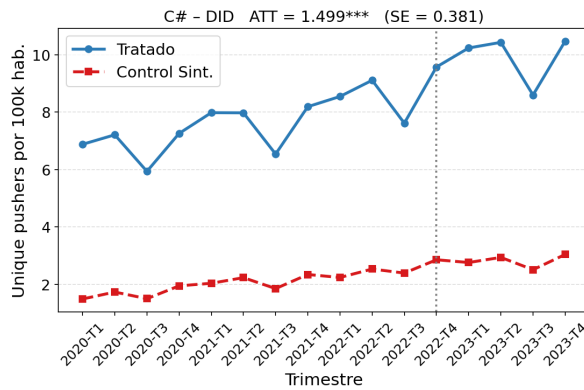


(f) Pesos SDiD estimados

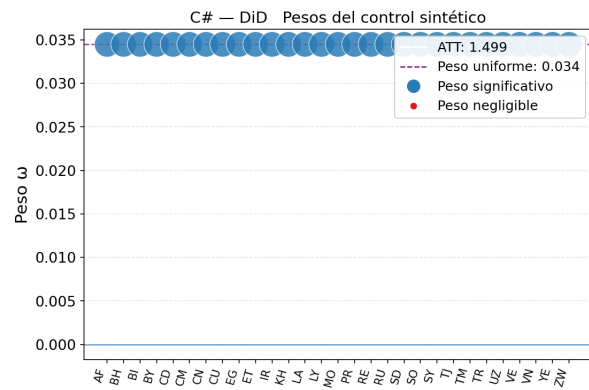
Nota. Elaboración propia.

Figura 12

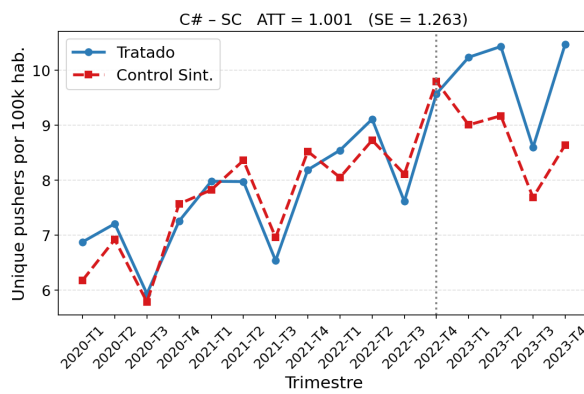
Tendencias y ponderaciones estimadas de *C# unique pushers* por cada 100 mil personas



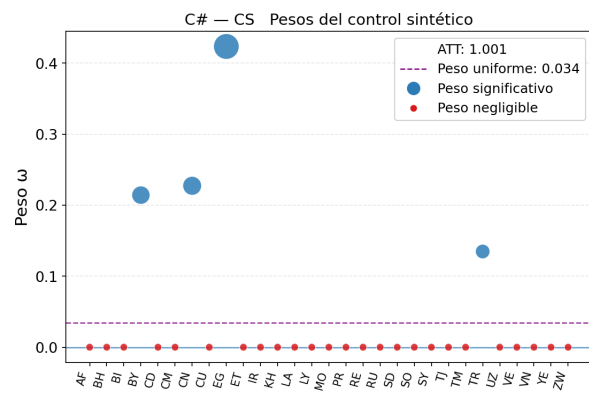
(a) Tendencias estimadas de DiD



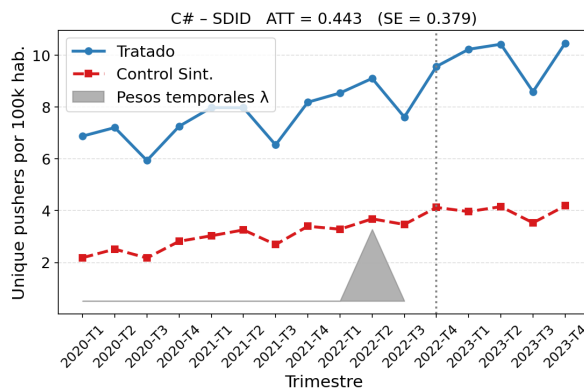
(b) Pesos DiD estimados



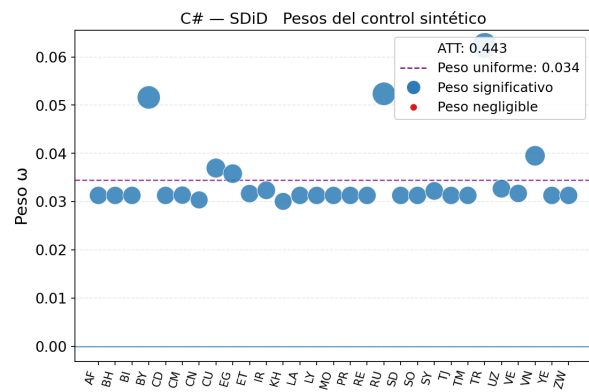
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



(e) Tendencias estimadas de SDiD

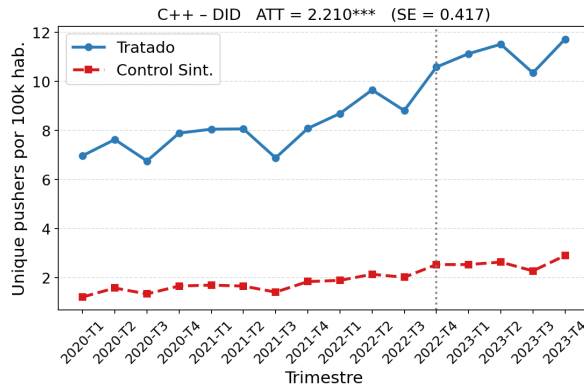


(f) Pesos SDiD estimados

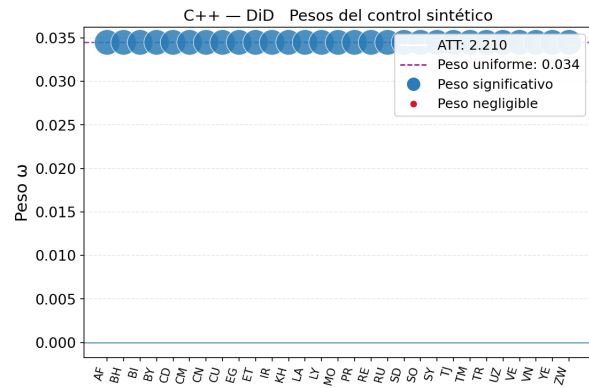
Nota. Elaboración propia.

Figura 13

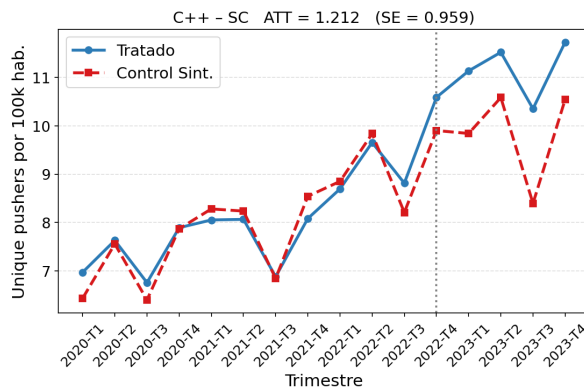
Tendencias y ponderaciones estimadas de C++ *unique pushers* por cada 100 mil personas



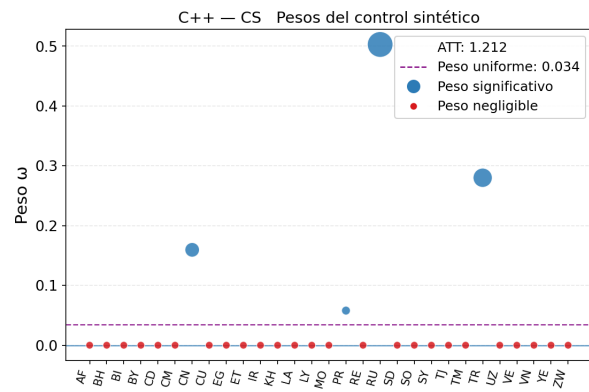
(a) Tendencias estimadas de DiD



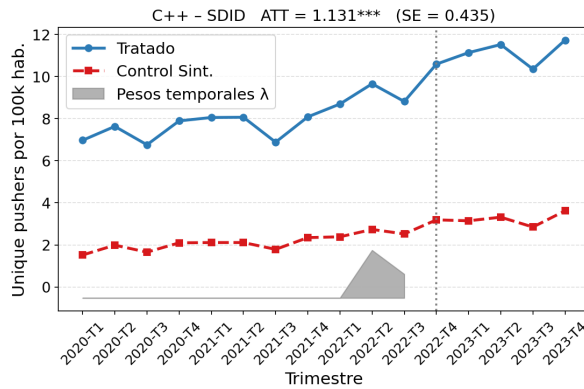
(b) Pesos DiD estimados



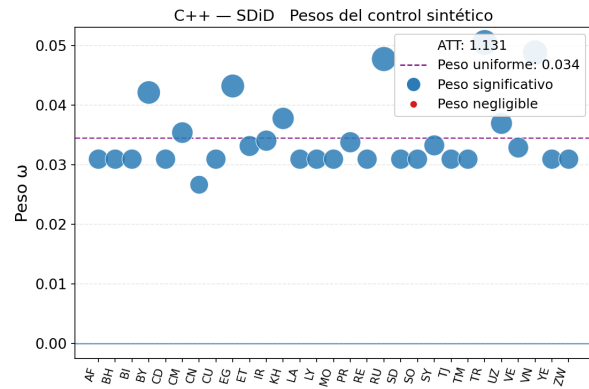
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



(e) Tendencias estimadas de SDiD

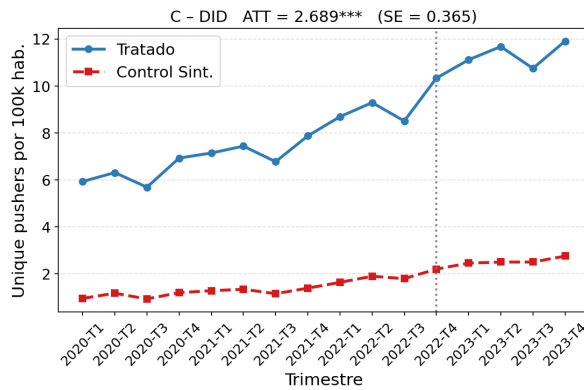


(f) Pesos SDiD estimados

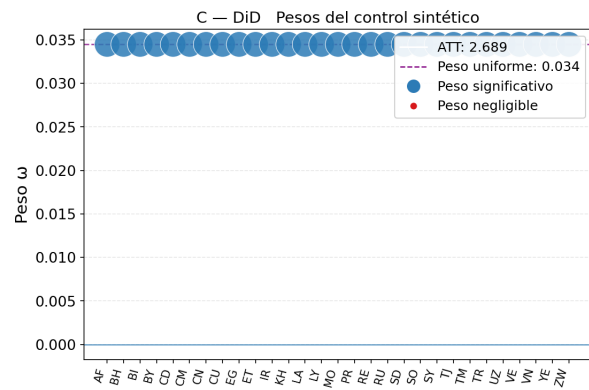
Nota. Elaboración propia.

Figura 14

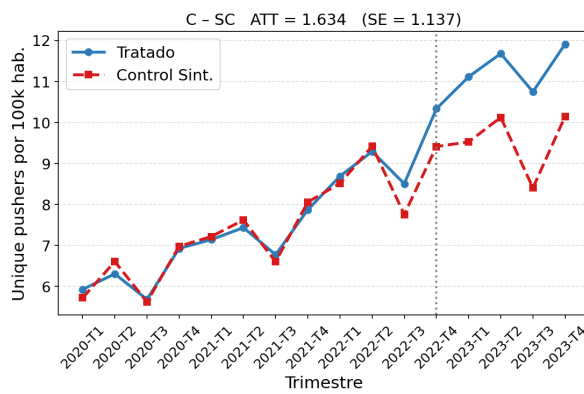
Tendencias y ponderaciones estimadas de *C unique pushers* por cada 100 mil personas



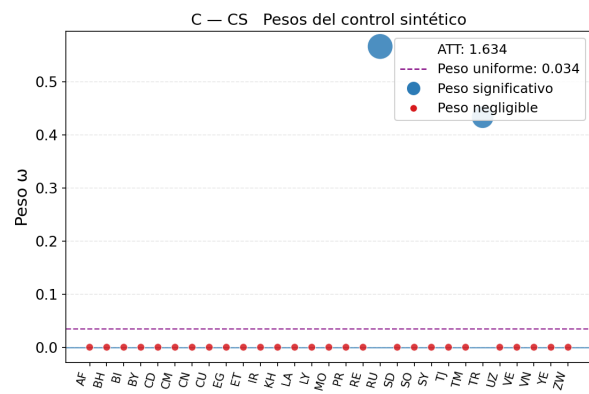
(a) Tendencias estimadas de DiD



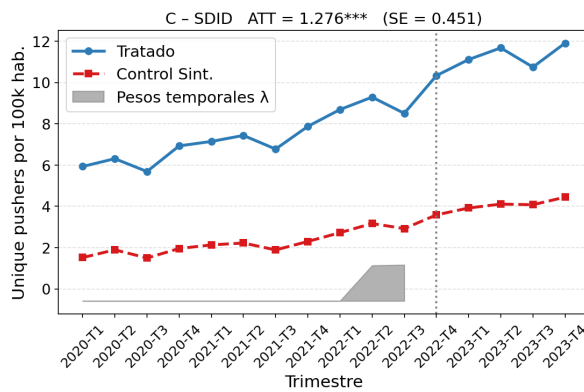
(b) Pesos DiD estimados



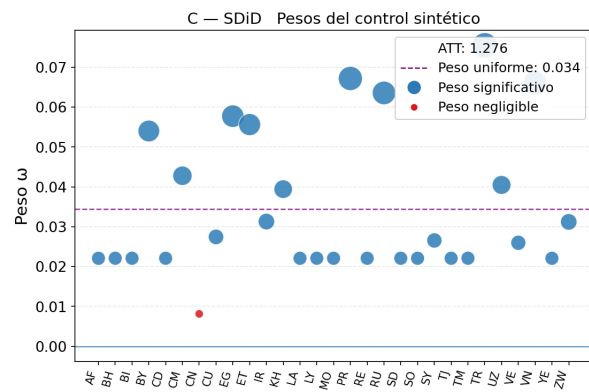
(c) Tendencias estimadas de SC



(d) Pesos SC estimados



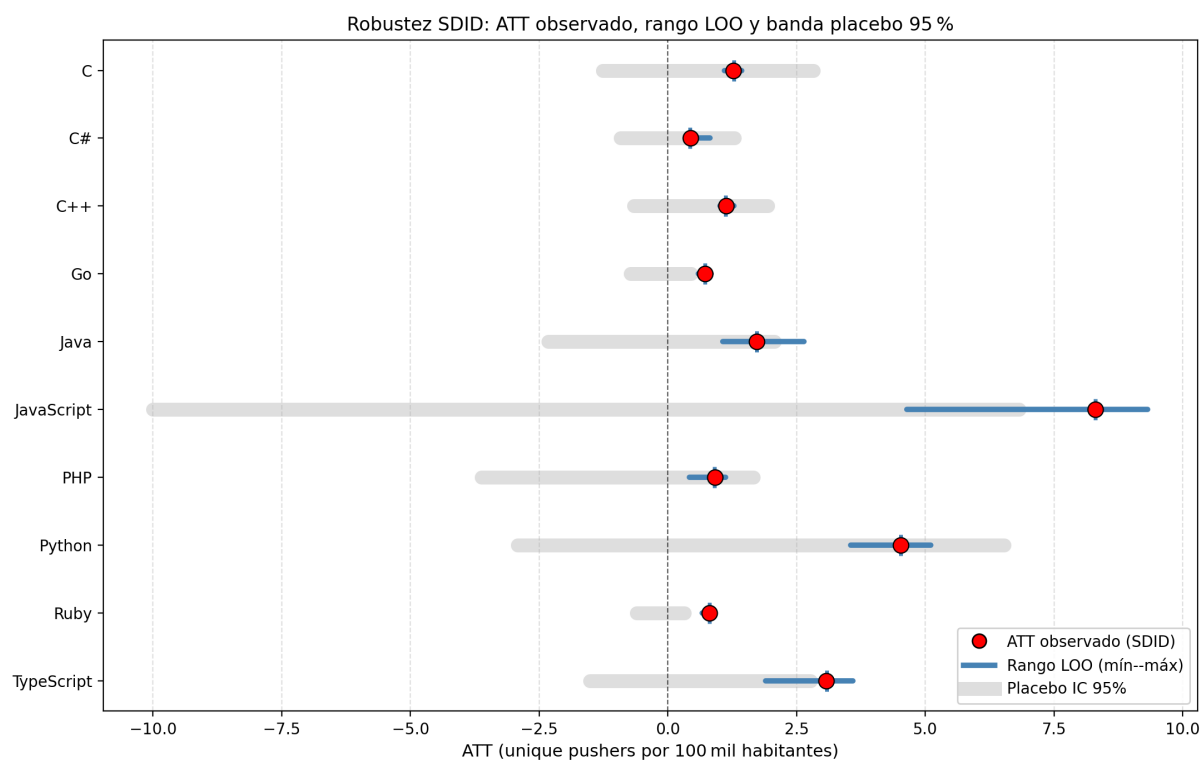
(e) Tendencias estimadas de SDiD



(f) Pesos SDiD estimados

Nota. Elaboración propia.

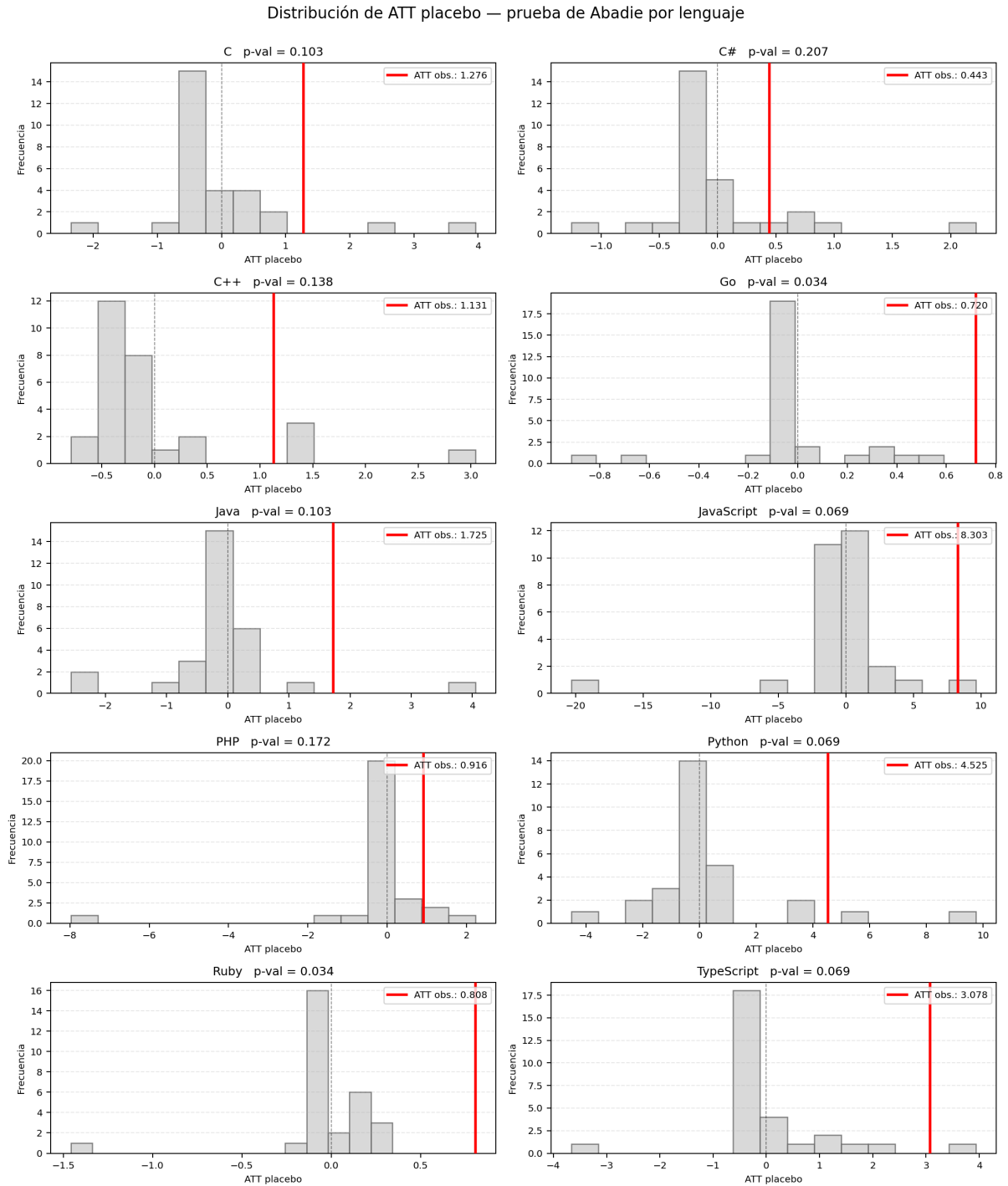
Figura 15
Robustez SDID: ATT observado, rango *leave-one-out* y banda placebo 95 %



Nota. Para cada lenguaje de programación: el punto rojo indica el ATT estimado por SDiD en la especificación principal; la barra azul representa el rango *leave-one-out* (mínimo–máximo) obtenido al eliminar cada uno de los 29 países de control del pool de donantes y reestimar el SDID; la banda gris indica el intervalo de confianza al 95 % de la distribución de 29 placebos in-space construida al asignar el tratamiento a un país de control a la vez (*a la Abadie, 2010, 2021*). Elaboración propia.

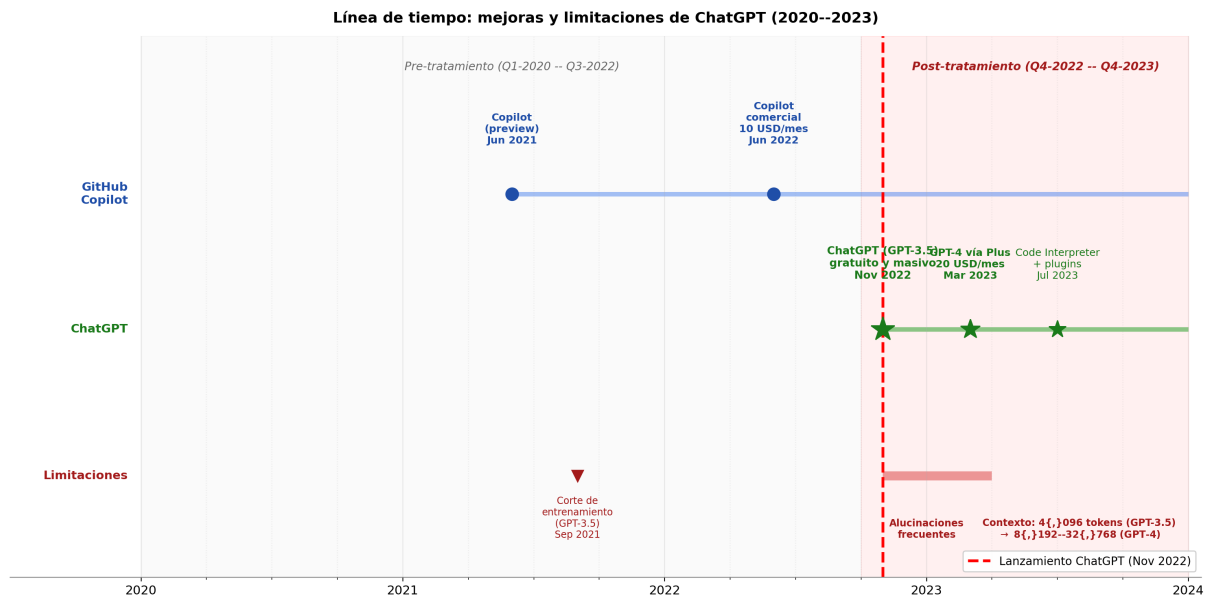
Figura 16

Distribución de ATT placebo por lenguaje (prueba no paramétrica de Abadie)



Nota. Histograma de los 29 ATT placebo por lenguaje, construido asignando un tratamiento placebo a un país de control a la vez y reestimando el SDiD restringiendo la muestra al grupo de control. La línea roja vertical indica el ATT observado en la especificación principal y el p-valor placebo en el título de cada panel corresponde a la fracción de placebos cuyo $|ATT|$ es mayor o igual al $|ATT|$ observado. Elaboración propia.

Figura 17
Línea de tiempo de ChatGPT: mejoras y limitaciones (2020–2023)



Nota. La figura organiza en tres carriles los hitos relevantes durante el período de análisis. **GitHub Copilot** (azul): *preview* técnico en junio de 2021 y lanzamiento comercial pagado en junio de 2022. **ChatGPT** (verde): GPT-3.5 gratuito en noviembre de 2022 (inicio del tratamiento, marcado por la línea vertical roja), GPT-4 vía ChatGPT Plus en marzo de 2023 y Code Interpreter más *plugins* en julio de 2023. **Limitaciones** (rojo): corte de entrenamiento en septiembre de 2021, alucinaciones frecuentes durante los primeros meses post-lanzamiento y ventanas de contexto restringidas (4,096 *tokens* en GPT-3.5; 8,192–32,768 en GPT-4). La banda gris representa el período pretratamiento y la banda roja, el período postratamiento. Elaboración propia.

Referencias bibliográficas

- Abadie, A., Diamond, A., & Hainmueller, A. J. (2010). Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California's Tobacco Control Program. *Journal of the American Statistical Association*, 105(490), 493–505. <https://doi.org/10.1198/JASA.2009.AP08746>
- Abadie, A. (2021). Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects. *Journal of Economic Literature*, 59(2), 391–425. <https://doi.org/10.1257/JEL.20191450>
- Abu Jaber, M., Beganovic, A., & Abd Almisreb, A. (2023). Methods and Applications of ChatGPT in Software Development: A Literature Review. *Southeast Europe Journal of Soft Computing*, 12(1), 08–12. https://www.researchgate.net/publication/370872615_Methods_and_Applications_of_ChatGPT_in_Software_Development_A_Literature_Review
- Acemoglu, D. (2024). The Simple Macroeconomics of AI. *Economic Policy*, 40(121), 13–58. <https://doi.org/10.1093/epolic/eiae042>
- Acemoglu, D., & Restrepo, P. (2018). The Race between Man and Machine: Implications of Technology for Growth, Factor Shares, and Employment. *American Economic Review*, 108(6), 1488–1542. <https://doi.org/10.1257/AER.20160696>
- Acemoglu, D., & Restrepo, P. (2019). Automation and New Tasks: How Technology Displaces and Reinstates Labor. *Journal of Economic Perspectives*, 33(2), 3–30. <https://doi.org/10.1257/JEP.33.2.3>
- Arkhangelsky, D., Athey, S., Hirshberg, D. A., Imbens, G. W., & Wager, S. (2021). Synthetic Difference-in-Differences. *American Economic Review*, 111(12), 4088–4118. <https://doi.org/10.1257/AER.20190159>
- BBC News Mundo. (2023, marzo 31). *ChatGPT: Italia se convierte en el primer país occidental en bloquear el acceso al programa de inteligencia artificial*. BBC News Mundo. <https://www.bbc.com/mundo/noticias-65142505>
- Brynjolfsson, W., Li, D., Raymond, L., Nber, S., Acemoglu, D., Autor, D., Axelrod, A., Dillon, E., Enam, Z., Garicano, L., Frankel, A., Manning, S., Mullainathan, S., Pierson, E., Stern, S., Rambachan, A., Reenen, J. Van, Sadun, R., Shaw, K., & Thrun, S. (2023). Generative AI at Work. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.4426942>
- Buscemi, A. (2023). A Comparative Study of Code Generation using ChatGPT 3.5 across 10 Programming Languages. *arXiv preprint arXiv:2308.04477*. <https://arxiv.org/abs/2308.04477>
- Chai, Z., Wang, S., Liang, J., Liu, Y., Guo, H., Tian, R., Zhang, N., Zan, D., Dou, S., Huang, Y., Gao, J., Yu, D., Liu, J., Li, G., Yu, J., & Liu, Z. (2024). McE-

- val: Massively Multilingual Code Evaluation. *arXiv preprint arXiv:2406.07436*. <https://arxiv.org/abs/2406.07436>
- Chui, M., Hazan, E., Roberts, R., Singla, A., Smaje, K., Sukharevsky, A., Yee, L., & Zemmell, R. (2023). The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. <http://arxiv.org/abs/2302.06590>
- Clarke, D., Pailańir, D., Carleton Athey, S., & Imbens, G. W. (2023). Synthetic Difference In Differences Estimation. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.4346540>
- Coursera Staff. (2024). Most Popular Programming Languages in 2024 | Coursera. *Coursera*. <https://www.coursera.org/articles/popular-programming-languages>
- Data Camp. (2023). Top programming languages for data scientists in 2023 | DataCamp. *Datacamp*. <https://www.datacamp.com/blog/top-programming-languages-for-data-scientists-in-2022>
- Del Rio-Chanona, M., Laurentsyeve, N., & Wachs, J. (2023). Are Large Language Models a Threat to Digital Public Goods? Evidence from Activity on Stack Overflow. <https://arxiv.org/abs/2307.07367v1>
- Demirci, O., Hannane, J., & Zhu, X. (2023). Who Is AI Replacing? The Impact of Generative AI on Online Freelancing Platforms. *SSRN Electronic Journal*. <https://doi.org/10.2139/SSRN.4602944>
- Eloundou, T., Manning, S., Mishkin, P., & Rock, D. (2023). GPTs are GPTs: An Early Look at the Labor Market Impact Potential of Large Language Models. *arXiv preprint arXiv:2303.10130*. <https://arxiv.org/abs/2303.10130>
- Gallea, Q. (2023). From Mundane to Meaningful: AI's Influence on Work Dynamics – evidence from ChatGPT and Stack Overflow. <https://arxiv.org/abs/2308.11302v1>
- Hui, X., Reshef, O., & Zhou, L. (2023). The Short-Term Effects of Generative Artificial Intelligence on Employment: Evidence from an Online Labor Market. *SSRN Electronic Journal*. <https://doi.org/10.2139/SSRN.4527336>
- Ijaz, S. (2023, julio 6). 15 Countries That Banned ChatGPT. Insider Monkey. <https://www.insidermonkey.com/blog/15-countries-that-banned-chatgpt-1165549/>
- Kalla, D., Smith, N., Samaah, F., & Kuraku, S. (2023). Study and Analysis of Chat GPT and its Impact on Different Fields of Study. <https://papers.ssrn.com/abstract=4402499>
- Kreitmeir, D., & Raschky, P. A. (2023). The Unintended Consequences of Censo-

- ring Digital Technology: Evidence from Italy's ChatGPT Ban. *arXiv preprint arXiv:2304.09339*. <https://arxiv.org/abs/2304.09339>
- Li, M., & Krishnamachari, B. (2024). Evaluating ChatGPT-3.5 efficiency in solving coding problems of different complexity levels: An empirical analysis. *arXiv preprint arXiv:2411.07529*. <https://doi.org/10.48550/arXiv.2411.07529>
- Liu, J., Foster, M. G., Nan, X., Li, Y., & Tan, Y. (2023). "Generate"the Future of Work through AI: Empirical Evidence from Online Labor Markets. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.4529739>
- Martinez, J. (2023). Guerra abierta contra ChatGPT: estos son los países que ya lo han prohibido. *ABC*. <https://www.abc.es/tecnologia/guerra-abierta-chatgpt-paises-prohibido-20230427200409-nt.html>
- ONU. (2023). El debate de la Inteligencia Artificial en la ONU. *Naciones Unidas*. <https://unric.org/es/el-debate-de-la-inteligencia-artificial-en-la-onu/>
- OpenAI. (2018). OpenAI charter. OpenAI. <https://openai.com/>
- OpenAI. (2024). Models - OpenAI API. <https://platform.openai.com/docs/models>
- Rice CS. (2023). Top 12 Data Science Programming Languages | MDS@Rice. *Rice University*. <https://csweb.rice.edu/academics/graduate-programs/online-mds/blog/programming-languages-for-data-science>
- Shen, Y., Ai, X., Soosai Raj, A. G., Leo John, R. J., & Syamkumar, M. (2024). Implications of ChatGPT for Data Science Education. *SIGCSE 2024 - Proceedings of the 55th ACM Technical Symposium on Computer Science Education*, 1, 1230–1236. <https://doi.org/10.1145/3626252.3630874>
- Statista. (2023). Software - Worldwide. Market Forecast. <https://www.statista.com/outlook/tmo/software/worldwide#revenue>
- Stigler, G. J. (1961). The Economics of Information. *Journal of Political Economy*, 69(3), 213–225. <https://doi.org/10.1086/258464>
- Tlili, A., Shehata, B., Adarkwah, M. A., Bozkurt, A., Hickey, D. T., Huang, R., & Ag-yemang, B. (2023). What if the devil is my guardian angel: ChatGPT as a case study of using chatbots in education. *Smart Learning Environments*, 10(1), 1–24. <https://doi.org/10.1186/S40561-023-00237-X/FIGURES/13>
- World Intellectual Property Organization. (2022). World Intellectual Property Report 2022 — The Direction of Innovation. <https://www.wipo.int/web/world-ip-report/2022-the-direction-of-innovation>
- Wu, Q., Zhuang, Q., Liu, Y., & Han, L. (2024). Technology Shock of ChatGPT, Social Attention and Firm Value: Evidence from China. *SSRN*. <https://doi.org/10.>

Anexos

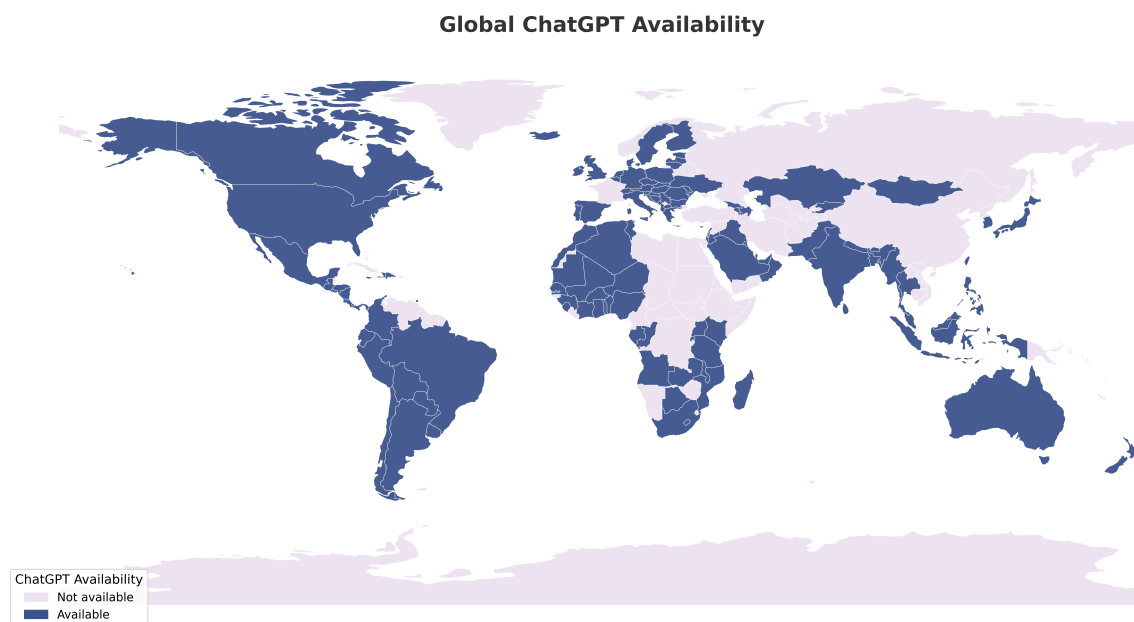
Anexo 1

Disponibilidad global de ChatGPT

Listado de países según la disponibilidad de acceso al servicio de ChatGPT

Figura 18

Mapa de disponibilidad global de ChatGPT



Nota. Elaboración propia.

Lista de países según disponibilidad de acceso a ChatGPT

ISO2	País con GPT	ISO2	País sin GPT
AL	Albania	AF	Afghanistan
DZ	Algeria	BH	Bahrain
AD	Andorra	BY	Belarus
AO	Angola	BI	Burundi
AR	Argentina	KH	Cambodia
AM	Armenia	CM	Cameroon
AU	Australia	CN	China
AT	Austria	CD	Congo, The Democratic Republic of the
AZ	Azerbaijan	CU	Cuba
BD	Bangladesh	EG	Egypt
BB	Barbados	SZ	Eswatini
BE	Belgium	ET	Ethiopia
BZ	Belize	GP	Guadeloupe
BJ	Benin	HK	Hong Kong
BT	Bhutan	IR	Iran, Islamic Republic of
BO	Bolivia, Plurinational State of	LA	Lao People's Democratic Republic
BA	Bosnia and Herzegovina	LY	Libya
BW	Botswana	MO	Macao
BR	Brazil	PR	Puerto Rico

(Continuación)

ISO2	País con GPT	ISO2	País sin GPT
BN	Brunei Darussalam	RU	Russian Federation
BG	Bulgaria	RE	Réunion
BF	Burkina Faso	SO	Somalia
CV	Cabo Verde	SD	Sudan
CA	Canada	SY	Syrian Arab Republic
CL	Chile	TJ	Tajikistan
CO	Colombia	TM	Turkmenistan
CG	Congo	TR	Türkiye
CR	Costa Rica	UZ	Uzbekistan
HR	Croatia	VE	Venezuela, Bolivarian Republic of
CY	Cyprus	VN	Viet Nam
CZ	Czechia	YE	Yemen
CI	Côte d'Ivoire	ZW	Zimbabwe
DK	Denmark		
DO	Dominican Republic		
EC	Ecuador		
SV	El Salvador		
EE	Estonia		
FI	Finland		
FR	France		
GA	Gabon		
GM	Gambia		
GE	Georgia		
DE	Germany		
GH	Ghana		
GR	Greece		
GT	Guatemala		
GN	Guinea		
HT	Haiti		
HN	Honduras		
HU	Hungary		
IS	Iceland		
IN	India		
ID	Indonesia		
IQ	Iraq		
IE	Ireland		
IL	Israel		
IT	Italy		
JM	Jamaica		
JP	Japan		
JO	Jordan		
KZ	Kazakhstan		
KE	Kenya		
KR	Korea, Republic of		
KW	Kuwait		
KG	Kyrgyzstan		
LV	Latvia		
LB	Lebanon		
LS	Lesotho		
LR	Liberia		
LT	Lithuania		
LU	Luxembourg		
MG	Madagascar		
MW	Malawi		
MY	Malaysia		

(Continuación)

ISO2	País con GPT	ISO2	País sin GPT
MV	Maldives		
ML	Mali		
MT	Malta		
MR	Mauritania		
MU	Mauritius		
MX	Mexico		
MD	Moldova, Republic of		
MN	Mongolia		
ME	Montenegro		
MA	Morocco		
MZ	Mozambique		
MM	Myanmar		
NP	Nepal		
NL	Netherlands		
NZ	New Zealand		
NI	Nicaragua		
NE	Niger		
NG	Nigeria		
MK	North Macedonia		
NO	Norway		
OM	Oman		
PK	Pakistan		
PS	Palestine, State of		
PA	Panama		
PG	Papua New Guinea		
PY	Paraguay		
PE	Peru		
PH	Philippines		
PL	Poland		
PT	Portugal		
QA	Qatar		
RO	Romania		
RW	Rwanda		
SA	Saudi Arabia		
SN	Senegal		
RS	Serbia		
SC	Seychelles		
SL	Sierra Leone		
SG	Singapore		
SK	Slovakia		
SI	Slovenia		
ZA	South Africa		
ES	Spain		
LK	Sri Lanka		
SE	Sweden		
CH	Switzerland		
TW	Taiwan, Province of China		
TZ	Tanzania, United Republic of		
TH	Thailand		
TG	Togo		
TT	Trinidad and Tobago		
TN	Tunisia		
UG	Uganda		
UA	Ukraine		
AE	United Arab Emirates		

(Continuación)

ISO2	País con GPT	ISO2	País sin GPT
GB	United Kingdom		
US	United States		
UY	Uruguay		
ZM	Zambia		

Anexo 2

Variables de control y justificación de su no inclusión en el modelo base

Variables de control utilizadas

Las variables de control empleadas en la estimación con controles (Tabla 4) fueron extraídas del **Banco Mundial** y corresponden a dos indicadores de infraestructura digital:

- **Porcentaje de individuos que usan computadora:** obtenido del indicador *Percentage of individuals using a computer* (ITU DataHub, vía World Bank Data360, código ITU_DH_INV_COMP). Mide la proporción de la población que utiliza una computadora, capturando el nivel de adopción tecnológica en el hogar y la capacidad técnica promedio del país.
- **Porcentaje de individuos que usan internet:** extraído de los Indicadores de Desarrollo Mundial del Banco Mundial (indicador IT.NET.USER.ZS). Mide el porcentaje de la población con acceso a internet, variable fundamental puesto que constituye un prerrequisito para utilizar plataformas de desarrollo como GitHub y herramientas basadas en la nube.

Estas variables fueron consideradas como controles porque capturan diferencias estructurales en infraestructura digital entre países que podrían influir de manera independiente sobre la actividad de desarrollo en GitHub. Al incluirlas, se busca aislar el efecto específico de la disponibilidad de ChatGPT de tendencias más amplias de desarrollo digital preexistentes. Ambos indicadores se encuentran disponibles con frecuencia anual y fueron interpolados a frecuencia trimestral para su incorporación en el panel.

Justificación de la no inclusión en el modelo base

En el presente estudio, no se incorporaron variables de control adicionales en las estimaciones del modelo base realizadas mediante los métodos Difference-in-Differences (DID), Synthetic Control (SC) y Synthetic Difference-in-Differences (SDID). Esta decisión se fundamenta principalmente en la limitada disponibilidad de variables comparables para todos los países incluidos en el análisis. Si bien existen diversas bases de datos internacionales con información potencialmente relevante, como las del Banco Mundial, muchas de ellas no ofrecen una cobertura completa o no son compatibles con la estructura de panel utilizada en esta investigación.

El panel base empleado en el modelo sin controles comprende 159 países, 16 períodos trimestrales y 10 lenguajes de programación, lo que resulta en un total de 25.440 observaciones trimestrales, equivalentes a 6.360 en términos anuales. De estos, 130 países pertenecen al grupo de tratamiento (con acceso a ChatGPT) y 29 al grupo de control. Como se muestra en la Tabla 9, tres entidades fueron excluidas del

análisis: la Unión Europea (EU), que no constituye una unidad nacional independiente; Kosovo (XK), cuyo código no puede ser procesado por las librerías estándar de georeferenciación; y Hong Kong (HK), cuya actividad de desarrollo en GitHub resultó atípicamente elevada en comparación con el resto de los países del panel, lo que podría distorsionar las estimaciones.

Adicionalmente, la incorporación de las variables de control del Banco Mundial reveló una cobertura incompleta para 19 países, que carecen de datos disponibles para el indicador de uso de internet (IT.NET.USER.ZS), tal como se detalla en la Tabla 10. Estas 2.640 observaciones (10,4 % del panel) fueron imputadas como cero para permitir la estimación con controles, si bien esta imputación puede afectar la precisión de los coeficientes en el modelo de la Sección 6. En conjunto, estos factores —la necesidad de imputar valores faltantes en las variables de control y el potencial desequilibrio que ello introduce entre grupos— justifican la presentación del modelo sin controles como especificación base, siendo la estimación con controles interpretada como análisis de robustez.

Tabla 7
Mapeo de la cantidad de muestras

Tipo	Cantidad
Quarter	25440
Year	6360
Final	6360

Nota. Elaboración propia.

Tabla 8
Comparación por grupo de muestras

Grupo	Obs. trimestrales	Obs. anuales	Obs. finales	Obs. perdidas
0 (control)	4640	1160	1160	3480
1 (tratamiento)	20800	5200	5200	15600

Nota. Elaboración propia.

Tabla 9
Países eliminados

Código ISO2	ChatGPT disponible
EU	—
XK	0
HK	1

Nota. Elaboración propia.

Tabla 10
Missing values o datos vacíos por país

Código ISO2	ChatGPT disponible	Obs. faltantes
CD	0	160
CG	1	160
EG	0	160
ET	0	80
HT	1	160
IN	1	80
IR	0	160
KE	1	120
KR	1	160
MO	0	160
PR	0	80
RE	0	160
SD	0	120
SO	0	80
SY	0	160
TM	0	160
TW	1	160
VE	0	160
YE	0	160

Nota. Elaboración propia.

Tabla 4
Impacto de ChatGPT en el número de programadores (con controles)

Lenguaje	DID	SC	SDID	Obs.	Baseline Mean
C	2,774*** (0,367)	1,433 (1,240)	1,396*** (0,424)	2544	6,232
C#	1,475*** (0,383)	1,049 (1,252)	0,422 (0,386)	2544	6,557
C++	2,236*** (0,419)	1,242 (0,979)	1,157*** (0,437)	2544	6,807
Go	1,411*** (0,176)	0,689 (0,606)	0,728*** (0,270)	2544	2,230
Java	2,831*** (0,832)	2,224 (2,038)	1,726** (0,874)	2544	11,467
JavaScript	14,106*** (2,856)	10,470** (4,738)	9,142*** (3,228)	2544	36,571
PHP	0,994* (0,523)	0,974 (1,029)	1,034** (0,470)	2544	6,381
Python	7,831*** (1,147)	2,028 (3,531)	4,700*** (1,324)	2544	20,003
Ruby	0,312 (0,272)	1,605* (0,855)	0,864*** (0,278)	2544	5,592
TypeScript	7,189*** (0,846)	2,742 (2,244)	3,058*** (0,870)	2544	8,571

Nota. Estimaciones del efecto promedio del tratamiento (ATT) del acceso a ChatGPT sobre el número de *unique pushers* por cada 100 000 habitantes. Las columnas DID, SC y SDID corresponden a Diferencias en Diferencias, Control Sintético (Abadie et al., 2010) y Diferencias en Diferencias Sintética (Arkhangelsky et al., 2021), respectivamente, todos estimados mediante la librería *synthdid* de Python (Clarke et al., 2023). Se incluyen como variables de control el porcentaje de individuos que usan computadora y el porcentaje que usa internet, extraídas del Banco Mundial, incorporadas mediante un *partial-out* de efectos fijos de unidad y tiempo. Los errores estándar se obtienen por *bootstrap* (100 replicaciones). El periodo de tratamiento inicia en Q4-2022. Fuentes: GitHub Innovation Graph (<https://github.com/github/innovationgraph>); Banco Mundial, Indicadores de Desarrollo Mundial. Elaboración propia. Errores estándar entre paréntesis. * p<0.10, ** p<0.05, *** p<0.01

Tabla 5

Robustez: SDID con grupo de control restringido (excluye censores severos)

Lenguaje	SDID (completo)	SDID (restringido)	Obs. pleno	Obs. restringido	Baseline Mean
C	1,276*** (0,451)	0,922* (0,486)	2544	2448	1,339
C#	0,443 (0,379)	0,402 (0,616)	2544	2448	2,032
C++	1,131*** (0,435)	1,281** (0,571)	2544	2448	1,666
Go	0,720*** (0,269)	0,598 (0,467)	2544	2448	0,468
Java	1,725** (0,872)	1,112 (0,971)	2544	2448	3,669
JavaScript	8,303*** (3,180)	3,728* (2,054)	2544	2448	12,652
PHP	0,916** (0,465)	0,697 (0,452)	2544	2448	2,044
Python	4,525*** (1,321)	4,949*** (1,574)	2544	2448	5,502
Ruby	0,808*** (0,274)	0,564** (0,280)	2544	2448	1,367
TypeScript	3,078*** (0,873)	1,601 (0,991)	2544	2448	2,056

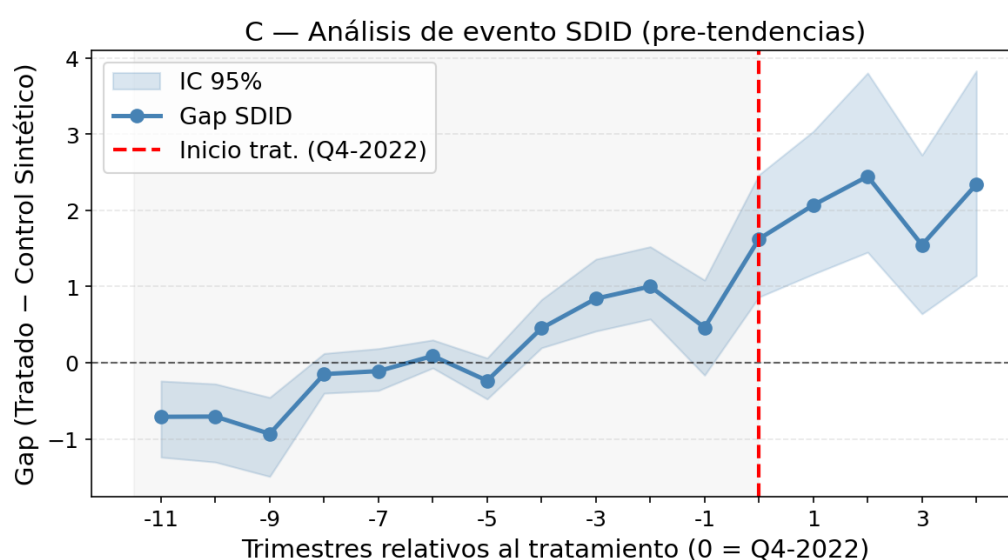
Nota. Estimaciones del efecto promedio del tratamiento (ATT) del acceso a ChatGPT sobre el número de *unique pushers* por cada 100 000 habitantes, mediante Diferencias en Diferencias Sintética (SDID; Arkhangelsky et al., 2021), estimada con la librería *synthdid* de Python (Clarke et al., 2023). La columna “SDID (completo)” emplea los 29 países de control y corresponde a la estimación principal; la columna “SDID (restringido)” excluye del grupo de control los seis censores severos de internet: China (CN), Cuba (CU), Irán (IR), Bielorrusia (BY), Rusia (RU) y Siria (SY), clasificados como “No Libres” por Freedom House (Freedom on the Net, 2022). Los errores estándar se obtienen por *bootstrap* (100 replicaciones). El periodo de tratamiento inicia en Q4-2022. Fuente: GitHub Innovation Graph (<https://github.com/github/innovationgraph>). Elaboración propia. Errores estándar entre paréntesis. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$

Anexo 3

Análisis de evento SDID: validación de pre-tendencias

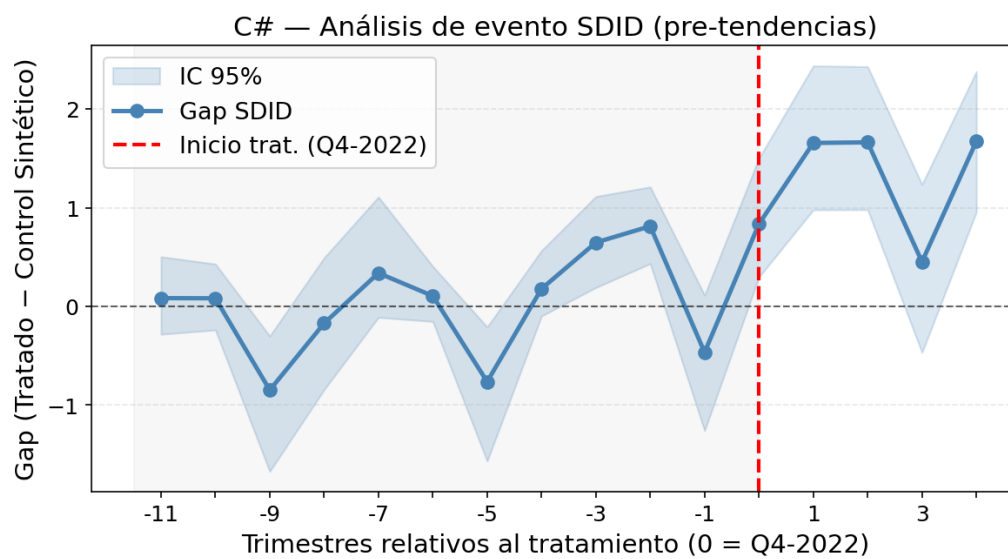
Los siguientes gráficos muestran el gap período a período entre el promedio de los países tratados y el control sintético construido por SDID, usando los pesos ω fijados en la estimación principal. Cada serie es normalizada restando la media pre-tratamiento, de modo que los períodos anteriores a Q4-2022 deben concentrarse en torno a cero si el supuesto de tendencias paralelas se cumple. La banda sombreada corresponde al intervalo de confianza al 95 %, obtenido por *bootstrap* sobre las unidades tratadas (200 replicaciones). La línea vertical roja señala el inicio del tratamiento (Q4-2022, período relativo $-0,5$).

Figura 19
Análisis de evento SDID — C



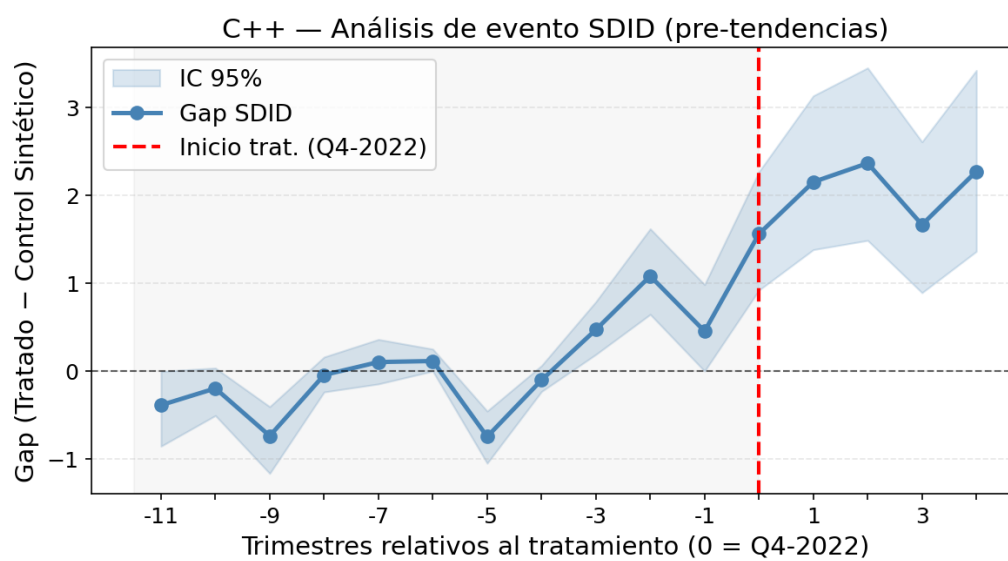
Nota. Elaboración propia.

Figura 20
Análisis de evento SDID — C#



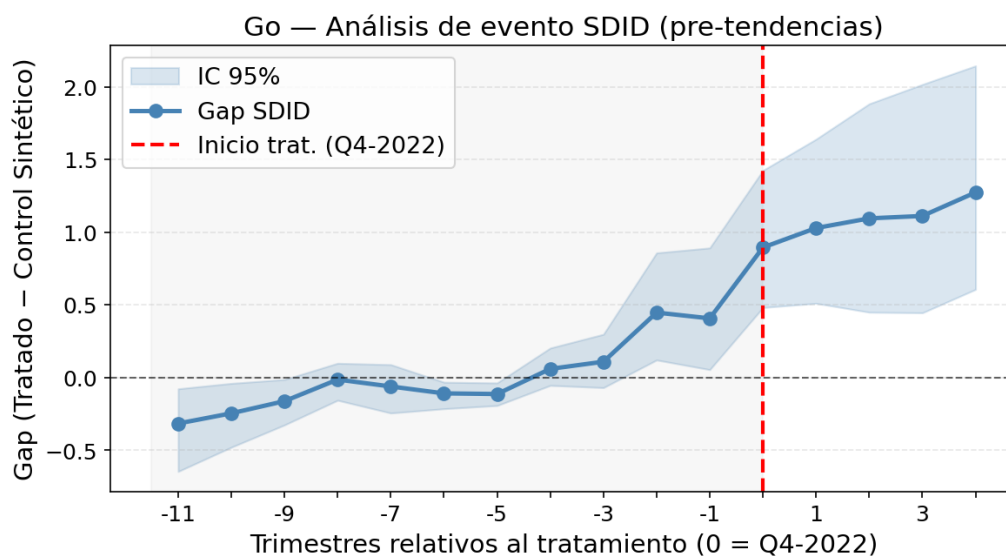
Nota. Elaboración propia.

Figura 21
Análisis de evento SDID — C++



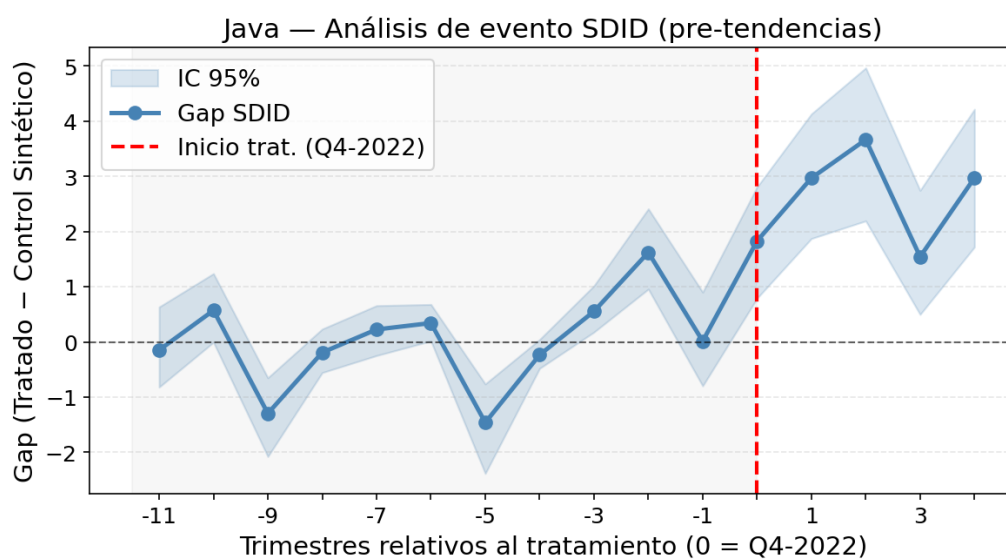
Nota. Elaboración propia.

Figura 22
Análisis de evento SDID — Go



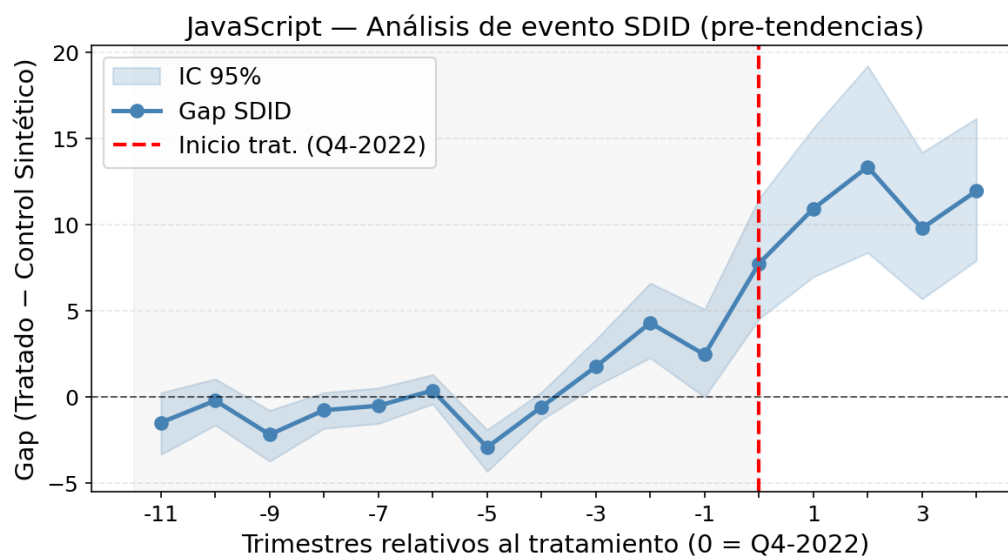
Nota. Elaboración propia.

Figura 23
Análisis de evento SDID — Java



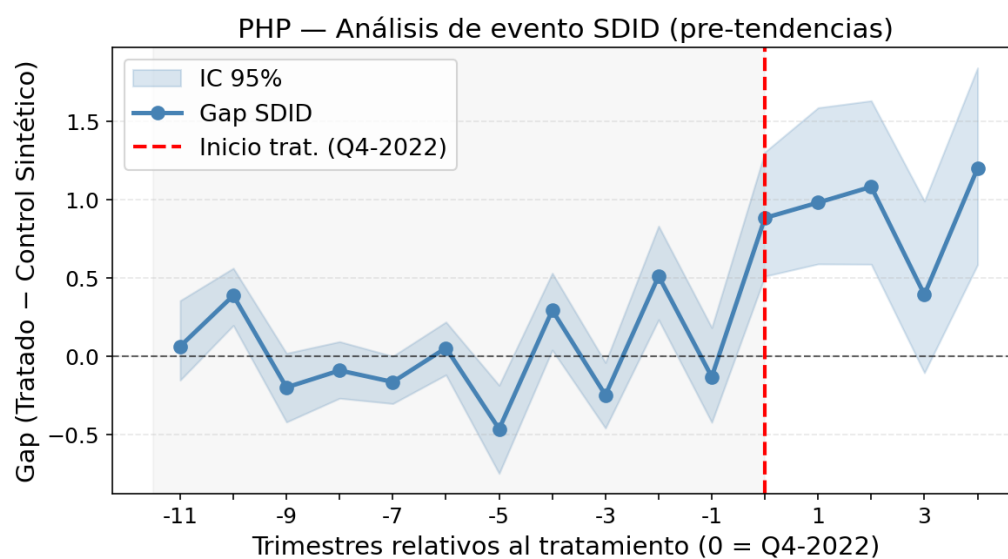
Nota. Elaboración propia.

Figura 24
Análisis de evento SDID — JavaScript



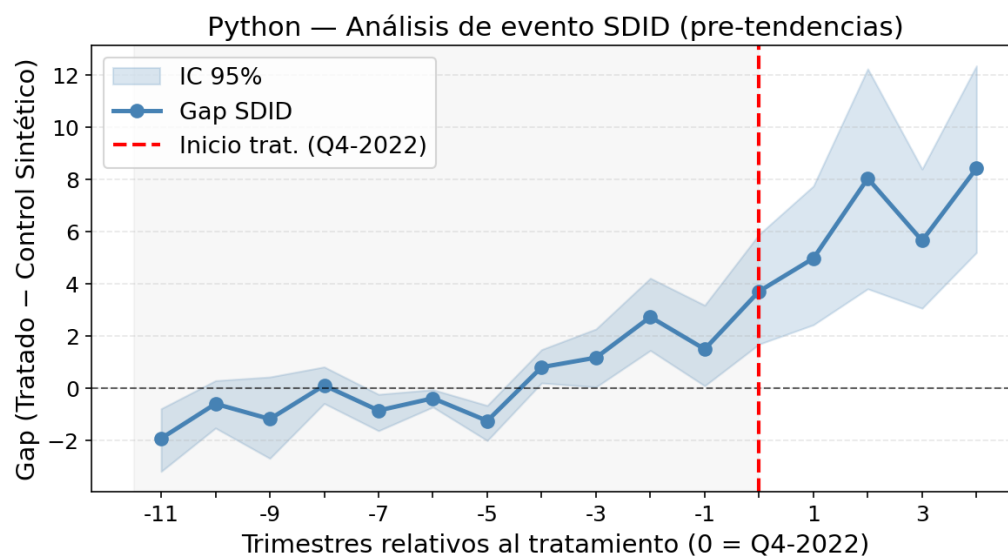
Nota. Elaboración propia.

Figura 25
Análisis de evento SDID — PHP



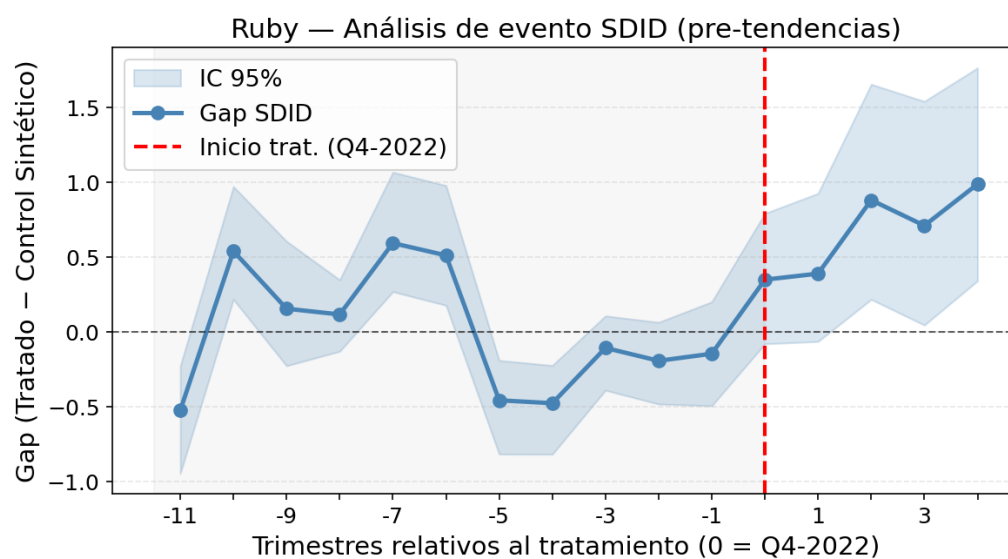
Nota. Elaboración propia.

Figura 26
Análisis de evento SDID — Python



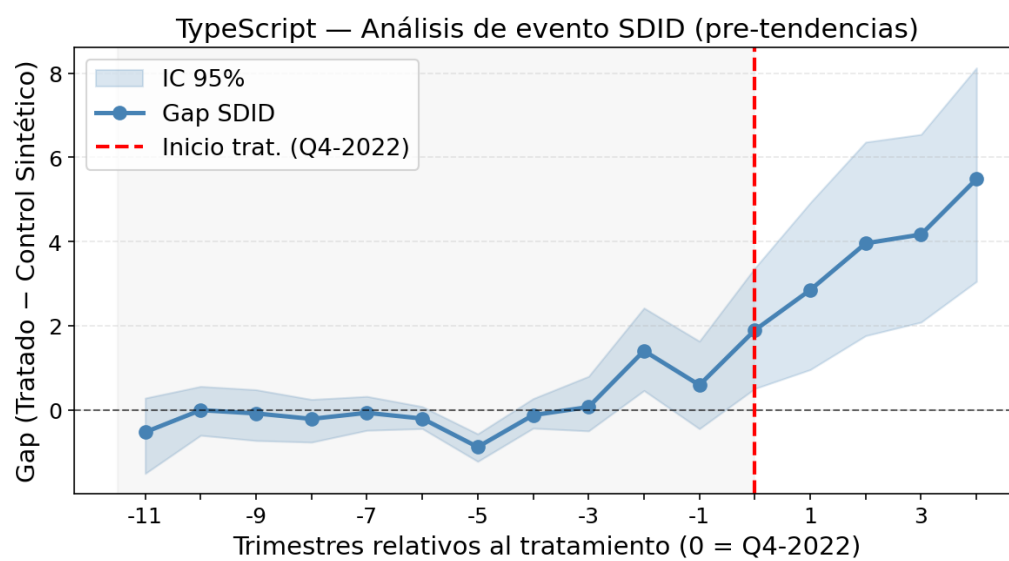
Nota. Elaboración propia.

Figura 27
Análisis de evento SDID — Ruby



Nota. Elaboración propia.

Figura 28
Análisis de evento SDID — TypeScript



Nota. Elaboración propia.

Anexo 4

Derivación formal del modelo teórico de exposición y absorción

Este anexo desarrolla, paso a paso, la derivación del modelo presentado en la tercera vertiente del marco teórico económico (sección 1.4), desde la función de producción a nivel de tarea hasta la especificación de panel estimable. El modelo adapta al contexto de los lenguajes de programación el marco de tareas con automatización parcial de Acemoglu (2024) y el constructo de exposición laboral a modelos de lenguaje de Eloundou et al. (2023). La derivación distingue dos tipos de operaciones: las *identidades exactas*, que son reescrituras algebraicas y no agregan supuestos, y las *aproximaciones o supuestos*, que simplifican la expresión para obtener una forma interpretable y estimable. Cada supuesto no trivial se señala explícitamente en el paso en que se introduce y se recopila al final del anexo.

Notación

La Tabla siguiente resume la notación empleada a lo largo de la derivación.

Símbolo	Significado
$z \in [0, 1]$	Tarea elemental de programación.
c	Lenguaje de programación.
i	País.
j	Desarrollador o persona potencialmente desarrolladora.
$h_c(z) > 0$	Productividad humana en la tarea z del lenguaje c .
$a_c(z) \geq 0$	Productividad de ChatGPT en la tarea z .
$\mu_j \in \{0, 1\}$	Acceso individual a ChatGPT.
$y_{jc}(\mu_j)$	Producción total del individuo j en lenguaje c .
$\mathcal{Z}_c^{\text{auto}}$	Conjunto de tareas donde ChatGPT supera al humano.
τ_c	Fracción de tareas automatizables.
$s_c(z)$	Ahorro relativo de ChatGPT en la tarea z .
ρ_c	Ahorro relativo promedio en tareas automatizables.
η	Habilidad individual para programar.
r_i	Utilidad de reserva o alternativa fuera de programar.
ϕ_i	Capacidad de absorción tecnológica del país i .
θ_c	Exposición del lenguaje c a ChatGPT.
Y_{ict}	Desarrolladores activos por 100,000 habitantes.

Paso 1: función de producción a nivel de tarea

La producción del individuo j en la tarea z del lenguaje c se define como

$$y_{jc}(z; \mu_j) = \text{máx}\{h_c(z), \mu_j a_c(z)\},$$

donde $h_c(z)$ es la producción del humano en la tarea z , $a_c(z)$ es la producción de ChatGPT en dicha tarea, μ_j indica si el individuo tiene acceso a ChatGPT y el operador $\max\{\cdot, \cdot\}$ refleja que en cada tarea se escoge el método más productivo. Si $\mu_j = 0$, no hay acceso a ChatGPT y se cumple

$$\mu_j a_c(z) = 0 \cdot a_c(z) = 0,$$

de modo que la producción por tarea queda

$$y_{jc}(z; 0) = \max\{h_c(z), 0\}.$$

Como se asume $h_c(z) > 0$, el máximo entre $h_c(z)$ y 0 es $h_c(z)$; por tanto,

$$y_{jc}(z; 0) = h_c(z).$$

Si $\mu_j = 1$, sí hay acceso a ChatGPT, de modo que

$$\mu_j a_c(z) = 1 \cdot a_c(z) = a_c(z)$$

y la producción por tarea queda

$$y_{jc}(z; 1) = \max\{h_c(z), a_c(z)\}.$$

El intervalo $[0, 1]$ representa el continuo de tareas, y la producción total se obtiene integrando —es decir, sumando en el continuo— la contribución de todas las tareas:

$$y_{jc}(\mu_j) = \int_0^1 \max\{h_c(z), \mu_j a_c(z)\} dz.$$

En consecuencia, la producción sin IA es

$$y_{jc}(0) = \int_0^1 h_c(z) dz,$$

y la producción con IA es

$$y_{jc}(1) = \int_0^1 \max\{h_c(z), a_c(z)\} dz.$$

Paso 2: conjunto de tareas automatizables

Se define el conjunto de tareas automatizables del lenguaje c como

$$\mathcal{Z}_c^{\text{auto}} = \{z \in [0, 1] : a_c(z) > h_c(z)\},$$

es decir, el conjunto de tareas donde ChatGPT es estrictamente más productivo

que el humano:

$$z \in \mathcal{Z}_c^{\text{auto}} \iff a_c(z) > h_c(z).$$

Su complemento es $[0, 1] \setminus \mathcal{Z}_c^{\text{auto}}$, caracterizado por

$$z \in [0, 1] \setminus \mathcal{Z}_c^{\text{auto}} \iff z \in [0, 1] \text{ y } z \notin \mathcal{Z}_c^{\text{auto}}.$$

Si $z \notin \mathcal{Z}_c^{\text{auto}}$, no se cumple $a_c(z) > h_c(z)$ y, por tanto,

$$a_c(z) \leq h_c(z).$$

De este modo, el intervalo de tareas se particiona en dos regiones disjuntas:

$$[0, 1] = ([0, 1] \setminus \mathcal{Z}_c^{\text{auto}}) \cup \mathcal{Z}_c^{\text{auto}}, \quad ([0, 1] \setminus \mathcal{Z}_c^{\text{auto}}) \cap \mathcal{Z}_c^{\text{auto}} = \emptyset.$$

Resulta útil introducir la función indicadora,

$$\mathbf{1}[z \in A] = \begin{cases} 1, & \text{si } z \in A, \\ 0, & \text{si } z \notin A, \end{cases}$$

que satisface

$$f(z)\mathbf{1}[z \in A] = \begin{cases} f(z), & \text{si } z \in A, \\ 0, & \text{si } z \notin A, \end{cases}$$

y, al integrar,

$$\int_0^1 f(z)\mathbf{1}[z \in A] dz = \int_A f(z) dz,$$

puesto que la indicadora anula la función fuera de A . Con esta herramienta, el máximo puede reescribirse, para cada tarea z , como

$$\max\{h_c(z), a_c(z)\} = h_c(z)\mathbf{1}[z \in [0, 1] \setminus \mathcal{Z}_c^{\text{auto}}] + a_c(z)\mathbf{1}[z \in \mathcal{Z}_c^{\text{auto}}].$$

La verificación procede por casos. Si $z \in \mathcal{Z}_c^{\text{auto}}$, entonces $a_c(z) > h_c(z)$, el máximo es $a_c(z)$ y las indicadoras toman los valores

$$\mathbf{1}[z \in \mathcal{Z}_c^{\text{auto}}] = 1, \quad \mathbf{1}[z \in [0, 1] \setminus \mathcal{Z}_c^{\text{auto}}] = 0,$$

de modo que el lado derecho es

$$h_c(z) \cdot 0 + a_c(z) \cdot 1 = a_c(z).$$

Si $z \in [0, 1] \setminus \mathcal{Z}_c^{\text{auto}}$, entonces $a_c(z) \leq h_c(z)$, el máximo es $h_c(z)$ y las indicadoras toman los valores

$$\mathbf{1}[z \in \mathcal{Z}_c^{\text{auto}}] = 0, \quad \mathbf{1}[z \in [0, 1] \setminus \mathcal{Z}_c^{\text{auto}}] = 1,$$

de modo que el lado derecho es

$$h_c(z) \cdot 1 + a_c(z) \cdot 0 = h_c(z).$$

La fracción de tareas automatizables del lenguaje se define como

$$\tau_c = \int_0^1 \mathbf{1}[a_c(z) > h_c(z)] dz.$$

Como $\mathbf{1}[a_c(z) > h_c(z)]$ vale 1 dentro de $\mathcal{Z}_c^{\text{auto}}$ y 0 fuera, esto equivale a

$$\tau_c = \int_{\mathcal{Z}_c^{\text{auto}}} 1 dz,$$

de modo que τ_c mide el tamaño de $\mathcal{Z}_c^{\text{auto}}$. Dado que $\mathcal{Z}_c^{\text{auto}} \subseteq [0, 1]$, se cumple

$$0 \leq \tau_c \leq 1.$$

Por ejemplo, $\tau_c = 0,30$ significa que ChatGPT supera al humano en el 30 % de las tareas del lenguaje c .

Paso 3: ahorro relativo por tarea

Para $z \in \mathcal{Z}_c^{\text{auto}}$ se cumple

$$a_c(z) > h_c(z) > 0.$$

El ahorro relativo de ChatGPT en la tarea z se define como

$$s_c(z) = \frac{a_c(z) - h_c(z)}{h_c(z)},$$

expresión que mide la ganancia proporcional de ChatGPT respecto al humano. A modo de ilustración, si

$$h_c(z) = 10, \quad a_c(z) = 15,$$

entonces

$$s_c(z) = \frac{15 - 10}{10} = 0,5,$$

es decir, ChatGPT produce 50 % más que el humano en esa tarea. Como $a_c(z) >$

$h_c(z)$, el numerador $a_c(z) - h_c(z)$ es positivo y, dado que $h_c(z) > 0$, se cumple

$$s_c(z) > 0.$$

Multiplicando ambos lados de la definición por $h_c(z)$ se obtiene el despeje

$$h_c(z)s_c(z) = a_c(z) - h_c(z),$$

esto es,

$$a_c(z) - h_c(z) = h_c(z)s_c(z),$$

que será clave para expresar la ganancia individual. Finalmente, el ahorro promedio condicional a tareas automatizables se define como

$$\rho_c = \frac{1}{\tau_c} \int_{\mathcal{Z}_c^{\text{auto}}} s_c(z) dz, \quad \tau_c > 0,$$

donde la división entre τ_c responde a que τ_c es la medida del conjunto sobre el cual se promedia, en analogía directa con el promedio discreto (suma dividida entre cantidad).

Paso 4: ganancia individual por usar IA

La ganancia individual de acceder a ChatGPT se define como la diferencia entre la producción con IA y la producción sin IA:

$$\Delta y_{jc} = y_{jc}(1) - y_{jc}(0).$$

Para la producción con IA, se parte de

$$y_{jc}(1) = \int_0^1 \text{máx}\{h_c(z), a_c(z)\} dz$$

y se utiliza la partición

$$[0, 1] = ([0, 1] \setminus \mathcal{Z}_c^{\text{auto}}) \cup \mathcal{Z}_c^{\text{auto}},$$

con lo cual la integral se separa en dos términos:

$$y_{jc}(1) = \int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} \text{máx}\{h_c(z), a_c(z)\} dz + \int_{\mathcal{Z}_c^{\text{auto}}} \text{máx}\{h_c(z), a_c(z)\} dz.$$

En $[0, 1] \setminus \mathcal{Z}_c^{\text{auto}}$, ChatGPT no supera al humano, es decir, $a_c(z) \leq h_c(z)$; por tanto,

$$\text{máx}\{h_c(z), a_c(z)\} = h_c(z).$$

En $\mathcal{Z}_c^{\text{auto}}$, ChatGPT sí supera al humano, es decir, $a_c(z) > h_c(z)$; por tanto,

$$\text{máx}\{h_c(z), a_c(z)\} = a_c(z).$$

Sustituyendo ambos resultados:

$$y_{jc}(1) = \int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c(z) dz + \int_{\mathcal{Z}_c^{\text{auto}}} a_c(z) dz.$$

Para la producción sin IA, se parte de

$$y_{jc}(0) = \int_0^1 h_c(z) dz$$

y se particiona el intervalo en las dos mismas regiones:

$$y_{jc}(0) = \int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c(z) dz + \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) dz.$$

Restando ambas expresiones:

$$\Delta y_{jc} = \left[\int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c(z) dz + \int_{\mathcal{Z}_c^{\text{auto}}} a_c(z) dz \right] - \left[\int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c(z) dz + \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) dz \right].$$

Distribuyendo el signo negativo:

$$\Delta y_{jc} = \int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c(z) dz + \int_{\mathcal{Z}_c^{\text{auto}}} a_c(z) dz - \int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c(z) dz - \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) dz.$$

Los términos sobre tareas no automatizables se cancelan:

$$\int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c(z) dz - \int_{[0,1] \setminus \mathcal{Z}_c^{\text{auto}}} h_c(z) dz = 0,$$

de modo que queda

$$\Delta y_{jc} = \int_{\mathcal{Z}_c^{\text{auto}}} a_c(z) dz - \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) dz.$$

Como ambas integrales se toman sobre el mismo conjunto, pueden unirse:

$$\Delta y_{jc} = \int_{\mathcal{Z}_c^{\text{auto}}} [a_c(z) - h_c(z)] dz.$$

Usando el despeje del Paso 3, $a_c(z) - h_c(z) = h_c(z)s_c(z)$, se obtiene

$$\Delta y_{jc} = \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z)s_c(z) dz.$$

Conviene subrayar que hasta este punto no se ha impuesto ningún supuesto no trivial: la ganancia individual exacta es la suma, sobre las tareas automatizables, de la productividad humana base multiplicada por el ahorro relativo de ChatGPT.

Paso 4.5: identidad de covarianza

El objetivo de este paso es convertir la expresión exacta

$$\Delta y_{jc} = \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) s_c(z) dz$$

en una expresión en términos de promedios. Para cualquier función $g(z)$, el promedio (esperanza condicional) dentro de $\mathcal{Z}_c^{\text{auto}}$ se define como

$$\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[g(z)] = \frac{1}{\tau_c} \int_{\mathcal{Z}_c^{\text{auto}}} g(z) dz,$$

lo cual implica

$$\int_{\mathcal{Z}_c^{\text{auto}}} g(z) dz = \tau_c \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[g(z)].$$

Tomando $g(z) = h_c(z) s_c(z)$,

$$\int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) s_c(z) dz = \tau_c \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c s_c],$$

y, como la integral de la izquierda es Δy_{jc} ,

$$\Delta y_{jc} = \tau_c \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c s_c].$$

El promedio de productividad humana dentro de tareas automatizables es

$$\bar{h}_c = \frac{1}{\tau_c} \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) dz = \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c],$$

y el promedio de ahorro relativo dentro de tareas automatizables es

$$\rho_c = \frac{1}{\tau_c} \int_{\mathcal{Z}_c^{\text{auto}}} s_c(z) dz = \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c].$$

La covarianza condicional dentro de $\mathcal{Z}_c^{\text{auto}}$ se define como

$$\text{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c) = \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}} [(h_c - \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c]) (s_c - \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c])].$$

Expandiendo el producto:

$$(h_c - \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c]) (s_c - \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c]) = h_c s_c - h_c \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c] - s_c \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] + \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c].$$

Tomando esperanza condicional:

$$\begin{aligned}\mathbf{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c) &= \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c s_c] - \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c] \\ &\quad - \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] + \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c]].\end{aligned}$$

Los términos $\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c]$ y $\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c]$ son constantes y, por tanto, pueden extraerse de la esperanza:

$$\begin{aligned}\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c]] &= \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c], \\ \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c]] &= \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c], \\ \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c]] &= \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c].\end{aligned}$$

Entonces:

$$\begin{aligned}\mathbf{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c) &= \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c s_c] - \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c] \\ &\quad - \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c] + \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c] \\ &= \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c s_c] - \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c].\end{aligned}$$

Reordenando:

$$\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c s_c] = \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c] + \mathbf{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c).$$

Como

$$\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c] = \bar{h}_c, \quad \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[s_c] = \rho_c,$$

se obtiene

$$\mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c s_c] = \bar{h}_c \rho_c + \mathbf{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c).$$

Sustituyendo esta identidad en $\Delta y_{jc} = \tau_c \mathbb{E}_{\mathcal{Z}_c^{\text{auto}}}[h_c s_c]$ resulta la forma exacta con covarianza:

$$\Delta y_{jc} = \tau_c [\bar{h}_c \rho_c + \mathbf{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c)],$$

donde el factor τ_c multiplica tanto al producto de promedios como a la covarianza, porque esta última está definida como promedio condicional dentro de $\mathcal{Z}_c^{\text{auto}}$. La covarianza mide si la productividad humana $h_c(z)$ y el ahorro relativo $s_c(z)$ se mueven juntos dentro de las tareas automatizables: si $\mathbf{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c) > 0$, ChatGPT me-

jora relativamente más las tareas donde la productividad humana base era alta; si $\text{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c) < 0$, mejora relativamente más las tareas donde dicha productividad era baja; y si $\text{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c) = 0$, no existe relación sistemática entre ambas dentro de las tareas automatizables. Para simplificar, el modelo impone como *supuesto no trivial* la covarianza nula, $\text{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c) = 0$, con lo cual

$$\Delta y_{jc} = \tau_c \bar{h}_c \rho_c.$$

Paso 4.6: normalización de escala

El modelo elige la unidad de producción de forma que

$$\bar{h}_c = 1.$$

Se trata de una normalización: no significa que, empíricamente, la productividad humana promedio sea literalmente 1 en unidades naturales, sino que la producción se mide en unidades en las cuales ese promedio vale 1. Partiendo de

$$\Delta y_{jc} = \tau_c \bar{h}_c \rho_c$$

y sustituyendo $\bar{h}_c = 1$:

$$\Delta y_{jc} = \tau_c (1) \rho_c.$$

Por tanto,

$$\Delta y_{jc} = \tau_c \rho_c.$$

Se define entonces la exposición del lenguaje como

$$\theta_c = \tau_c \rho_c,$$

con lo cual

$$\Delta y_{jc} = \theta_c.$$

La exposición del lenguaje combina, así, dos componentes: cuántas tareas del lenguaje son automatizables (τ_c) y cuán grande es la mejora promedio en esas tareas (ρ_c).

Paso 5: decisión individual de entrada al mercado

Este paso traslada el análisis de la productividad a la decisión ocupacional. La utilidad de programar se define como

$$U_{\text{prog}} = \eta + y_{jc}(\mu_j),$$

donde η es la habilidad individual —talento, experiencia o componente personal que aumenta la utilidad o productividad de programar— y $y_{jc}(\mu_j)$ es la producción asociada a programar en el lenguaje c , con o sin ChatGPT. La utilidad fuera de programar es

$$U_{\text{out}} = r_i,$$

donde r_i es la utilidad de reserva en el país i , que puede representar el salario alternativo, las oportunidades fuera de la programación o el costo de oportunidad de otra actividad. La persona entra a programar si

$$U_{\text{prog}} \geq U_{\text{out}}.$$

Sustituyendo:

$$\eta + y_{jc}(\mu_j) \geq r_i.$$

Restando $y_{jc}(\mu_j)$ a ambos lados se despeja η :

$$\eta \geq r_i - y_{jc}(\mu_j).$$

El lado derecho constituye el umbral mínimo de habilidad para entrar:

$$\eta_c^*(\mu) = r_i - y_{jc}(\mu),$$

de modo que la regla de entrada es

$$\text{entra si } \eta \geq \eta_c^*(\mu).$$

Comparando ambos regímenes, el umbral sin IA es

$$\eta_c^*(0) = r_i - y_{jc}(0)$$

y el umbral con IA es

$$\eta_c^*(1) = r_i - y_{jc}(1).$$

Dado que

$$\Delta y_{jc} = y_{jc}(1) - y_{jc}(0),$$

se tiene

$$y_{jc}(1) = y_{jc}(0) + \Delta y_{jc}.$$

Sustituyendo en el umbral con IA:

$$\eta_c^*(1) = r_i - [y_{jc}(0) + \Delta y_{jc}].$$

Distribuyendo el signo negativo:

$$\eta_c^*(1) = r_i - y_{jc}(0) - \Delta y_{jc}.$$

Pero

$$r_i - y_{jc}(0) = \eta_c^*(0);$$

por tanto,

$$\eta_c^*(1) = \eta_c^*(0) - \Delta y_{jc}.$$

Usando la versión simplificada $\Delta y_{jc} = \tau_c \rho_c$, se obtiene

$$\eta_c^*(1) = \eta_c^*(0) - \tau_c \rho_c,$$

y usando la versión exacta con covarianza,

$$\eta_c^*(1) = \eta_c^*(0) - \tau_c [\bar{h}_c \rho_c + \text{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c)].$$

En consecuencia, el acceso a la IA reduce el umbral de entrada: las personas con habilidad comprendida entre el nuevo umbral y el antiguo se convierten en programadores marginales que antes no entraban al mercado.

Paso 6: agregación a desarrolladores activos

Para pasar del individuo a la población, sea N_i la población del país i y sea $F_i(\eta)$ la función de distribución acumulada de la habilidad en dicho país:

$$F_i(x) = \Pr(\eta \leq x),$$

es decir, $F_i(x)$ es la proporción de personas cuya habilidad es menor o igual que x . En consecuencia,

$$1 - F_i(x) = \Pr(\eta > x).$$

Como una persona entra si

$$\eta \geq \eta_c^*(\mu),$$

la proporción de personas que entra es aproximadamente

$$1 - F_i(\eta_c^*(\mu)),$$

y, multiplicando por la población, el número de desarrolladores activos es

$$\text{Devs}_{ic}(\mu) = N_i [1 - F_i(\eta_c^*(\mu))].$$

Con IA:

$$\text{Devs}_{ic}(1) = N_i [1 - F_i(\eta_c^*(1))].$$

Sin IA:

$$\text{Devs}_{ic}(0) = N_i [1 - F_i(\eta_c^*(0))].$$

Restando:

$$\begin{aligned} \text{Devs}_{ic}(1) - \text{Devs}_{ic}(0) &= N_i [1 - F_i(\eta_c^*(1))] - N_i [1 - F_i(\eta_c^*(0))] \\ &= N_i ([1 - F_i(\eta_c^*(1))] - [1 - F_i(\eta_c^*(0))]) \\ &= N_i [1 - F_i(\eta_c^*(1)) - 1 + F_i(\eta_c^*(0))] \\ &= N_i [F_i(\eta_c^*(0)) - F_i(\eta_c^*(1))]. \end{aligned}$$

Como

$$\eta_c^*(1) = \eta_c^*(0) - \tau_c \rho_c,$$

se define el desplazamiento del umbral

$$\Delta\eta = \tau_c \rho_c,$$

de modo que

$$\eta_c^*(1) = \eta_c^*(0) - \Delta\eta.$$

Sustituyendo:

$$\text{Devs}_{ic}(1) - \text{Devs}_{ic}(0) = N_i [F_i(\eta_c^*(0)) - F_i(\eta_c^*(0) - \Delta\eta)].$$

Para evaluar esta diferencia se emplea una aproximación de Taylor de primer

orden. Sea

$$x = \eta_c^*(0);$$

se desea aproximar

$$F_i(x) - F_i(x - \Delta\eta).$$

La expansión de Taylor de primer orden de F_i alrededor de x es

$$F_i(x + \Delta) \approx F_i(x) + F_i'(x)\Delta.$$

Como se requiere evaluar en $x - \Delta\eta$, se toma $\Delta = -\Delta\eta$:

$$F_i(x - \Delta\eta) \approx F_i(x) + F_i'(x)(-\Delta\eta),$$

es decir,

$$F_i(x - \Delta\eta) \approx F_i(x) - F_i'(x)\Delta\eta.$$

La densidad f_i es la derivada de la función de distribución acumulada,

$$f_i(x) = F_i'(x),$$

por lo que

$$F_i(x - \Delta\eta) \approx F_i(x) - f_i(x)\Delta\eta.$$

Entonces:

$$\begin{aligned} F_i(x) - F_i(x - \Delta\eta) &\approx F_i(x) - [F_i(x) - f_i(x)\Delta\eta] \\ &= F_i(x) - F_i(x) + f_i(x)\Delta\eta \\ &= f_i(x)\Delta\eta. \end{aligned}$$

Volviendo a $x = \eta_c^*(0)$:

$$F_i(\eta_c^*(0)) - F_i(\eta_c^*(0) - \Delta\eta) \approx f_i(\eta_c^*(0))\Delta\eta.$$

Como $\Delta\eta = \tau_c \rho_c$:

$$F_i(\eta_c^*(0)) - F_i(\eta_c^*(0) - \Delta\eta) \approx f_i(\eta_c^*(0))\tau_c \rho_c.$$

Sustituyendo en la diferencia de desarrolladores:

$$\text{Devs}_{ic}(1) - \text{Devs}_{ic}(0) \approx N_i f_i(\eta_c^*(0)) \tau_c \rho_c.$$

Esta aproximación constituye un *supuesto no trivial*: es válida si $\Delta\eta = \tau_c \rho_c$ es pequeño respecto a la escala en la que cambia la densidad f_i ; si el cambio de umbral fuera grande, los términos de segundo orden podrían ser relevantes. El término $f_i(\eta_c^*(0))$ mide cuánta gente se encuentra cerca del umbral inicial: si hay mucha masa justo debajo del umbral, una pequeña reducción de este genera muchos nuevos desarrolladores.

Paso 7: resultado per cápita, absorción y separabilidad

El resultado anterior está expresado en número total de desarrolladores:

$$\text{Devs}_{ic}(1) - \text{Devs}_{ic}(0) \approx N_i f_i(\eta_c^*(0)) \tau_c \rho_c.$$

Sin embargo, el resultado del estudio se expresa como desarrolladores por cada 100,000 habitantes, por lo que se define

$$\Delta Y_{ic} = \frac{\text{Devs}_{ic}(1) - \text{Devs}_{ic}(0)}{N_i} \cdot 10^5,$$

donde $10^5 = 100,000$: el cambio total de desarrolladores se divide entre la población y se multiplica por 100,000. Sustituyendo la aproximación anterior:

$$\Delta Y_{ic} \approx \frac{N_i f_i(\eta_c^*(0)) \tau_c \rho_c}{N_i} \cdot 10^5.$$

Cancelando N_i :

$$\Delta Y_{ic} \approx f_i(\eta_c^*(0)) \tau_c \rho_c \cdot 10^5,$$

y reordenando:

$$\Delta Y_{ic} \approx 10^5 f_i(\eta_c^*(0)) \tau_c \rho_c.$$

Esta expresión se factoriza en dos partes naturales. La primera,

$$10^5 f_i(\eta_c^*(0)),$$

depende del país, porque f_i es la distribución de habilidad del país i , y mide cuánta gente hay cerca del umbral de entrada; se interpreta como capacidad de absorción. La segunda,

$$\tau_c \rho_c,$$

depende del lenguaje c y mide cuántas tareas del lenguaje son automatizables y cuánto mejora ChatGPT en promedio. Por eso se define

$$\theta_c = \tau_c \rho_c$$

y, bajo un supuesto de separabilidad,

$$\phi_i = 10^5 f_i(\eta_c^*(0)),$$

con lo cual el aumento per cápita se escribe como el producto entre la exposición del lenguaje y la absorción del país:

$$\Delta Y_{ic} \approx \phi_i \theta_c.$$

Conviene precisar por qué la separabilidad es un supuesto. El umbral inicial era

$$\eta_c^*(0) = r_i - y_{jc}(0),$$

donde r_i depende del país i y $y_{jc}(0)$ depende del lenguaje c , porque la productividad base puede diferir entre lenguajes. Por tanto, en rigor,

$$f_i(\eta_c^*(0))$$

también puede depender de c : si el umbral cambia por lenguaje, la densidad evaluada en ese umbral también cambia por lenguaje. La forma más general sería

$$\phi_{ic} = 10^5 f_i(\eta_c^*(0)),$$

con lo cual

$$\Delta Y_{ic} \approx \phi_{ic} \theta_c.$$

Para escribir

$$\Delta Y_{ic} \approx \phi_i \theta_c$$

se necesita asumir que

$$\phi_{ic} \approx \phi_i,$$

es decir, que la capacidad de absorción depende principalmente del país y no varía sustancialmente entre lenguajes. Una manera de justificarlo es asumir que las productividades base $y_{jc}(0)$ son similares entre lenguajes, de modo que los umbrales $\eta_c^*(0)$ no se mueven demasiado; otra es asumir que la densidad f_i es relativamente

plana alrededor de esos umbrales, de modo que evaluarla en umbrales ligeramente distintos arroja valores parecidos. La separabilidad país–lenguaje constituye, así, un *supuesto no trivial*: sin él, el efecto sigue pudiendo interpretarse por lenguaje, pero la absorción promedio podría depender también del lenguaje.

Paso 8: especificación de panel

Finalmente, el resultado teórico se traslada a una ecuación empírica. La predicción teórica era

$$\Delta Y_{ic} \approx \phi_i \theta_c,$$

pero ese cambio solo ocurre cuando ChatGPT está disponible. Sea

$$A_{it} \in \{0, 1\}$$

un indicador de disponibilidad de ChatGPT en el país i y el periodo t . El componente de tratamiento es entonces

$$\theta_c \phi_i A_{it}.$$

Si $A_{it} = 0$, no hay tratamiento:

$$\theta_c \phi_i A_{it} = 0.$$

Si $A_{it} = 1$, el efecto esperado es

$$\theta_c \phi_i A_{it} = \theta_c \phi_i.$$

La línea base sin tratamiento puede descomponerse como

$$Y_{ic}(0) = \alpha_i + \beta_t + \delta_c + \eta_{ict},$$

donde α_i recoge las diferencias permanentes entre países, β_t los *shocks* comunes de tiempo, δ_c las diferencias permanentes entre lenguajes y η_{ict} los factores no observados restantes. Sumando el componente de tratamiento se obtiene la especificación de panel:

$$Y_{ict} = \alpha_i + \beta_t + \delta_c + \theta_c \phi_i A_{it} + \eta_{ict}.$$

Para interpretar causalmente el coeficiente de tratamiento se requiere la condición de exogeneidad condicional

$$\mathbb{E}[\eta_{ict} \mid \alpha_i, \beta_t, \delta_c, A_{it}] = 0,$$

es decir, después de controlar por los efectos fijos de país, tiempo y lenguaje, la disponibilidad de ChatGPT no debe estar correlacionada con *shocks* no observados que también afecten Y_{ict} . Si esta condición falla, parte del cambio atribuido a ChatGPT podría provenir de otros factores.

Resumen de la derivación completa

Paso	Ecuación clave	Tipo
1	$y_{jc}(\mu_j) = \int_0^1 \text{máx}\{h_c(z), \mu_j a_c(z)\} dz$	Definición
2	$\mathcal{Z}_c^{\text{auto}} = \{z : a_c(z) > h_c(z)\}, \tau_c = \int_{\mathcal{Z}_c^{\text{auto}}} 1 dz$	Definición exacta
3	$s_c(z) = \frac{a_c(z) - h_c(z)}{h_c(z)}, \rho_c = \frac{1}{\tau_c} \int_{\mathcal{Z}_c^{\text{auto}}} s_c(z) dz$	Definición exacta
4	$\Delta y_{jc} = \int_{\mathcal{Z}_c^{\text{auto}}} h_c(z) s_c(z) dz$	Identidad exacta
4.5	$\Delta y_{jc} = \tau_c [\bar{h}_c \rho_c + \text{Cov}_{\mathcal{Z}_c^{\text{auto}}}(h_c, s_c)]$	Identidad de covarianza
4.6	$\Delta y_{jc} = \tau_c \rho_c = \theta_c$	Covarianza cero + normalización
5	$\eta_c^*(1) = \eta_c^*(0) - \tau_c \rho_c$	Decisión de entrada
6	$\text{Devs}_{ic}(1) - \text{Devs}_{ic}(0) \approx N_i f_i(\eta_c^*(0)) \tau_c \rho_c$	Taylor de primer orden
7	$\Delta Y_{ic} \approx \phi_i \theta_c$	Per cápita + separabilidad
8	$Y_{ict} = \alpha_i + \beta_t + \delta_c + \theta_c \phi_i A_{it} + \eta_{ict}$	Ecuación de panel

Ejemplo numérico ilustrativo

Supóngase que en un lenguaje

$$\tau_c = 0,30, \quad \rho_c = 0,50,$$

es decir, ChatGPT supera al humano en el 30 % de las tareas y, en esas tareas, la mejora relativa promedio es de 50 %. Entonces:

$$\theta_c = \tau_c \rho_c = 0,30 \times 0,50 = 0,15,$$

y la ganancia individual simplificada es

$$\Delta y_{jc} = 0,15.$$

Si el umbral inicial era

$$\eta_c^*(0) = 2,00,$$

el nuevo umbral es

$$\eta_c^*(1) = 2,00 - 0,15 = 1,85.$$

Las personas con habilidad

$$1,85 \leq \eta < 2,00$$

no entraban antes, pero entran después. Si el país tiene

$$N_i = 10,000,000$$

habitantes y la densidad alrededor del umbral inicial es

$$f_i(\eta_c^*(0)) = 0,08,$$

entonces

$$\text{Devs}_{ic}(1) - \text{Devs}_{ic}(0) \approx 10,000,000 \times 0,08 \times 0,15 = 120,000.$$

Por cada 100,000 habitantes:

$$\Delta Y_{ic} = \frac{120,000}{10,000,000} \times 100,000 = 1,200.$$

El ejemplo es puramente ilustrativo: permite observar cómo interactúan los componentes del modelo, no afirmar magnitudes reales.

Supuestos del modelo

La derivación descansa en los siguientes supuestos:

1. **Elección del mejor método por tarea.** El modelo asume que el desarrollador puede usar el método más productivo en cada tarea.
2. **Covarianza cero.** Para obtener $\Delta y_{jc} = \tau_c \rho_c \bar{h}_c$, se impone $\text{Cov}_{\mathcal{Z}^{\text{auto}}}(h_c, s_c) = 0$.
3. **Normalización de escala.** Se elige $\bar{h}_c = 1$, lo que permite $\Delta y_{jc} = \tau_c \rho_c$.
4. **Taylor de primer orden.** Se aproxima el cambio en desarrolladores usando la densidad en el umbral inicial.
5. **Separabilidad país–lenguaje.** Se reemplaza ϕ_{ic} por ϕ_i , asumiendo que la absorción depende principalmente del país.
6. **Exogeneidad condicional.** Para la interpretación causal en el panel, se requiere que la disponibilidad de ChatGPT no esté correlacionada con *shocks* no observados después de controlar por efectos fijos.

Lectura económica del modelo

En síntesis, la cadena causal del modelo puede leerse en tres eslabones:

$$\underbrace{\text{más tareas automatizables}}_{\tau_c} \times \underbrace{\text{mayor ahorro promedio}}_{\rho_c} \implies \underbrace{\text{mayor ganancia individual}}_{\Delta y_{jc}}.$$

Luego:

$$\underbrace{\text{mayor ganancia individual}}_{\Delta y_{jc}} \implies \underbrace{\text{menor umbral de entrada}}_{\eta_c^*(1) < \eta_c^*(0)}.$$

Luego:

$$\underbrace{\text{menor umbral}}_{\text{baja en } \eta_c^*} \times \underbrace{\text{gente cerca del umbral}}_{f_i(\eta_c^*(0))} \implies \underbrace{\text{más desarrolladores activos}}_{\text{Devs}_{ic}(1) - \text{Devs}_{ic}(0)}.$$

Finalmente:

$$\Delta Y_{ic} \approx \phi_i \theta_c.$$

En una frase: ChatGPT tiene mayor impacto donde los lenguajes están más expuestos a la automatización y donde los países tienen más personas cerca del umbral de entrada a la programación.